

Project: Simple RAG System

Dương Trường Bình

Nguyễn Anh Khôi

Trần Đại Nhân

Dương Đình Thắng

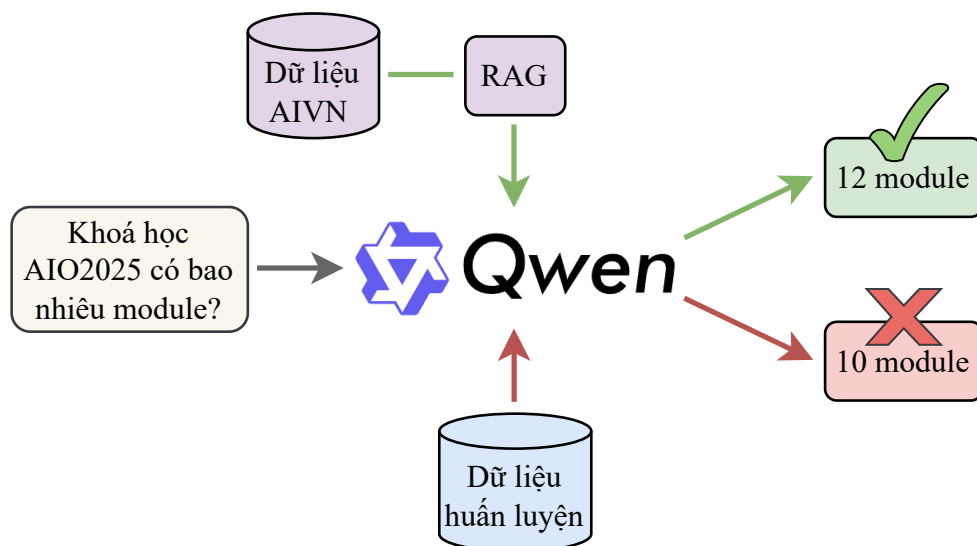
Đinh Quang Vinh

I. Dẫn nhập

Sự bùng nổ của các Large Language Models (LLMs) như ChatGPT, Gemini, Claude hay Llama đã định hình lại lĩnh vực NLP. Tuy nhiên, dù sở hữu khả năng tổng quát và suy luận ấn tượng, các mô hình này vẫn đối mặt với những hạn chế cố hữu: tri thức bị giới hạn tại thời điểm huấn luyện, hiện tượng ảo giác khi gặp các câu hỏi nằm ngoài vùng tri thức, và đặc biệt là sự thiếu hụt kiến thức về dữ liệu riêng tư của doanh nghiệp.

Để giải quyết vấn đề này, kỹ thuật Retrieval Augmented Generation (RAG) đã ra đời. RAG cho phép LLMs tiếp cận nguồn dữ liệu bên ngoài mà không cần trải qua quá trình fine-tuning tốn kém hay phải huấn luyện lại. Để giới thiệu về RAG, bài viết này đi sâu vào kiến trúc và cách triển khai, bao gồm:

1. Phân tích sâu khái niệm, kiến trúc, pipeline cơ bản của RAG.
2. Giới thiệu framework LangChain, công cụ mạnh mẽ cho các ứng dụng sử dụng LLM.
3. Xây dựng hệ thống QA System trên tài liệu PDF học thuật.



Hình 1: Minh hoạ LLM khi có (đường xanh lá) và không sử dụng RAG (đường màu đỏ).

Mục lục

I.	Dẫn nhập	1
II.	Cơ sở lý thuyết về RAG	4
II.1.	Sự chuyển dịch sang In-Context RAG	5
II.2.	Kiến trúc RAG hiện đại	5
II.3.	Mở rộng: Phân tích vai trò của các thành phần	15
III.	Thư viện LangChain	16
III.1.	Giới thiệu	16
III.2.	Các thành phần cốt lõi	17
IV.	Thực hành	21
IV.1.	Chuẩn bị môi trường và cấu trúc dự án	21
IV.2.	Chuẩn bị dữ liệu PDF đầu vào	23
IV.3.	Tiền xử lý văn bản và Chunking	24
IV.4.	Xây dựng Vector Database	25
IV.5.	Khởi tạo LLM và xây dựng RAG Chain	26
IV.6.	Xây dựng giao diện ứng dụng	29
V.	Câu hỏi trắc nghiệm	31
	Phụ lục	35

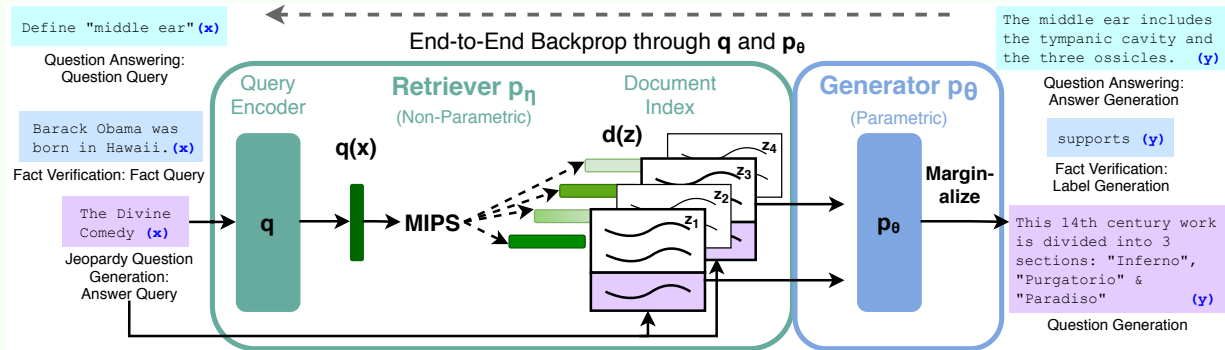
Bảng thuật ngữ

Thuật ngữ	Mô tả
Hallucination	Hiện tượng mô hình sinh ra thông tin sai lệch, bịa đặt hoặc không có thực nhưng với văn phong tự tin.
Knowledge Cutoff	Mốc thời gian giới hạn của dữ liệu huấn luyện, khiến mô hình không biết các sự kiện xảy ra sau đó.
Fine-tuning	Quá trình huấn luyện tiếp mô hình đã pre-trained trên tập dữ liệu chuyên biệt để cập nhật trọng số.
In-Context Learning	Khả năng LLM học và thực hiện tác vụ dựa trên ngữ cảnh hoặc ví dụ được cung cấp trong prompt mà không cần cập nhật tham số.
Vector Embeddings	Biểu diễn dữ liệu (text, image) dưới dạng vector số thực trong không gian n-chiều.
Semantic Search	Tìm kiếm dựa trên sự tương đồng về ý nghĩa thay vì chỉ khớp từ khóa.
Chunking	Kỹ thuật chia nhỏ văn bản dài thành các đoạn ngắn để tối ưu hóa việc mã hóa và phù hợp với giới hạn Context Window.
Context Window	Giới hạn số lượng tokens (đơn vị văn bản) tối đa mà LLM có thể tiếp nhận và xử lý trong một lần prompt.
Grounding	Kỹ thuật “neo” câu trả lời của mô hình vào dữ liệu thực tế được cung cấp để đảm bảo tính xác thực.

II. Cơ sở lý thuyết về RAG

i Nguồn gốc

Khái niệm RAG lần đầu tiên được đề xuất chính thức trong bài báo khoa học “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” bởi Patrick Lewis và các cộng sự tại Facebook AI Research (FAIR) vào năm 2020^a.



Hình 2: Tổng quan kiến trúc RAG trong bài báo gốc của Patrick Lewis (2020).

Trong công trình này, nhóm tác giả định nghĩa RAG là một mô hình xác suất lai (hybrid probabilistic model) kết hợp giữa hai dạng bộ nhớ nhằm khắc phục nhược điểm của các mô hình Pre-trained Seq2Seq truyền thống:

- **Parametric Memory (Bộ nhớ tham số):** Là tri thức ẩn được lưu trữ trong trọng số của một mô hình sinh chuỗi (Pre-trained Seq2Seq Transformer). Cụ thể trong bài báo, tác giả sử dụng mô hình BART (Bidirectional and Auto-Regressive Transformers) làm Generator.
- **Non-Parametric Memory (Bộ nhớ phi tham số):** Là tri thức tường minh từ bên ngoài, cụ thể là một dense vector index chứa các đoạn văn bản Wikipedia. Thành phần này được truy cập thông qua một bộ Neural Retriever dựa trên kiến trúc Dense Passage Retriever.

Cơ chế hoạt động của RAG gốc cho phép Generator (BART) sử dụng đầu vào kết hợp với các tài liệu ẩn (latent documents) tìm được từ Retriever để sinh văn bản. Điểm đặc biệt là toàn bộ kiến trúc này được fine-tuning end-to-end, cho phép cập nhật trọng số của cả Query Encoder và Generator để tối ưu hóa tác vụ đích.

^aXem chi tiết tại: <https://arxiv.org/abs/2005.11401>

II.1. Sự chuyển dịch sang In-Context RAG

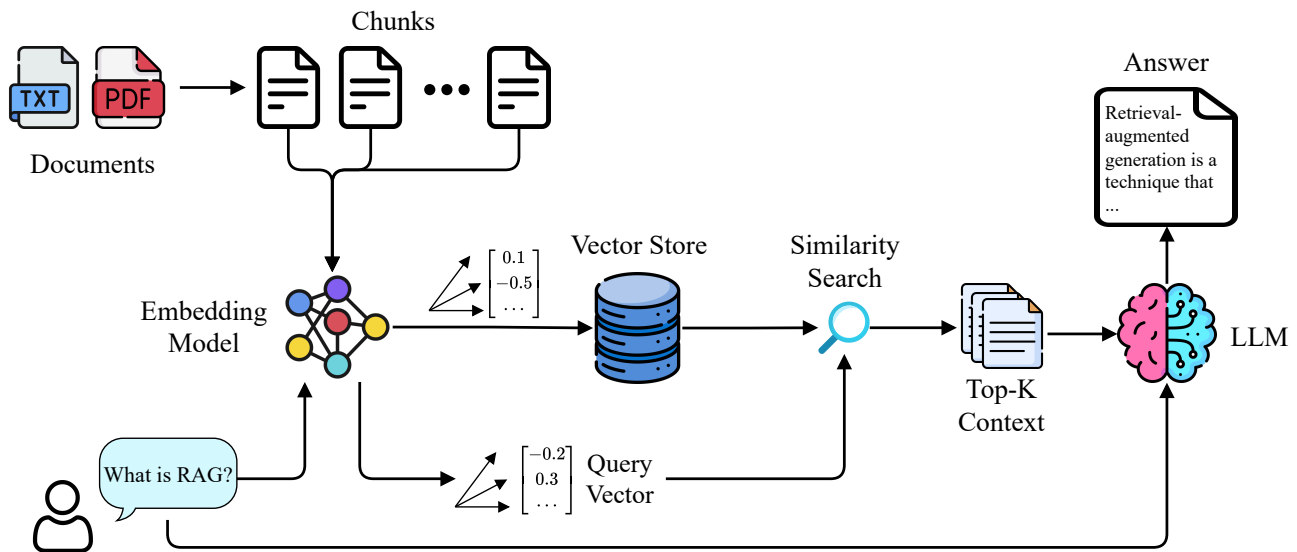
Mặc dù thuật ngữ RAG vẫn được giữ nguyên, tư duy triển khai kỹ thuật này đã thay đổi căn bản cùng với sự phát triển của các LLM:

- **Original RAG (2020):** Là hướng tiếp cận dựa trên fine-tuning. Như đã đề cập ở phần trên, mô hình gốc yêu cầu huấn luyện đồng thời cả bộ truy xuất và mô hình sinh văn bản để chúng học cách phối hợp với nhau. Trọng số của mô hình thay đổi trong quá trình này.
- **Modern RAG (Hiện nay):** Là hướng tiếp cận dựa trên In-Context Learning. Với sự bùng nổ của các LLM cực lớn có khả năng hiểu ngữ cảnh rộng, RAG hiện đại thường ám chỉ quy trình “Retrieve and Prompt”.

Trong mô hình hiện đại, chúng ta thường giữ nguyên trọng số của LLM và chỉ tập trung tối ưu hóa việc truy xuất dữ liệu, sau đó đưa dữ liệu này vào đầu vào (Prompt) để mô hình xử lý. Cách tiếp cận này linh hoạt, chi phí thấp và dễ dàng áp dụng cho dữ liệu riêng tư mà không cần quy trình huấn luyện phức tạp.

II.2. Kiến trúc RAG hiện đại

Một hệ thống RAG tiêu chuẩn hiện nay thường được mô hình hóa thành một quy trình gồm 3 giai đoạn chính: Indexing, Retrieval và Generation.



Hình 3: Sơ đồ luồng hoạt động cơ bản của RAG.

II.2.1. Phase 1: Indexing

Giai đoạn này tương đồng với quy trình ETL (Extract-Transform-Load) trong kỹ thuật dữ liệu. Mục tiêu của giai đoạn này là chuyển đổi dữ liệu thô từ nhiều định dạng khác nhau thành một định dạng thống nhất để hệ thống có thể tìm kiếm được.

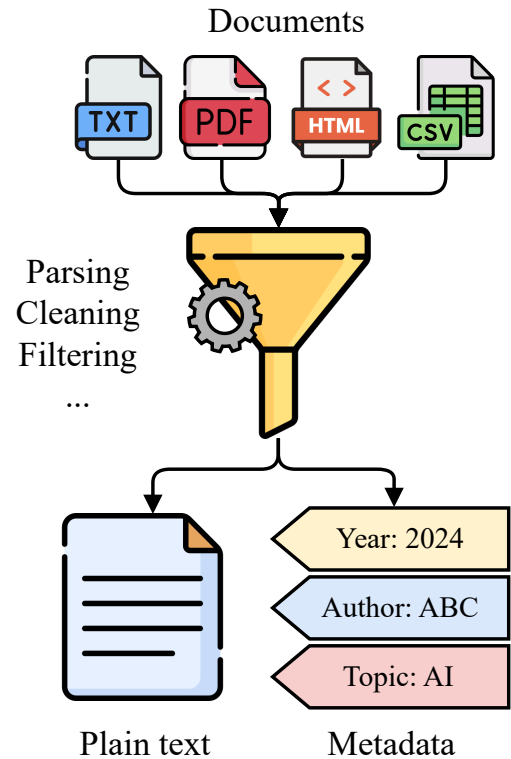
Quy trình này bao gồm các bước từ cơ bản đến nâng cao như sau:

1. Document Loading:

Bắt đầu với việc thu thập nguồn dữ liệu đầu vào, có thể là dữ liệu đã có sẵn trong nội bộ doanh nghiệp hoặc dữ liệu được cào từ internet.

- **Trích xuất nội dung:** Hệ thống cần có khả năng xử lý đa dạng các loại tệp tin để có thể loại bỏ các định dạng hiển thị phức tạp như font chữ, màu sắc, layout và giữ lại phần quan trọng nhất là văn bản thuần túy.
- **Thu thập siêu dữ liệu:** Trong các ứng dụng thực tế, chỉ lấy nội dung văn bản là chưa đủ. Hệ thống RAG tối ưu cần trích xuất kèm cả các thông tin ngữ cảnh đi kèm tài liệu, hay còn gọi là metadata, ví dụ: chủ đề, số trang, ngày xuất bản, tác giả, ...

Vai trò của metadata là cực kỳ quan trọng để hỗ trợ tính năng Pre-filtering. Ví dụ như nếu người dùng hỏi về “Doanh thu năm 2024”, hệ thống sẽ dùng metadata để lọc đúng các tài liệu trong năm 2024 thay vì tìm kiếm toàn bộ dữ liệu.

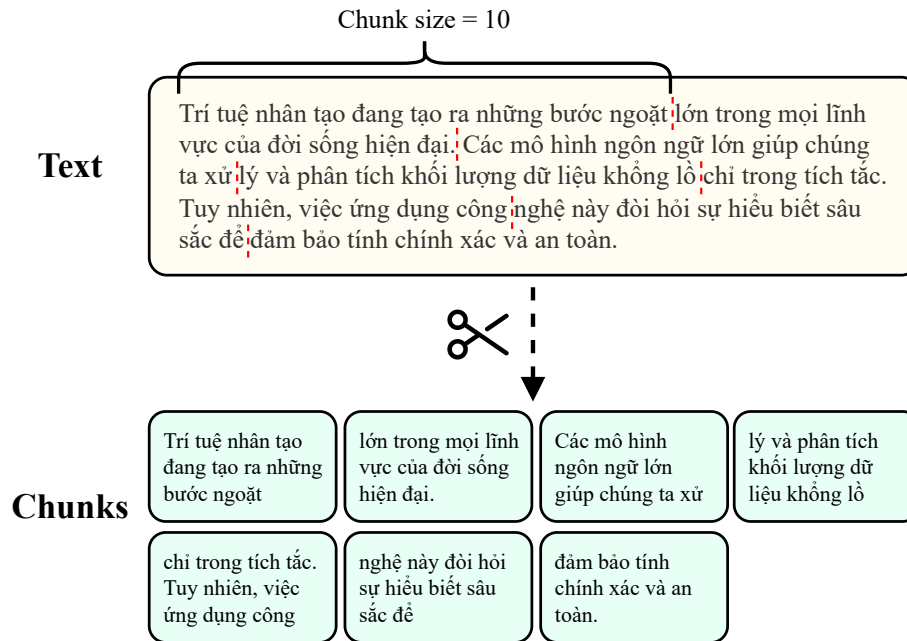


Hình 4: Minh họa quy trình rút trích văn bản từ các nguồn dữ liệu.

2. **Text Splitting (Chunking):** Đây là bước chia nhỏ tài liệu dài thành các đoạn nhỏ hơn gọi là “chunks”.

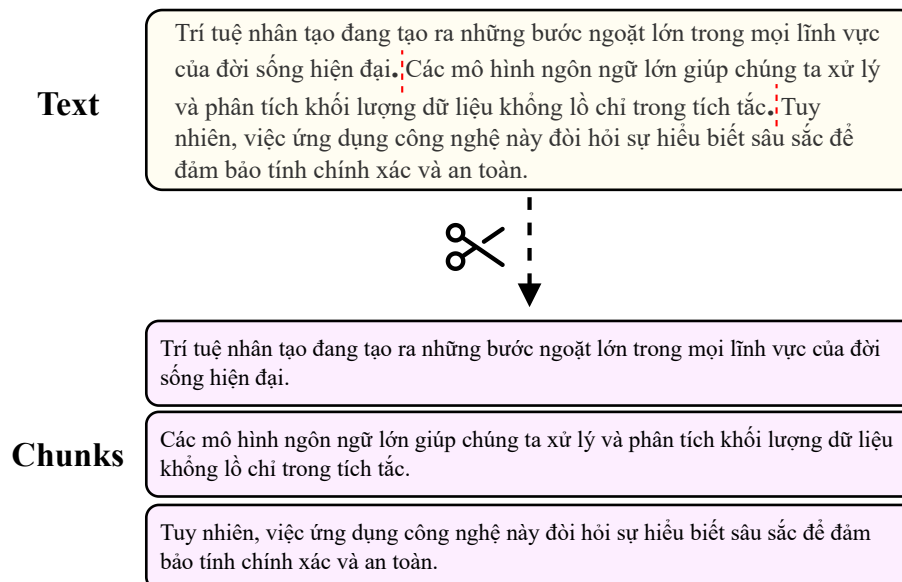
• Tại sao cần Chunking?

- (a) Giới hạn Context Window: Các mô hình LLM và Embedding đều có giới hạn về số lượng token đầu vào. Không thể đưa toàn bộ nội dung vào mô hình cùng lúc.
- (b) Độ chính xác của tìm kiếm: Vector của một đoạn văn ngắn tập trung vào một ý tưởng cụ thể sẽ đại diện cho ngữ nghĩa tốt hơn so với vector trung bình cộng của cả một trang giấy chứa nhiều chủ đề hỗn tạp.
- **Fixed-size Chunking:** Chia văn bản dựa trên số lượng ký tự hoặc token cố định (ví dụ: cứ 500 ký tự thì cắt một lần). Cách này đơn giản nhưng dễ làm mất ngữ nghĩa nếu điểm cắt rơi vào giữa câu hoặc giữa một ý đang trình bày dở.



Hình 5: Minh họa Fixed-size Chunking với Chunk size = 10.

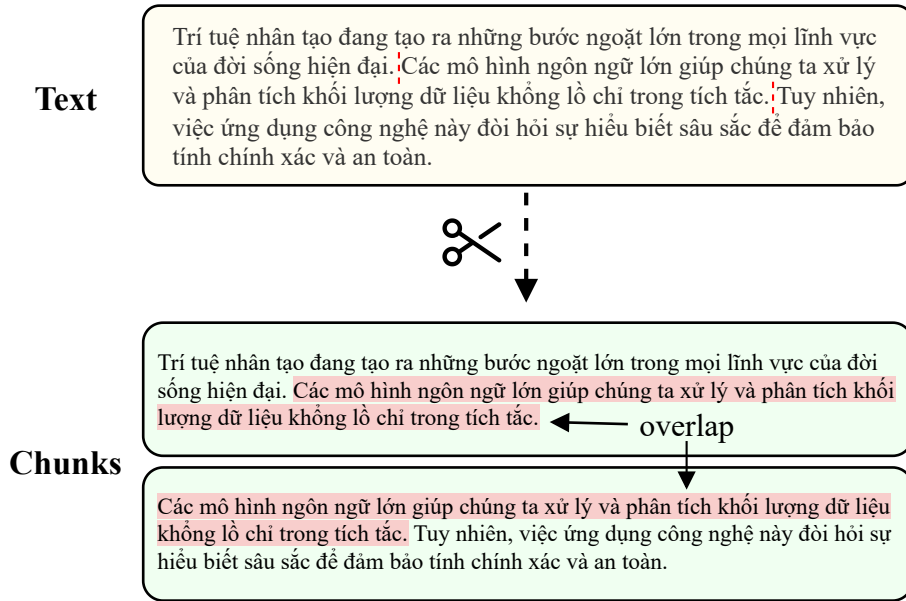
- **Recursive Chunking:** Đây là phương pháp phổ biến nhất hiện nay. Hệ thống sẽ cố gắng cắt dựa trên cấu trúc tự nhiên của văn bản theo thứ tự ưu tiên: dấu ngắt đoạn văn ($\backslash n \backslash n$) $\rightarrow$ dấu xuống dòng ($\backslash n$) $\rightarrow$ dấu chấm câu $\rightarrow$ khoảng trắng. Cách này giúp giữ trọn vẹn cấu trúc câu và đoạn văn.



Hình 6: Minh họa Recursive Chunking bằng dấu chấm câu.

- **Chunk Overlap:** Để đảm bảo ngữ nghĩa không bị mất tại điểm cắt giữa hai chunk liên kề, ta thiết lập tham số `chunk_overlap` (thường là 10-20% độ dài của chunk).

Ví dụ: Chunk 1 kết thúc ở từ thứ 100, thì Chunk 2 sẽ bắt đầu từ từ thứ 80. Phần giao thoa này đóng vai trò như “cầu nối” ngữ cảnh.



Hình 7: Minh họa cho khái niệm Chunk Overlap.

3. Chiến lược Indexing nâng cao

Ở RAG cơ bản, đoạn văn bản dùng để tìm kiếm và đoạn văn bản đưa cho LLM cùng là một. Tuy nhiên, điều này có thể nảy sinh mâu thuẫn: chunk nhỏ tốt cho tìm kiếm, chunk lớn tốt cho LLM do có nhiều ngữ cảnh hơn. Để giải quyết, ta có các chiến lược mở rộng:

- **Parent-Child Indexing (Small-to-Big):**
 - Chia văn bản thành các khối lớn, ví dụ 1000 tokens, gọi là Parent chunks để chứa ngữ cảnh đầy đủ.
 - Chia Parent chunks thành các khối nhỏ hơn, ví dụ 200 tokens, gọi là Child chunks.
 - **Cơ chế:** Hệ thống sẽ index và tìm kiếm trên Child chunks để đạt độ chính xác cao, nhưng khi trả về kết quả cho LLM, hệ thống sẽ lấy Parent chunk tương ứng.
- **Summary Indexing:** Sử dụng LLM để tóm tắt chunk gốc. Ta sẽ index phiên bản tóm tắt này với ý nghĩa được cô đọng hơn, nhưng khi cần sẽ trả về văn bản gốc chi tiết.

4. Embedding

- Sử dụng một mô hình Embedding để chuyển đổi các chunks văn bản thành các vector số học dạng dense vector trong không gian nhiều chiều.
- Theo đó, các đoạn văn bản có nội dung ngữ nghĩa giống nhau sẽ có vector nằm gần nhau trong không gian này.

5. Vector Store

- Lưu trữ các vector cùng với ID và metadata tương ứng vào cơ sở dữ liệu vector chuyên dụng để phục vụ việc truy xuất ở giai đoạn sau.

II.2.2. Phase 2: Retrieval

Đây là thành phần quyết định sự thành bại của hệ thống RAG. Nếu bước này trích xuất thông tin sai hoặc thiếu, LLM sẽ không có đủ dữ liệu để trả lời. Quá trình này không chỉ đơn thuần là tìm kiếm vector mà còn bao gồm các kỹ thuật tối ưu hóa sau:

1. Query Processing

- Ở bước này, hệ thống nhận câu hỏi từ người dùng và đưa qua mô hình Embedding để tạo ra vector truy vấn q .
- Trong thực tế, câu hỏi của người dùng thường ngắn, thiếu ngữ cảnh hoặc có thể mang nhiều nghĩa. Để khắc phục, ta có thể áp dụng:
 - Multi-Query: Sử dụng LLM để sinh ra 3-5 biến thể khác nhau của câu hỏi gốc, sau đó tìm kiếm tất cả và gộp kết quả lại. Điều này giúp tăng khả năng tìm thấy tài liệu liên quan mà không phụ thuộc vào cách diễn đạt duy nhất của người dùng.



Ví dụ:

Input (User): “Lỗi kết nối db” (*Câu hỏi ngắn, thiếu ngữ cảnh cụ thể*)

LLM Generated Queries:

- (a) “Cách khắc phục lỗi connection timeout khi kết nối database.”
- (b) “Xử lý lỗi Access Denied cho user root trong MySQL/PostgreSQL.”
- (c) “Hướng dẫn kiểm tra firewall chặn port 5432 hoặc 3306.”

→ Hệ thống sẽ tìm kiếm cả 3 vấn đề (*Timeout, Permission, Network*) để đảm bảo không bỏ sót tài liệu kỹ thuật liên quan.

- HyDE (Hypothetical Document Embeddings): Yêu cầu LLM viết một câu trả lời giả định cho câu hỏi, sau đó dùng vector embedding của câu trả lời giả định này để tìm kiếm. Cách này giúp thu hẹp khoảng cách ngữ nghĩa giữa “câu hỏi” và “tài liệu chứa câu trả lời”.



Ví dụ:

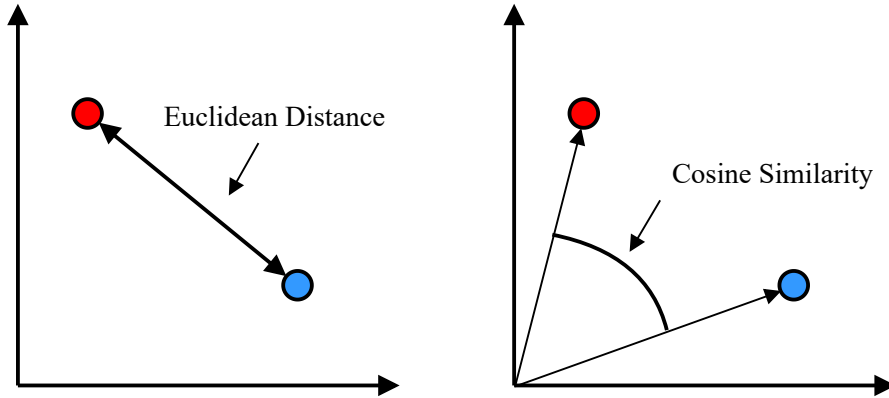
Input (User): “Quy định mang laptop ra ngoài”

Hypothetical Document (LLM giả định): “Theo chính sách bảo mật thông tin, nhân viên muốn mang thiết bị tài sản công ty (laptop, tablet) ra khỏi văn phòng cần điền phiếu ‘Đăng ký mang thiết bị’ trên hệ thống và phải được Trưởng bộ phận phê duyệt...”

→ Vector của đoạn văn giả định này sẽ khớp tốt với tài liệu “Sổ tay nhân viên” hoặc “Quy trình an ninh” trong database, tốt hơn nhiều so với câu hỏi gốc.

2. Similarity Search

- Dense Retrieval: Hệ thống tính toán độ tương đồng giữa vector truy vấn q và các vector tài liệu d trong cơ sở dữ liệu.



Hình 8: Minh họa hai metric đo độ tương đồng hoặc khoảng cách thường được dùng.

- Metric: Sử dụng Cosine Similarity hoặc Euclidean Distance.
- Algorithm: Để đảm bảo tốc độ khi dữ liệu lớn, thay vì so khớp từng đôi một (thường được gọi là Brute-force), ta sử dụng thuật toán ANN (Approximate Nearest Neighbor) như HNSW để tìm kiếm lân cận gần đúng với độ trễ thấp.
- **Vấn đề:** Tìm kiếm vector rất tốt về ngữ nghĩa nhưng lại yếu về từ khóa chính xác. Ví dụ: vector của “iPhone 14” và “iPhone 15” rất gần nhau về ngữ nghĩa, nhưng người dùng cần tìm chính xác thông tin về phiên bản 15.

3. Hybrid Search

Đây là kỹ thuật mở rộng tiêu chuẩn để khắc phục nhược điểm trên.

- **Cơ chế:** Kết hợp kết quả từ hai luồng tìm kiếm song song:
 - Dense Retrieval (Vector Search): Tìm kiếm dựa trên độ tương đồng ngữ nghĩa trong không gian vector. Phương pháp này giải việc tìm kiếm các nội dung có ý nghĩa tương đương (ví dụ: “giá xe” ↔ “chi phí mua ô tô”).
 - Sparse Retrieval (Keyword Search): Sử dụng các thuật toán thống kê truyền thống như BM25 hoặc TF-IDF.
 - *Giải thích:* Các thuật toán này hoạt động bằng cách đếm tần suất xuất hiện chính xác của từ khóa. Nó cực kỳ quan trọng để bắt chính xác các tên riêng, từ khóa hoặc thuật ngữ chuyên ngành mà Vector Search thường bỏ qua.
- **Reciprocal Rank Fusion (RRF):** Là thuật toán hậu xử lý dùng để gộp và xếp hạng lại kết quả từ hai luồng trên.

i Cách hoạt động của thuật toán RRF

Vấn đề: Điểm số của Vector Search và BM25 có thang đo hoàn toàn khác nhau, việc cộng trực tiếp sẽ không hợp lí.

Giải pháp: RRF bỏ qua điểm số gốc và chuẩn hóa dựa trên **thứ hạng**.

$$Score = \sum_i \frac{1}{k + r_i}$$

- Cơ chế: Sử dụng nghịch đảo của thứ hạng (r_i). Hạng 1 (mẫu số nhỏ) sẽ cho điểm rất lớn, Hạng 100 (mẫu số lớn) sẽ cho điểm rất nhỏ.
- Hằng số k : Thường là 60, giúp làm mượt điểm số, ngăn không cho các tài liệu Top 1 chiếm ưu thế quá tuyệt đối so với Top 2, Top 3.
- Kết quả: Tài liệu đứng hạng trung bình ở cả 2 danh sách thường có tổng điểm cao hơn tài liệu chỉ đứng nhất ở 1 danh sách nhưng mất tích ở danh sách kia.

4. Re-ranking

Sau khi có được tập ứng viên (ví dụ: Top 50 tài liệu) từ bước trên, thứ tự của chúng có thể chưa hoàn toàn chính xác do vector chỉ là dạng nén thông tin.

- **Cross-Encoder:** Sử dụng một mô hình Deep Learning chuyên biệt để chấm điểm lại mức độ liên quan giữa câu hỏi và từng tài liệu trong tập ứng viên ban đầu.
- **Tại sao cần Cross-Encoder?**
 - Bi-Encoder (Dùng ở bước Indexing/Retrieval): Mã hóa câu hỏi và tài liệu thành 2 vector riêng biệt một cách độc lập. Ưu điểm là cực nhanh, nhưng nhược điểm là làm mất mối quan hệ ngữ pháp và ngữ nghĩa phức tạp giữa câu hỏi và văn bản.
 - Cross-Encoder (Dùng ở bước Re-ranking này): Đưa cả câu hỏi và văn bản vào mô hình cùng một lúc (như con người đọc song song). Nó có thể nhận biết các sắc thái phủ định, quan hệ nguyên nhân-kết quả phức tạp. Chính xác hơn nhưng chậm hơn nhiều.
- **Chiến lược “Hình phễu”:** Retrieve Many (Lấy 50 tài liệu nhanh bằng Bi-Encoder) → Re-rank Few (Lấy 5 tài liệu tốt nhất bằng Cross-Encoder) → Đưa vào LLM. Cách này cân bằng giữa tốc độ và độ chính xác.

💡 Ví dụ:

Query (User): “Tại sao tôi không nhận được thông báo qua email?”

1. Kết quả từ Vector Search (Bi-Encoder): (Tìm dựa trên độ tương đồng chung về chủ đề “email” và “thông báo”)

- Rank 1: “Hướng dẫn cài đặt chữ ký email.” (Sai: cùng chủ đề nhưng sai ý định)
- Rank 2: “Quy định về văn hóa gửi email công ty.” (Sai)
- Rank 3: “Xử lý lỗi email bị rơi vào thư mục Spam.” (Đúng, nhưng bị xếp hạng thấp)

2. Sau khi qua Cross-Encoder (Re-ranking): (Mô hình đọc hiểu mối quan hệ “không nhận được” → “lỗi/spam”)

- A. Rank 1: “Xử lý lỗi email bị rơi vào thư mục Spam.” ↑ (Được đẩy lên đầu)
- B. Rank 2: “Hướng dẫn cài đặt chữ ký email.”
- C. Rank 3: “Quy định về văn hóa gửi email công ty.”

II.2.3. Phase 3: Generation

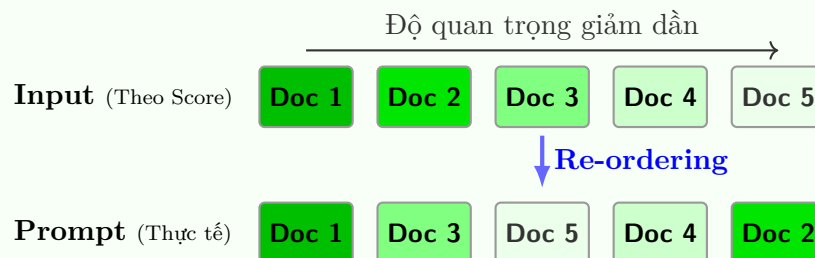
Sau khi Retrieval trả về danh sách các tài liệu liên quan, hệ thống chuyển sang giai đoạn sinh văn bản. Đây là lúc LLM tổng hợp thông tin để trả lời người dùng. Quá trình này không chỉ đơn giản là đưa dữ liệu vào, mà còn đòi hỏi các chiến lược xử lý ngữ cảnh tinh tế.

1. Context Preparation

- Context Stuffing: Đây là phương pháp đơn giản nhất. Hệ thống gộp toàn bộ văn bản của các tài liệu tìm được (Top-K) thành một đoạn văn dài, sau đó ghép vào Prompt.
- Vấn đề:
 - Chi phí & Độ trễ: Input càng dài, LLM xử lý càng chậm và tốn token.
 - Nhiều thông tin: Quá nhiều thông tin không liên quan có thể khiến LLM bị “loãng” chủ đề.
- Context Selection & Compression:
 - Context Reordering: Dựa trên hiện tượng “Lost in the Middle”: LLM thường chú ý tốt nhất vào thông tin ở đầu và cuối prompt, nhưng hay bỏ quên thông tin ở giữa.

i Chiến thuật Reordering: Tối ưu hóa vị trí theo dạng hình chữ U

Thay vì đưa tài liệu vào theo thứ tự độ liên quan giảm dần, ta sắp xếp lại để đưa các tài liệu quan trọng nhất nằm ở hai đầu và các tài liệu ít quan trọng vào giữa:



→ **Kết quả:** Các tài liệu quan trọng (màu đậm) bao bọc lấy các tài liệu ít quan trọng hơn (màu nhạt), tận dụng tối đa khả năng tập trung của LLM.

- Context Compression: Sử dụng một LLM nhỏ hoặc thuật toán NLP để tóm tắt ý chính trước khi đưa vào LLM chính.

**Ví dụ:**

Query: “Phí hủy dịch vụ là bao nhiêu?”

1. Raw Context (300 tokens): “...Theo điều khoản 7.2 của hợp đồng dịch vụ được ký kết vào ngày... (văn bản pháp lý dài dòng)... trong trường hợp khách hàng muốn chấm dứt hợp đồng trước thời hạn, cần thông báo bằng văn bản trước 30 ngày và chịu khoản phí phạt tương đương 02 tháng cước sử dụng...”

2. Compressed Context (20 tokens): “Điều khoản 7.2: Phí hủy dịch vụ là 02 tháng cước sử dụng.”

→ LLM chính chỉ cần nhận đoạn ngắn gọn này, giúp trả lời nhanh hơn và giảm thiểu hallucination.

2. Prompt Engineering

Chất lượng câu trả lời phụ thuộc rất lớn vào cách ta ra lệnh cho mô hình. Một context tốt nhưng prompt tệ vẫn có thể dẫn đến câu trả lời sai.

- **Zero-shot Prompting:** Sử dụng một template cố định để hướng dẫn mô hình trả lời trực tiếp dựa trên ngữ cảnh mà không cần ví dụ mẫu.

RAG Prompt Template (Standard)

System: You are an assistant for question-answering tasks. Use the following pieces of context to answer the question. If you don't know the answer, just say that you don't know.

Context: {context}

Question: {question}

Answer:

- **Few-shot Learning:** Cung cấp thêm 1-2 ví dụ mẫu (gồm bộ ba Context - Question - Answer chuẩn) vào prompt để LLM học được phong cách trả lời và định dạng mong muốn.

**Ví dụ**

Instruction: Trả lời ngắn gọn, chỉ đưa ra con số.

Example 1:

Context: “Doanh thu năm 2022 là 10 tỷ, năm 2023 tăng lên 12 tỷ.”

Question: Doanh thu 2023 bao nhiêu?

Answer: 12 tỷ.

Actual Task:

Context: {real_context}

Question: {user_question}

Answer:

- **Chain-of-Thought (CoT - Chuỗi suy luận):** Yêu cầu mô hình suy nghĩ từng bước (*Let's think step by step*) dựa trên các dữ kiện trong context trước khi đưa ra kết luận cuối cùng.

Tại sao cần CoT?

Đối với các câu hỏi phức tạp đòi hỏi logic, nếu ép LLM trả lời ngay lập tức, nó rất dễ đoán mò.

Với CoT: LLM tự sinh ra quy trình: “*Bước 1: Tìm giá gói A... Bước 2: Tìm giá gói B... Bước 3: So sánh...*” → Kết luận chính xác hơn đáng kể^a.

^aXem chi tiết tại: <https://arxiv.org/abs/2201.11903>

3. Generation & Attribution

- **Cơ bản:** LLM sinh ra câu trả lời dưới dạng văn bản thông thường. Mục tiêu chính là tính trôi chảy và đúng ngữ pháp.
- **Citation:** Một trong những lợi thế cạnh tranh lớn nhất của RAG so với các chatbot truyền thống là khả năng minh bạch hóa nguồn tin.
 - Cơ chế: Trong prompt, ta yêu cầu LLM: “Mọi thông tin đưa ra phải kèm theo ID của tài liệu nguồn”.
 - Lợi ích: Giúp người dùng dễ dàng kiểm chứng, xây dựng niềm tin và giảm thiểu rủi ro khi mô hình bịa đặt thông tin.

Minh họa output có trích dẫn nguồn

User Question: “Quy định làm thêm giờ tính thế nào?”

RAG Response: “Theo quy định công ty, nhân viên làm thêm giờ vào ngày thường được hưởng 150% lương cơ bản [**Sổ tay nhân viên, Tr.12**]. Đối với ngày lễ tết, mức hưởng là 300% lương [**Luật lao động 2019, Điều 98**].”

→ Người dùng có thể click vào [...] để mở trực tiếp tài liệu gốc đối chiếu.

II.3. Mở rộng: Phân tích vai trò của các thành phần

Để đánh giá chính xác tầm quan trọng của từng module trong kiến trúc RAG, chúng ta cùng phân tích các giả định theo câu hỏi: “Hệ thống sẽ hoạt động như thế nào nếu loại bỏ một thành phần cụ thể?”.

Kịch bản 1: Loại bỏ Embedding Model

Vai trò Embedding Model: Chuyển đổi văn bản sang vector để hỗ trợ truy vấn theo ngữ nghĩa.

- **Cơ chế hoạt động:** Khi không có Embedding Model, hệ thống không thể mã hóa văn bản thành vector và mất khả năng so sánh tương đồng ngữ nghĩa. Buộc phải quay về các phương pháp truy vấn theo mặt chữ (Lexical Retrieval) như BM25, TF-IDF hoặc Exact Match.
- **Tác động:** Giảm mạnh khả năng bắt ngữ cảnh, từ đồng nghĩa và cách diễn đạt tương đương.
Ví dụ: Người dùng tìm “giá xe hơi”. Tài liệu chỉ chứa “chi phí mua ô tô” có thể bị bỏ qua vì không khớp từ khóa trực tiếp.
- **Hệ quả:** Hệ thống trở thành Lexical Retrieval-driven RAG: Vẫn có LLM để trả lời, nhưng chất lượng truy vấn theo ngữ nghĩa giảm rõ rệt và dễ bỏ sót thông tin.

Kịch bản 2: Loại bỏ Vector Store/ANN Index

Vai trò Vector Store/ANN Index: Lưu trữ vector, quản lý metadata và hỗ trợ tìm kiếm nhanh ở quy mô lớn.

- **Cơ chế hoạt động:** Hệ thống vẫn có thể tạo vector, nhưng không có lớp chỉ mục ANN, việc tìm kiếm thường phải so sánh với nhiều vector theo kiểu tuyến tính (Brute-force).
- **Tác động:** Độ trễ truy vấn tăng mạnh khi dữ liệu lớn ($O(N)$). Với vài trăm tài liệu vẫn chấp nhận được, nhưng với hàng trăm nghìn đến hàng triệu vector, hệ thống dễ chậm, tốn tài nguyên và khó đáp ứng thời gian thực.
Ngoài ra, việc lọc theo metadata, cập nhật/chèn/xóa dữ liệu, và vận hành cũng trở nên khó khăn nếu không có lớp lưu trữ phù hợp.
- **Hệ quả:** Hệ thống trở thành Unscalable Prototype, có thể chạy demo ở quy mô nhỏ, nhưng khó mở rộng và khó vận hành ổn định trong môi trường thực tế.

Kịch bản 3: Loại bỏ LLM (Large Language Model)

Vai trò LLM: Bộ não tổng hợp thông tin, suy luận và sinh câu trả lời dựa trên ngữ cảnh truy vấn.

- **Cơ chế hoạt động:** Hệ thống vẫn có thể truy vấn tốt các tài liệu liên quan nhờ Embedding Model và Vector Store, nhưng quy trình dừng ở bước Retrieval. Output trả về chỉ là danh sách các đoạn văn bản (Top-K) kèm điểm số/metadata.
- **Tác động:** Mất khả năng tổng hợp, diễn giải và trả lời trực tiếp theo dạng hội thoại (Question Answering).
Ví dụ: Thay vì trả lời “Doanh thu là 10 tỷ”, hệ thống chỉ trả về các đoạn trích từ báo cáo, và người dùng phải tự đọc để rút kết luận.
- **Hệ quả:** Hệ thống trở thành Semantic Search / Retrieval System, mạnh về truy vấn theo ngữ nghĩa, nhưng không còn thành phần Generation, nên chưa phải là chatbot hay hệ RAG hoàn chỉnh.

Thành phần thiếu	Hệ thống trở thành	Hạn chế cốt lõi
Embedding Model	Lexical Retrieval-driven RAG RAG dựa trên truy vấn từ khóa	Giảm mạnh truy vấn theo ngữ nghĩa, khó bắt từ đồng nghĩa, và dễ bỏ sót thông tin.
Vector Store/ANN Index	Unscalable Prototype Mô hình khó mở rộng	Độ trễ tăng mạnh ở dữ liệu lớn, khó vận hành hệ thống.
LLM	Semantic Search / Retrieval System Hệ truy vấn ngữ nghĩa	Không thể tổng hợp, diễn giải, và trả lời hội thoại; người dùng phải tự đọc và rút kết luận từ các đoạn trích.

Bảng 1: Tổng hợp sự thay đổi của hệ thống RAG theo các kịch bản.

III. Thư viện LangChain

III.1. Giới thiệu

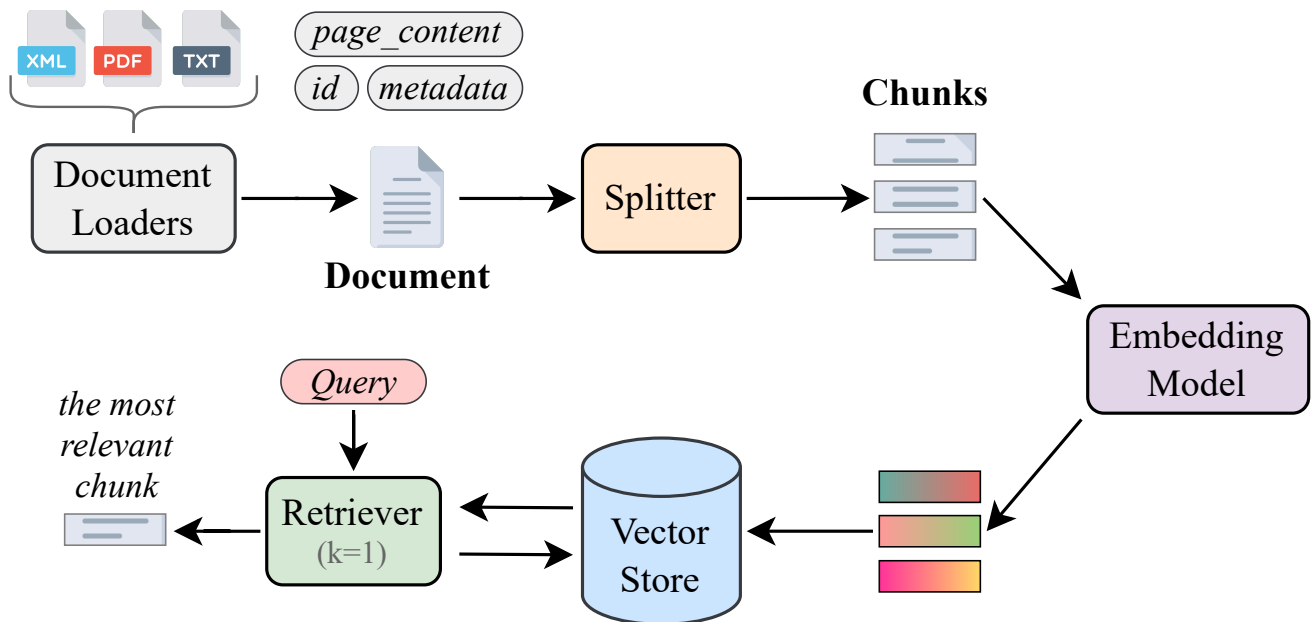
LangChain là một framework mã nguồn mở mạnh mẽ, được thiết kế để đơn giản hóa quá trình phát triển các ứng dụng sử dụng mô hình ngôn ngữ lớn (LLM). Một cách tổng quát, LangChain đóng vai trò liên kết các thành phần dữ liệu, xử lý logic và mô hình lại với nhau.

Trong bài toán Retrieval Augmented Generation (RAG), LangChain giải quyết vấn đề cốt lõi là sự rời rạc giữa nguồn dữ liệu và khả năng của LLM. Thư viện này hỗ trợ toàn bộ vòng đời của

ứng dụng RAG mà không cần người lập trình phải tự xây dựng các kết nối thủ công phức tạp.

III.2. Các thành phần cốt lõi

Để xây dựng một ứng dụng RAG với LangChain, ta cần phải hiểu về các thành phần chính như bên dưới:



Hình 9: Tổng quan các thành phần chính của RAG trong LangChain.

1. Documents và Document Loaders

Trong LangChain, đơn vị cơ bản để biểu diễn thông tin là đối tượng **Document**. Trước khi xây dựng RAG, dữ liệu từ các nguồn khác nhau cần được chuyển đổi về định dạng này.

Cấu trúc Document: LangChain triển khai lớp **Document** để đại diện cho một đơn vị văn bản và các siêu dữ liệu đi kèm, thường được gọi là **metadata**. Một đối tượng **Document** bao gồm ba thuộc tính chính:

- **page_content:** Chuỗi ký tự chứa nội dung văn bản.
- **metadata:** Một dictionary chứa thông tin bổ sung tùy ý, ví dụ như nguồn gốc của tài liệu, số trang, quan hệ với tài liệu khác.
- **id:** Mã định danh chuỗi cho tài liệu.

Loading Documents: LangChain cung cấp hệ sinh thái **Document Loaders** tích hợp với hàng trăm nguồn dữ liệu khác nhau như PDF, CSV, HTML. Ví dụ, sử dụng **PyPDFLoader** để nạp một tài liệu PDF thành một **Document**.

```

1 from langchain_community.document_loaders import PyPDFLoader
2

```

```

3 file_path = "../ai-vietnam-2025.pdf"
4 loader = PyPDFLoader(file_path)
5 doc = loader.load()

```

Splitting: Một trang tài liệu gốc thường quá dài hoặc chứa nhiều thông tin hỗn tạp. Việc chia nhỏ văn bản giúp tăng độ chính xác khi truy xuất. Công cụ **Text Splitters** được sử dụng để chia tài liệu thành các đoạn nhỏ, các đoạn này được gọi là các “chunk”. Phương pháp phổ biến là **RecursiveCharacterTextSplitter**, với chức năng:

- Phân chia đệ quy dựa trên các ký tự phân cách như xuống dòng, khoảng trắng, để giữ nguyên vẹn ngữ nghĩa.
- Giữ lại một phần ký tự trùng lặp giữa các đoạn liên kế để đảm bảo ngữ cảnh không bị ngắt quãng.

```

1 from langchain_text_splitters import RecursiveCharacterTextSplitter
2
3 text_splitter = RecursiveCharacterTextSplitter(
4     chunk_size=1000, # Kích thước của mỗi chunk
5     chunk_overlap=200, # Giữ 200 tokens của đoạn liền kề
6 )
7 all_splits = text_splitter.split_documents(doc)

```

2. Embeddings

Vector Search là phương pháp phổ biến để lưu trữ và tìm kiếm trên dữ liệu phi cấu trúc.

Nguyên lý hoạt động: Mô hình Embedding chuyển đổi một văn bản thành một vector số thực biểu diễn cho văn bản ấy. Việc biểu diễn dưới dạng số học cho phép:

- Những văn bản có ý nghĩa tương tự sẽ có vector gần nhau trong không gian hình học.
- Các chỉ số đo lường như cosine similarity được dùng để xác định độ tương đồng giữa các văn bản thông qua tính toán trên vector tương ứng của chúng.

LangChain hỗ trợ giao diện chuẩn cho nhiều nhà cung cấp mô hình embedding như OpenAI, Google, HuggingFace, ...

```

1 from langchain_openai import OpenAIEmbeddings
2
3 embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
4
5 # Tạo vector cho một đoạn văn bản
6 vector_1 = embeddings.embed_query(all_splits[0].page_content)
7 print(len(vector_1))
8 # Kết quả: 1536 (số chiều của vector)

```

3. Vector Stores

VectorStore trong LangChain chịu trách nhiệm lưu trữ các đối tượng **Document** và vector tương ứng, đồng thời cung cấp phương thức để truy vấn tới chúng. Gồm hai chức năng chính:

- **Indexing**: Thêm văn bản vào kho lưu trữ thông qua phương thức `add_documents`.
- **Querying**: Tìm kiếm văn bản dựa trên độ tương đồng giữa hai vector với phương thức `similarity_search`.

LangChain tích hợp với nhiều loại vector store: từ dạng lưu trong bộ nhớ như **FAISS** và **InMemoryVectorStore**, đến các cơ sở dữ liệu chuyên dụng với **Chroma**, **Pinecone**, **Postgres**.

```
1 from langchain_core.vectorstores import InMemoryVectorStore
2
3 vector_store = InMemoryVectorStore(embeddings)
4 ids = vector_store.add_documents(documents=all_splits)
5 results = vector_store.similarity_search(
6     "How many distribution centers does Nike have in the US?"
7 )
```

4. Retrievers

Trong khi **VectorStore** là nơi lưu trữ, **Retriever** được dùng để truy vấn dữ liệu. Đặc biệt là Retrievers trong LangChain là các **Runnable**, cho phép chúng kết nối dễ dàng vào các chuỗi xử lý hay “chains”.

VectorStoreRetriever

Cách để tạo Retriever đơn giản nhất là từ một **VectorStore** thông qua hàm `.as_retriever()`. Trong đó, có thể bổ sung các tham số tìm kiếm như `search_kwargs` và loại tìm kiếm `search_type`:

- `similarity`: Tìm kiếm tương đồng mặc định.
- `mmr` (Maximum Marginal Relevance): Cân bằng giữa độ tương đồng và độ đa dạng của kết quả.
- `similarity_score_threshold`: Lọc bỏ các kết quả có điểm tương đồng thấp hơn ngưỡng quy định.

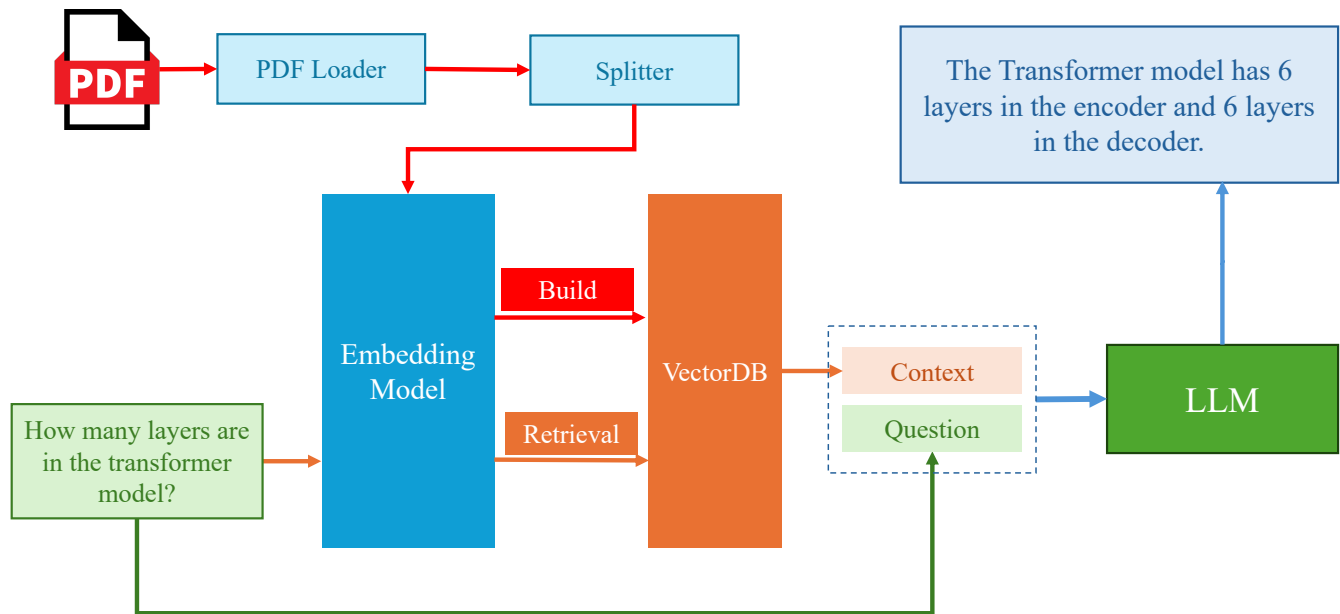
```
1 # Tạo Retriever từ Vector Store
2 retriever = vector_store.as_retriever(
3     search_type="similarity",
4     search_kwargs={"k": 1} # Chỉ lấy 1 kết quả tốt nhất
5 )
6
7 # Thực thi tìm kiếm (Batch)
8 retriever.batch([
9     "How many distribution centers does Nike have in the US?",
```

```

10 "When was Nike incorporated?"
11 ])
```

5. Phần đọc thêm thư viện LangChain

Nếu các bạn quan tâm và mong muốn thực hành nhiều hơn về thư viện LangChain có thể tham khảo bài đọc thêm tại [đây](#). Bài viết này sẽ xây dựng một ứng dụng về RAG (Retrieval Augmented Generation) trả lời các câu hỏi học thuật tận dụng nguồn tài liệu là các bài báo khoa học thu thập được (dưới dạng file pdf), sử dụng thư viện LangChain. Tổng quan, pipeline của bài đọc thêm như sau:



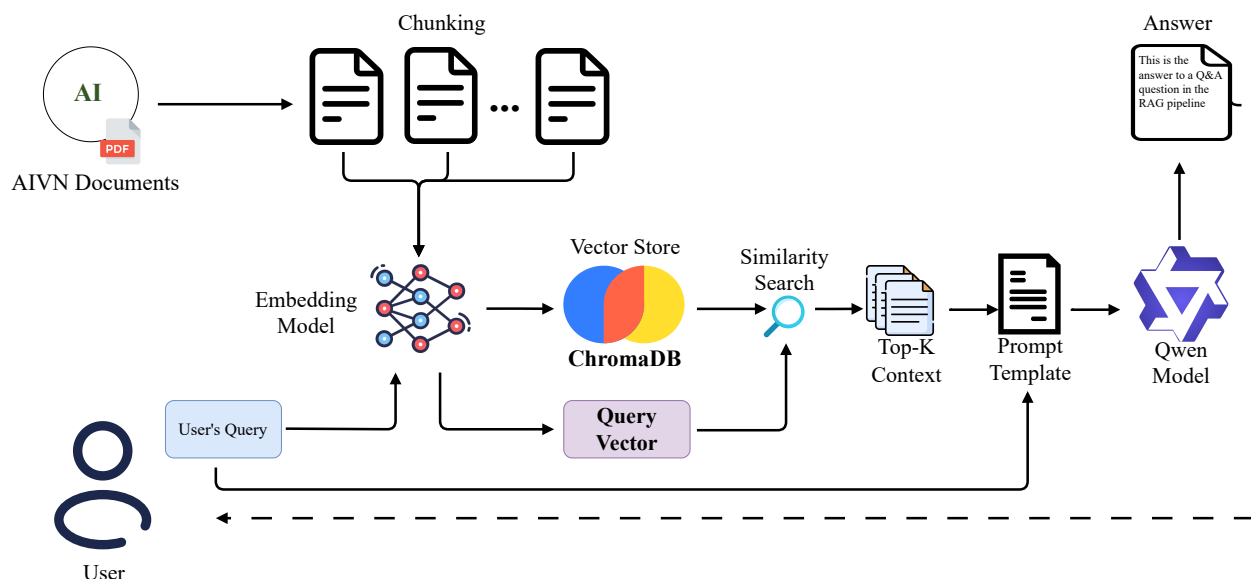
Hình 10: Tổng quan về pipeline của project.

Theo đó:

- Từ danh sách các bài báo khoa học, ta tách thành các văn bản nhỏ. Từ đó, xây dựng một hệ cơ sở dữ liệu vector với một embedding model.
- Bên cạnh câu hỏi đầu vào (question), ta truy vấn các mẫu văn bản có liên quan đến câu hỏi, dùng làm ngữ cảnh (context) trong câu prompt. Đây là nguồn thông tin mà LLMs có thể dựa vào để trả lời câu hỏi.
- Đưa câu prompt vào mô hình (question và context) để nhận câu trả lời từ mô hình.

IV. Thực hành

Trong phần này, chúng ta sẽ hiện thực hoá các khái niệm về RAG và LangChain bằng việc xây dựng một Question Answering System (tạm dịch: hệ thống hỏi đáp) dành cho các tài liệu PDF tiếng Việt.



Hình 11: Kiến trúc hệ thống RAG sử dụng ChromaDB và mô hình ngôn ngữ Qwen.

Quan sát Hình 11, quy trình phần thực hành bắt đầu từ việc xử lý tài liệu đầu vào, sau đó thực hiện chia nhỏ và vector hoá để lưu trữ vào ChromaDB. Khi người dùng đặt câu hỏi, hệ thống sẽ tìm kiếm ngữ cảnh phù hợp và đưa vào mô hình Qwen để sinh câu trả lời.

Về mặt kỹ thuật, chúng ta sẽ làm việc trên Google Colab với GPU T4, Python phiên bản 3.12, sử dụng nguồn dữ liệu từ các bài viết public của AI VIET NAM. Quy trình triển khai chi tiết sẽ đi qua các bước: thiết lập môi trường làm việc và tổ chức cấu trúc dự án, thu thập và xử lý dữ liệu, xây dựng cơ sở dữ liệu vector, khởi tạo mô hình ngôn ngữ và cuối cùng là phát triển giao diện tương tác.

Các giai đoạn dưới đây được trình bày theo đúng thứ tự của các cell trong notebook, giúp bạn vừa nắm được ý tưởng, vừa dễ dàng chạy thử từng đoạn mã minh họa.

IV.1. Chuẩn bị môi trường và cấu trúc dự án

Ở bước đầu tiên, chúng ta thiết lập môi trường làm việc và tổ chức lại cấu trúc thư mục nhằm đảm bảo toàn bộ mã nguồn vận hành có hệ thống, dễ mở rộng và dễ tích hợp các thành phần của pipeline RAG trong các bước tiếp theo.

```

1 # Cài đặt các thư viện chính cho mô hình ngôn ngữ, embedding, RAG và giao diện web
2 !pip install -q \
3     "torch>=2.0.0" \
4     "transformers>=4.40.0" \
5     "accelerate>=0.30.0" \
6     "huggingface-hub>=0.23.0" \
7     "sentence-transformers>=2.7.0" \
8     "langchain>=0.2.0" \
9     "langchain-core>=0.2.0" \
10    "langchain-community>=0.1.0" \
11    "langchain-text-splitters>=0.2.0" \
12    "chromadb>=0.5.0" \
13    "langchain-chroma>=0.2.0" \
14    "pypdf>=4.2.0" \
15    "langserve[all]>=0.1.0" \
16    "fastapi>=0.115.0" \
17    "uvicorn>=0.30.0" \
18    "gradio>=5.0.0" \
19    "langchain-huggingface" \
20    "wget"

```

Đoạn mã sau giúp thiết lập thư mục gốc của dự án trên Colab và tạo các thư mục con để lưu dữ liệu và mã nguồn. Cách tổ chức này giúp chúng ta dễ quản lý khi số lượng file tăng lên.

```

1 import os
2 import sys
3
4 PROJECT_ROOT = "/content/rag_langchain"
5
6 # Đặt token để tải mô hình private, hãy thay bằng token của bạn
7 os.environ["HF_TOKEN"] = "YOUR_HUGGINGFACE_TOKEN"
8 # Tạo thư mục cho dữ liệu và mã nguồn
9 os.makedirs(os.path.join(PROJECT_ROOT, "data_source", "generative_ai"), exist_ok=True)
10 os.makedirs(os.path.join(PROJECT_ROOT, "src", "base"), exist_ok=True)
11 os.makedirs(os.path.join(PROJECT_ROOT, "src", "rag"), exist_ok=True)
12
13 os.chdir(PROJECT_ROOT)
14
15 # Thêm PROJECT_ROOT vào sys.path để có thể import các module trong src
16 if PROJECT_ROOT not in sys.path:
17     sys.path.append(PROJECT_ROOT)
18
19 os.environ["TOKENIZERS_PARALLELISM"] = "false"

```

Cuối cùng, chúng ta tạo các tệp `__init__.py` để các thư mục mã nguồn có thể được sử dụng như các package Python, giúp việc import module trở nên rõ ràng hơn.

```

1 %%bash
2 touch /content/rag_langchain/src/__init__.py

```

```
3 touch /content/rag_langchain/src/base/__init__.py
4 touch /content/rag_langchain/src/rag/__init__.py
```

IV.2. Chuẩn bị dữ liệu PDF đầu vào

Tại bước này, ta tải các tài liệu PDF dùng làm dữ liệu cho hệ thống RAG. Các tài liệu này được trích từ các bài viết công khai của AI VIET NAM, sau đó được lưu vào một thư mục chung để phục vụ cho bước xử lý văn bản và xây dựng vector database.

```
1 import os
2 import wget
3
4 # Thư mục lưu các tài liệu PDF
5 DATA_DIR = "/content/rag_langchain/data_source/generative_ai"
6 os.makedirs(DATA_DIR, exist_ok=True)
7
8 # Danh sách các tài liệu PDF mẫu sẽ dùng làm dữ liệu cho hệ thống RAG
9 pdf_links = [
10     {
11         "title": "Vòng lặp for và ứng dụng",
12         "url": "https://docs.google.com/uc?export=download&id=
13             1_zJnj5qORwzMH6vgftGzkYu4fFew6Pq4"
14     },
15     {
16         "title": "Giám sát hệ thống AI với Grafana và Prometheus",
17         "url": "https://docs.google.com/uc?export=download&id=1gZWLJddiuLd-
18             ZZ8j_Nmfu_r7fISGRW6J"
19     },
20     {
21         "title": "Các Thuộc Đo Đánh Giá Mô Hình Hồi Quy",
22         "url": "https://docs.google.com/uc?export=download&id=1C-
23             f9pNW0mkMxaakDcpliN3isTVqRQtR3"
24     },
25     {
26         "title": "A simple, strong baseline for Long-Term Forecasts",
27         "url": "https://docs.google.com/uc?export=download&id=
28             16KFeWi00NqV3ZJYAgC_20y7bxUNGd9hU"
29     },
30 ]
31
32 # Tải các file PDF về thư mục dữ liệu nếu chưa tồn tại
33 for pdf_info in pdf_links:
34     save_path = os.path.join(DATA_DIR, f"{pdf_info['title']}.pdf")
35     if not os.path.exists(save_path):
36         try:
37             wget.download(pdf_info["url"], out=save_path)
38         except Exception as e:
39             pass
```

IV.3. Tiền xử lý văn bản và Chunking

Ở bước này, ta làm sạch văn bản trích xuất từ các file PDF và chia nội dung thành các đoạn nhỏ phù hợp. Việc tiền xử lý giúp văn bản gọn, ít lỗi, còn việc chia đoạn giúp mô hình embedding và mô hình ngôn ngữ làm việc hiệu quả hơn.

Làm sạch văn bản tiếng Việt

Văn bản trích xuất từ PDF thường chứa nhiều ký tự đặc biệt, khoảng trắng thừa, hoặc các vấn đề về encoding. Hàm `clean_vietnamese_text` dưới đây giúp chuẩn hóa văn bản tiếng Việt, loại bỏ các ký tự không mong muốn và đảm bảo encoding đồng nhất.

```

1 import re
2 import unicodedata
3 from typing import List
4
5 def clean_vietnamese_text(text: str) -> str:
6     # Chuẩn hóa Unicode về dạng NFC cho tiếng Việt
7     text = unicodedata.normalize('NFC', text)
8     text = "".join(
9         char for char in text
10        if not unicodedata.category(char).startswith('C') or char in '\n\t'
11    )
12
13    # Gộp khoảng trắng thừa và dòng trống
14    text = re.sub(r'\s+', ' ', text)
15    text = re.sub(r'\n\s*\n', '\n', text)
16    return text.strip()

```

Tải và chia nhỏ tài liệu PDF

Sau khi đã có hàm làm sạch văn bản, ta xây dựng hai class chính: `SimpleLoader` để tải tài liệu PDF và `TextSplitter` để chia văn bản thành các chunk phù hợp với mô hình.

```

1 import glob
2 from tqdm import tqdm
3 from langchain_community.document_loaders import PyPDFLoader
4 from langchain_text_splitters import RecursiveCharacterTextSplitter
5
6 class SimpleLoader:
7     # Tải một file PDF và gọi hàm làm sạch nội dung văn bản
8     def load_pdf(self, pdf_file: str):
9         docs = PyPDFLoader(pdf_file, extract_images=True).load()
10        for doc in docs:
11            doc.page_content = clean_vietnamese_text(doc.page_content)
12        return docs
13

```

```

14  # Tải tất cả file PDF trong một thư mục
15  def load_dir(self, dir_path: str) -> List:
16      pdf_files = glob.glob(f"{dir_path}/*.pdf")
17      if not pdf_files:
18          raise ValueError(f"No PDF files found in {dir_path}")
19
20      all_docs = []
21      for pdf_file in tqdm(pdf_files, desc="Loading PDFs"):
22          try:
23              all_docs.extend(self.load_pdf(pdf_file))
24          except Exception as e:
25              pass
26      return all_docs
27
28  class TextSplitter:
29
30      # Khởi tạo với chunk_size và chunk_overlap phù hợp cho tiếng Việt
31      def __init__(self,
32                  chunk_size: int = 400,
33                  chunk_overlap: int = 120,
34                  ):
35          self.splitter = RecursiveCharacterTextSplitter(
36              separators=["\n\n", "\n", " ", ""],
37              chunk_size=chunk_size,
38              chunk_overlap=chunk_overlap,
39              length_function=len,
40              )
41
42      # Chia danh sách documents thành các chunk nhỏ hơn
43      def split(self, documents):
44          return self.splitter.split_documents(documents)

```

IV.4. Xây dựng Vector Database

Tại bước này, các chunk đã được chia nhỏ sẽ được chuyển đổi thành vector thông qua mô hình embedding đa ngôn ngữ từ HuggingFace, sau đó lưu trữ vào Chroma. Class `VectorDB` lúc này đóng vai trò trung tâm, quản lý toàn bộ quy trình từ khởi tạo embedding model, index các document chunks, cho đến việc cung cấp retriever phục vụ cho việc tìm kiếm ngữ nghĩa.

```

1  from langchain_chroma import Chroma
2  from langchain_huggingface import HuggingFaceEmbeddings
3
4  class VectorDB:
5      def __init__(
6          self,
7          documents=None,
8          embedding_model: str = "sentence-transformers/paraphrase-multilingual-MiniLM-
                               L12-v2",
9          collection_name: str = "vietnamese_docs",

```

```

10     persist_dir: str = "/content/chroma_data",
11 ):
12     self.persist_dir = persist_dir
13     self.collection_name = collection_name
14
15     self.embedding = HuggingFaceEmbeddings(model_name=embedding_model)
16     self.db = self._build_db(documents)
17
18     # Xây dựng hoặc tải Vector Database
19     def _build_db(self, documents):
20         # Trường hợp tải database đã có từ persist_dir
21         if documents is None or len(documents) == 0:
22             db = Chroma(
23                 collection_name=self.collection_name,
24                 embedding_function=self.embedding,
25                 persist_directory=self.persist_dir,
26             )
27
28             # Trường hợp tạo mới database từ documents
29         else:
30             db = Chroma.from_documents(
31                 documents=documents,
32                 embedding=self.embedding,
33                 collection_name=self.collection_name,
34                 persist_directory=self.persist_dir,
35             )
36         return db
37
38     # Tạo retriever để tìm kiếm tương đồng
39     def get_retriever(self, search_kwargs: dict = None):
40         if search_kwargs is None:
41             search_kwargs = {"k": 4} # Mặc định lấy 4 document gần nhất
42
43         return self.db.as_retriever(
44             search_type="similarity",
45             search_kwargs=search_kwargs,
46         )

```

IV.5. Khởi tạo LLM và xây dựng RAG Chain

Ta khởi tạo mô hình ngôn ngữ để đảm nhiệm phần sinh câu trả lời, đồng thời định nghĩa một RAG chain kết hợp retriever, prompt và LLM thành một quy trình thống nhất để trả lời câu hỏi dựa trên ngữ cảnh đã truy xuất.

Khởi tạo LLM

Hàm `get_hf_llm` khởi tạo một mô hình ngôn ngữ từ HuggingFace và đóng gói thành LangChain pipeline. Ở đây, ta sử dụng Qwen2.5-3B-Instruct, một mô hình instruction-tuned với khả năng xử lý hiệu quả trong các tác vụ hỏi đáp và sinh văn bản.

```

1 import torch
2 from transformers import pipeline, AutoTokenizer, AutoModelForCausalLM
3 from langchain_huggingface import HuggingFacePipeline
4
5 def get_hf_llm(
6     model_name: str = "Qwen/Qwen2.5-3B-Instruct",
7     temperature: float = 0.2, max_new_tokens: int = 450,
8     **kwargs
9 ):
10     model = AutoModelForCausalLM.from_pretrained(
11         model_name,
12         torch_dtype=torch.float16,
13         device_map="auto",
14         low_cpu_mem_usage=True
15     )
16
17     tokenizer = AutoTokenizer.from_pretrained(model_name)
18
19     # Tạo text generation pipeline
20     model_pipeline = pipeline(
21         "text-generation",
22         model=model,
23         tokenizer=tokenizer,
24         temperature=temperature,
25         max_new_tokens=max_new_tokens,
26         pad_token_id=tokenizer.eos_token_id,
27         do_sample=True,
28         top_p=0.75
29     )
30
31     # Đóng gói pipeline thành LangChain LLM
32     llm = HuggingFacePipeline(pipeline=model_pipeline, model_kwargs=kwargs)
33     return llm

```

Định nghĩa RAG Chain và parser

Class `FocusedAnswerParser` giúp làm sạch và rút gọn câu trả lời của LLM, loại bỏ các phần thừa hoặc lặp lại. Trong khi đó, class `OfflineRAG` kết nối các thành phần retriever, prompt template và LLM thành một chain hoàn chỉnh.

```

1 from langchain_core.prompts import PromptTemplate
2 from langchain_core.runnables import RunnablePassthrough
3 from langchain_core.output_parsers import StrOutputParser
4
5 class FocusedAnswerParser(StrOutputParser):
6     def parse(self, text: str) -> str:
7         text = text.strip()
8         if "[TRẢ LỜI]:" in text:
9             answer = text.split("[TRẢ LỜI]:")[-1].strip()
10        else:

```

```

11         answer = text
12
13         answer = re.sub(r'^\s*[\-\*]\s*', '', answer, flags=re.MULTILINE)
14         answer = re.sub(r'\n+', ' ', answer)
15         lines = [line.strip() for line in answer.split('.') if line.strip() and len(
16             line.strip()) > 5]
17         return '. '.join(lines[:5]) + ('. ' if lines else '')
18
19 class OfflineRAG:
20     def __init__(self, llm):
21         self.llm = llm
22         self.prompt = PromptTemplate.from_template("""
23     Bạn là trợ lý AI phân tích tài liệu tiếng Việt.
24     [TÀI LIỆU]:
25     {context}
26
27     [CÂU HỎI]:
28     {question}
29
30     Hãy trả lời dựa trên tài liệu. Nếu tài liệu không có thông tin, nói rõ "Không có thông
31     tin".
32     Trả lời đầy đủ thông tin (3-5 câu chi tiết), không thêm bất kỳ chi tiết nào ngoài tài
33     liệu.
34     [TRẢ LỜI]: """)
35
36     self.answer_parser = FocusedAnswerParser()
37
38     # Xây dựng RAG chain từ retriever
39     def get_chain(self, retriever):
40         def format_docs(docs):
41             formatted = []
42             seen = set()
43             for doc in docs:
44                 content = doc.page_content.strip()
45                 if content and len(content) > 40 and content not in seen:
46                     formatted.append(content)
47                     seen.add(content)
48             return "\n\n".join(formatted)
49         rag_chain = (
50             {"context": retriever | format_docs, "question": RunnablePassthrough()}
51             | self.prompt
52             | self.llm
53             | self.answer_parser
54         )
55         return rag_chain

```

Kết nối toàn bộ pipeline

Sau khi đã có đủ các thành phần, ta kết nối chúng lại thành một hàm xử lý duy nhất. Sau đó sử dụng hàm `answer_question` thực hiện toàn bộ quy trình: nhận câu hỏi, truy xuất dữ liệu,

xây dựng prompt và sinh câu trả lời cuối cùng.

```

1 os.chdir("/content/rag_langchain")
2
3 # Khởi tạo LLM
4 llm = get_hf_llm()
5
6 data_dir = "/content/rag_langchain/data_source/generative_ai"
7
8 # Load và xử lý documents
9 loader = SimpleLoader()
10 text_splitter = TextSplitter(chunk_size=400, chunk_overlap=120)
11
12 raw_docs = loader.load_dir(data_dir)
13 split_docs = text_splitter.split(raw_docs)
14
15 # Xây dựng Vector Database và Retriever
16 vdb = VectorDB(documents=split_docs)
17 retriever = vdb.get_retriever(search_kwargs={"k": 4})
18
19 # Tạo RAG Chain
20 rag = OfflineRAG(llm)
21 rag_chain = rag.get_chain(retriever)
22
23 # Hàm xử lý câu hỏi
24 def answer_question(question: str) -> str:
25     try:
26         return rag_chain.invoke(question)
27     except Exception as e:
28         return f"Error: {str(e)}"

```

IV.6. Xây dựng giao diện ứng dụng

Cuối cùng, chúng ta xây dựng một giao diện đơn giản sử dụng Gradio để người dùng có thể tương tác trực tiếp với hệ thống RAG. Giao diện này không chỉ tạo điều kiện thuận lợi cho việc trải nghiệm hệ thống mà còn giúp minh họa trực quan toàn bộ quy trình hoạt động của pipeline RAG, từ việc tiếp nhận truy vấn đến khi trả về kết quả.

```

1 import gradio as gr
2
3 # Xây dựng giao diện với Gradio Blocks
4 with gr.Blocks(title="RAG Vietnamese QA") as demo:
5     gr.Markdown("# RAG - Hỏi Đáp về Tài Liệu")
6
7     # Thiết kế giao diện cho các thành phần hiển thị trên 1 dòng
8     with gr.Row():
9         # Cột bên trái: Input câu hỏi
10         with gr.Column(scale=1):
11             question_input = gr.Textbox(

```

```

12         label="Câu hỏi",
13         placeholder="Ví dụ: Vì sao classification lại không thể chỉ nhìn
14             accuracy để đánh giá?",
15         lines=3,
16     )
17     submit_btn = gr.Button("Gửi", variant="primary")
18
19     # Cột bên phải: Output câu trả lời
20     with gr.Column(scale=2):
21         answer_output = gr.Textbox(
22             label="Câu trả lời",
23             lines=6,
24             interactive=False
25         )
26
27     # Kết nối button với hàm xử lý
28     submit_btn.click(
29         fn=answer_question,
30         inputs=question_input,
31         outputs=answer_output,
32     )
33 demo.launch(share=True)

```

RAG - Hỏi Đáp về Tài Liệu

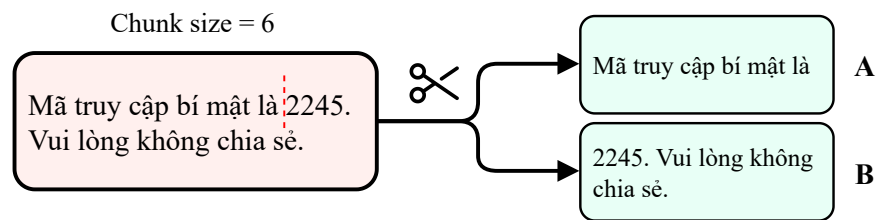
Câu hỏi	Câu trả lời
Vì sao trong bài toán classification lại không thể chỉ nhìn accuracy để đánh giá?	Trong bài toán phân loại, nếu chỉ nhìn vào độ chính xác (accuracy), có thể dẫn đến kết luận sai lầm. Mô hình có thể luôn dự đoán kết quả âm tính cho tất cả các mẫu dữ liệu, dẫn đến độ chính xác đạt 99%, mặc dù mô hình hoàn toàn vô dụng vì không phát hiện được bất kỳ ca bệnh nào. Do đó, chỉ dựa vào độ chính xác không đủ để đánh giá hiệu năng của mô hình, cần phải xem xét thêm các chỉ số khác như độ chính xác cụ thể đối với lớp dương và lớp âm, cũng như độ chính xác cụ thể trong việc dự đoán dương tính. Ngoài ra, cần xem xét độ chính xác trong việc dự đoán âm tính để tránh tình trạng mô hình dự đoán dương tính cho tất cả các mẫu dữ liệu. Vì vậy, trong bài toán phân loại, chỉ nhìn vào độ chính xác không đủ để đánh giá hiệu năng của mô hình.

Hình 12: Hình ảnh giao diện demo chương trình RAG.

Hình 12 minh họa giao diện người dùng sau khi hệ thống được khởi tạo thành công. Khi chạy cell này, một đường link Gradio sẽ được tạo ra, cung cấp môi trường tương tác trực quan. Tại đây, người dùng có thể nhập câu hỏi liên quan đến các tài liệu đã được index, sau đó quan sát cách hệ thống RAG truy xuất ngữ cảnh và sinh ra câu trả lời tương ứng.

V. Câu hỏi trắc nghiệm

- Trong cơ chế Hybrid Search, tại sao chúng ta không thể cộng trực tiếp điểm số từ Vector Search và Keyword Search mà phải cần đến thuật toán Reciprocal Rank Fusion (RRF)?
 - Vì Vector Search trả về kết quả quá chậm so với Keyword Search nên cần RRF để đồng bộ hóa thời gian.
 - Vì thang đo điểm số của hai phương pháp khác nhau, việc cộng trực tiếp sẽ khiến BM25 lấn át hoàn toàn Vector.
 - Vì Vector Search chỉ tìm được từ đồng nghĩa, trong khi RRF giúp loại bỏ các từ đồng nghĩa sai.
 - Vì RRF là thuật toán duy nhất có thể chạy trên GPU để tăng tốc độ xử lý.
- Quan sát hình minh họa dưới đây:



- Nếu người dùng hỏi: “Mã truy cập bí mật là gì?”, kết quả nào sau đây phản ánh chính xác nhất hoạt động của hệ thống RAG?
- Hệ thống tìm thấy Chunk B vì nó chứa con số cụ thể, và LLM trả lời chính xác mã số.
 - Hệ thống tìm thấy Chunk A, nhưng LLM sẽ không trả lời được mã số vì thông tin này nằm ở Chunk B, vốn không được tìm thấy.
 - Hệ thống sẽ tự động tìm thấy cả Chunk A và Chunk B nhờ cơ chế suy luận bắc cầu của Vector Database.
 - Hệ thống không tìm thấy cả hai chunk vì câu văn bị cắt đôi làm hỏng cấu trúc ngữ pháp.
- Tại sao mô hình Cross-Encoder lại có độ chính xác cao hơn Bi-Encoder, mặc dù Cross-Encoder chậm hơn rất nhiều?
 - Vì Cross-Encoder quan sát được sự tương tác trực tiếp giữa từng token của câu hỏi và từng token của tài liệu cùng lúc.
 - Vì Cross-Encoder nén câu hỏi và tài liệu thành các vector có số chiều lớn hơn.
 - Vì Cross-Encoder sử dụng cơ sở dữ liệu đồ thị Graph Database thay vì Vector Database.
 - Vì Cross-Encoder thực hiện việc tìm kiếm từ khóa chính xác thay vì tìm kiếm ngữ nghĩa.
 - Để khắc phục hiện tượng “Lost in the Middle”, chiến lược sắp xếp ngữ cảnh nào sau đây là tối ưu nhất trước khi gửi cho LLM?

- (a) Sắp xếp theo thứ tự độ tương đồng giảm dần.
 - (b) Sắp xếp ngẫu nhiên để tránh việc mô hình bị thiên kiến vào vị trí.
 - (c) Sắp xếp theo hình chữ U: Đưa các tài liệu quan trọng nhất vào đầu và cuối prompt, giấu các tài liệu ít quan trọng hơn vào giữa.
 - (d) Sắp xếp theo trình tự thời gian: Các tài liệu mới nhất được đưa lên đầu, cũ nhất xuống cuối.
5. Trong LangChain, “đơn vị cơ bản” để biểu diễn thông tin là gì?
- (a) Chunk.
 - (b) Vector.
 - (c) PromptTemplate.
 - (d) Document.
6. Trong Ví dụ 1 ở phần **Loading Documents**, PyPDFLoader tạo ra Document theo đơn vị nào?
- (a) Mỗi file PDF là một Document.
 - (b) Mỗi trang trong file PDF là một Document.
 - (c) Mỗi chunk là một Document.
 - (d) Mỗi câu là một Document.
7. Đối tượng nào biến các chunk thành vector embedding để lưu/tra cứu trong VectorStore?

```
1 docs = PyPDFLoader("sample.pdf").load()
2
3 splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
4 chunks = splitter.split_documents(docs)
5
6 emb = OpenAIEmbeddings(model="text-embedding-3-large")
7 vs = Chroma.from_documents(chunks, embedding=emb)
8
9 retriever = vs.as_retriever(search_type="similarity", search_kwargs={"k": 4})
```

- (a) PyPDFLoader.
 - (b) RecursiveCharacterTextSplitter.
 - (c) OpenAIEmbeddings.
 - (d) Chroma (Vector Store).
8. Quan sát đoạn code cấu hình Text Splitter dưới đây:

```
1 class TextSplitter:
2     def init(self, chunk_size=400, chunk_overlap=120):
3         self.splitter = RecursiveCharacterTextSplitter(
```

```

4         separators=["\n\n", "\n", " ", ""],
5         chunk_size=chunk_size,
6         chunk_overlap=chunk_overlap,
7         length_function=len,
8     )

```

Trong bối cảnh xây dựng RAG, vai trò cụ thể của tham số `chunk_overlap=120` trong đoạn code trên là gì?

- (a) Chia văn bản thành các đoạn nhỏ có kích thước cố định đúng bằng 120 ký tự bất kể cấu trúc câu.
 - (b) Tự động loại bỏ 120 ký tự đặc biệt hoặc khoảng trắng thừa ở đầu mỗi đoạn văn.
 - (c) Duy trì sự liên tục về ngữ cảnh giữa các đoạn liền kề bằng cách lặp lại 120 ký tự cuối của chunk trước sang chunk sau.
 - (d) Giới hạn độ dài tối đa của mỗi chunk là 120 ký tự để giảm tải bộ nhớ khi embedding.
9. Khi khởi tạo pipeline sinh văn bản cho hệ thống RAG, đoạn code cấu hình thường được thiết lập như sau:

```

1 model_pipeline = pipeline(
2     "text-generation",
3     model=model,
4     tokenizer=tokenizer,
5     temperature=0.2,      # Lưu ý tham số này
6     max_new_tokens=450,
7     do_sample=True,
8     top_p=0.75
9 )

```

Trong ngữ cảnh của bài toán RAG, tại sao chiến lược đặt `temperature` ở mức thấp lại là tiêu chuẩn được khuyến nghị thay vì đặt ở mức cao?

- (a) Để ngăn chặn mô hình sinh ra các vòng lặp vô tận khi gặp các đoạn văn bản khó.
 - (b) Để tăng tính đa dạng cho câu trả lời, giúp mô hình có thể diễn đạt cùng một ý theo nhiều văn phong khác nhau trong mỗi lần chạy.
 - (c) Để giảm thiểu sự sáng tạo ngẫu nhiên, buộc mô hình tập trung vào các từ có xác suất cao nhất nhằm đảm bảo câu trả lời bám sát vào tài liệu được cung cấp.
 - (d) Để tăng tốc độ xử lý của mô hình, vì giá trị `temperature` cao đòi hỏi nhiều tài nguyên tính toán hơn để sinh ra các từ vựng phong phú.
10. Trước khi đưa các tài liệu tìm được vào Prompt, hàm helper `format_docs` thực hiện một bước lọc dữ liệu như sau:

```

1 def format_docs(docs):
2     formatted = []

```

```
3     seen = set()
4     for doc in docs:
5         content = doc.page_content.strip()
6         if content and len(content) > 40 and content not in seen:
7             formatted.append(content)
8             seen.add(content)
9     return "\n\n".join(formatted)
```

Mục đích thực tế của điều kiện `len(content) > 40` và `content not in seen` trong đoạn code trên là gì?

- (a) Đảm bảo chỉ có tối đa 40 tài liệu quan trọng nhất được gửi vào mô hình để xử lý.
- (b) Loại bỏ các đoạn văn bản quá ngắn, thiếu ngữ nghĩa và ngăn chặn việc lặp lại thông tin để tối ưu hóa chất lượng Prompt.
- (c) Kiểm tra xem tài liệu có chứa ít nhất 40 từ khóa quan trọng liên quan đến câu hỏi hay không.
- (d) Giới hạn tổng độ dài của prompt không vượt quá 40 token để tiết kiệm chi phí API và tránh lỗi Context Window.

Phụ lục

1. **Solution:** File code cài đặt thực nghiệm có thể được tải tại [đây](#).
2. **Phần đọc thêm:** Nếu các bạn quan tâm và mong muốn thực hành nhiều hơn về thư viện LangChain có thể tham khảo bài đọc thêm tại [đây](#)
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,6K thành viên



Hình 13: Hình ảnh group facebook AIO Q&A.

Project: Insight into RAG

Dương Trường Bình

Nguyễn Anh Khôi

Trần Đại Nhân

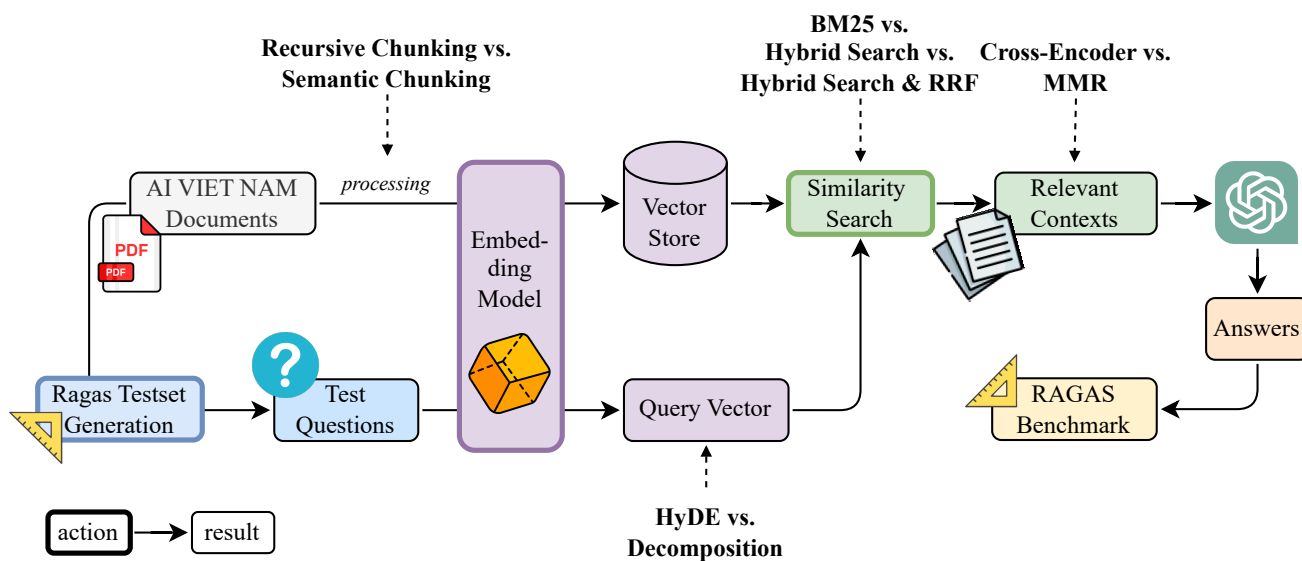
Dương Đình Thắng

Đinh Quang Vinh

I. Dẫn nhập

Trong bài viết trước, chúng ta đã cùng nhau xây dựng một hệ thống retrieval augmented generation (RAG) cơ bản. Chúng ta đã thấy được sức mạnh của việc kết hợp large language model (LLM) với dữ liệu riêng, từ việc xử lý tài liệu PDF, lập chỉ mục vector, cho đến việc yêu cầu mô hình trả lời câu hỏi dựa trên ngữ cảnh tìm được.

Tuy nhiên, khoảng cách giữa lý thuyết và một hệ thống thực tế là rất lớn. Khi đối mặt với dữ liệu phức tạp, câu hỏi đa nghĩa hoặc yêu cầu độ chính xác khắt khe, kiến trúc RAG cơ bản bắt đầu bộc lộ những hạn chế rõ rệt trong khả năng xử lý ngữ cảnh sâu, sự thiếu chính xác khi truy xuất các thông tin đặc thù và thường xuyên khiến mô hình bị quá tải bởi các dữ liệu nhiễu chưa được chất lọc hợp lý. Do đó, bài viết này sẽ tìm hiểu các khái niệm RAG nâng cao và cùng triển khai các thực nghiệm so sánh các phiên bản nâng cấp của một hệ thống RAG trên các tài liệu PDF của AI VIETNAM.



Hình 1: Hình minh hoạ tổng quát hệ thống RAG và các thực nghiệm so sánh.

Mục lục

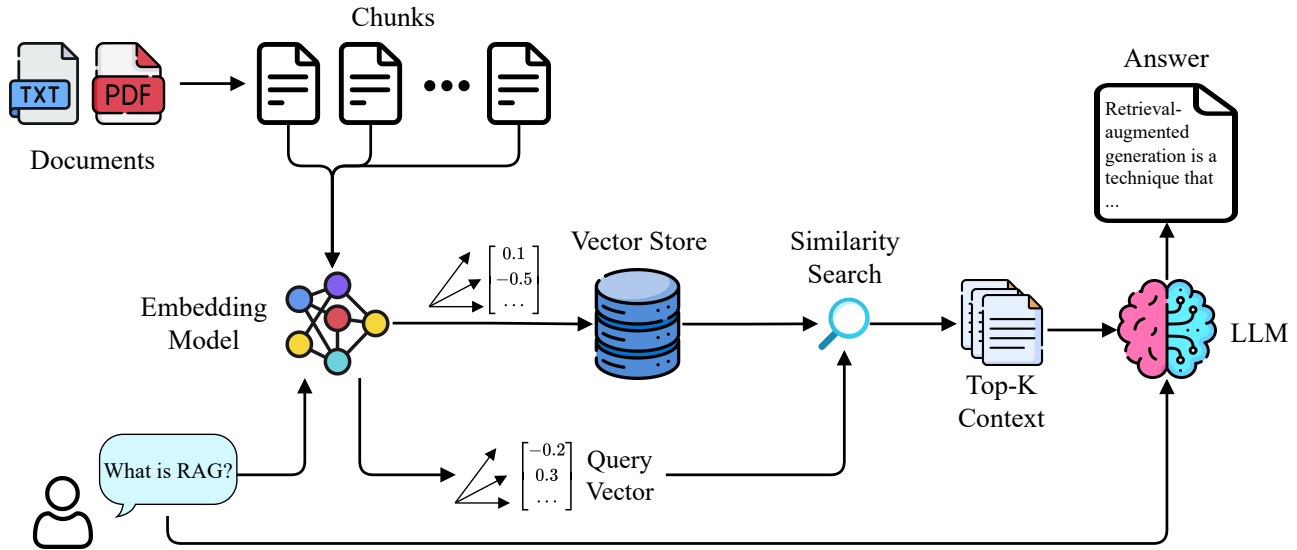
I.	Dẫn nhập	1
II.	Nhắc lại về RAG cơ bản	4
III.	Advanced Indexing	5
III.1.	Semantic Chunking	5
III.2.	Tối ưu hóa Vector Database: HNSW Index	8
IV.	Advanced Retrieval Strategies	11
IV.1.	Hạn chế của Vector Search và vai trò của BM25	11
IV.2.	Hybrid Search	13
IV.3.	Reciprocal rank fusion (RRF)	13
V.	Query Transformations	15
V.1.	Hypothetical document embeddings (HyDE)	15
V.2.	Query Decomposition	16
VI.	Re-ranking	17
VI.1.	Cross-Encoder	17
VI.2.	Maximal marginal relevance (MMR)	19
VII.	Thực hành	20
VII.1.	Định nghĩa bài toán	20
VII.2.	Bộ công cụ đánh giá – Ragas	21
VII.3.	Nguồn dữ liệu và bộ dữ liệu đánh giá	25
VII.4.	Baseline hệ thống RAG cơ bản	31
VII.5.	Thực nghiệm	45
VII.6.	Tổng kết thực nghiệm	72
VIII.	Câu hỏi trắc nghiệm	74
	Phụ lục	78

Bảng thuật ngữ

Thuật ngữ	Mô tả
Hallucination	Hiện tượng mô hình sinh ra thông tin sai lệch, bịa đặt hoặc không có thực nhưng với văn phong tự tin.
Recursive Chunking	Phương pháp chia nhỏ văn bản truyền thống dựa trên các ký tự phân tách như xuống dòng, dấu chấm.
Semantic Chunking	Kỹ thuật chia nhỏ văn bản dựa trên sự thay đổi về ngữ nghĩa thay vì cắt theo số lượng ký tự cố định.
ANN	Approximate Nearest Neighbor - Thuật toán tìm kiếm láng giềng gần nhất xấp xỉ, giúp tìm kiếm nhanh chóng vector lớn thay vì duyệt tuần tự.
HNSW	Hierarchical navigable small world - Thuật toán tìm kiếm vector xấp xỉ dựa trên cấu trúc đồ thị phân tầng, giúp cân bằng giữa tốc độ và độ chính xác.
Dense Retrieval	Phương pháp tìm kiếm dựa trên vector embedding, tập trung vào sự tương đồng về ngữ nghĩa.
Sparse Retrieval	Phương pháp tìm kiếm dựa trên từ khóa, tiêu biểu là thuật toán BM25.
BM25	Best matching 25 - Thuật toán xếp hạng tài liệu dựa trên thống kê tần suất từ khóa, là phiên bản nâng cấp của TF-IDF với cơ chế bão hòa và chuẩn hóa độ dài.
Hybrid Search	Chiến lược kết hợp kết quả từ cả Dense Retrieval và Sparse Retrieval để tận dụng ưu điểm của cả hai.
RRF	Reciprocal rank fusion - Thuật toán hợp nhất các danh sách kết quả tìm kiếm dựa trên thứ hạng thay vì điểm số.
HyDE	Hypothetical document embeddings - Kỹ thuật sử dụng LLM để sinh ra câu trả lời giả định, sau đó dùng vector của câu trả lời này để tìm kiếm tài liệu thực tế.
Bi-Encoder	Kiến trúc mô hình mã hóa câu hỏi và tài liệu thành hai vector riêng biệt, tối ưu cho tốc độ tìm kiếm.
Cross-Encoder	Kiến trúc mô hình xử lý câu hỏi và tài liệu cùng lúc để chấm điểm mức độ liên quan, có độ chính xác cao nhưng tốc độ chậm, thường dùng cho Re-ranking.
Re-ranking	Giai đoạn tái xếp hạng các tài liệu đã tìm được, nhằm lọc nhiễu và đẩy kết quả chính xác lên đầu trước khi gửi LLM.
MMR	Maximal marginal relevance - Thuật toán lựa chọn tài liệu giúp cân bằng mức độ liên quan và sự đa dạng thông tin.

II. Nhắc lại về RAG cơ bản

Trước khi tìm hiểu các kỹ thuật nâng cao, chúng ta cần ôn lại kiến trúc RAG tiêu chuẩn đã được xây dựng trong bài viết trước. Việc hiểu rõ luồng hoạt động này là tiền đề cốt yếu để xác định chính xác các điểm nghẽn cần tối ưu hóa của hệ thống.



Hình 2: Sơ đồ luồng dữ liệu và các thành phần trong kiến trúc RAG cơ bản.

Retrieval-augmented generation (RAG) là giải pháp kiến trúc nhằm khắc phục hai giới hạn cố hữu của LLM: *hallucination* và *sự thiếu hụt tri thức* cập nhật và riêng tư.

Thay vì phụ thuộc hoàn toàn vào bộ nhớ tham số được cố định sau khi huấn luyện, RAG trang bị cho LLM khả năng tra cứu thời gian thực từ các bộ nhớ ngoài. Một pipeline RAG hoàn chỉnh vận hành dựa trên sự phối hợp của 3 giai đoạn chính như Hình 2:

1. Indexing: Đây là giai đoạn chuẩn bị dữ liệu. Mục tiêu là chuyển đổi tri thức thô thành dạng biểu diễn mà máy tính có thể tìm kiếm hiệu quả.

- Input: Dữ liệu thô đa dạng (PDF, Docx, Text, Markdown...).
- Hoạt động:
 - Load & Extract: Trích xuất văn bản thuần túy từ các định dạng tệp tin.
 - Chunking: Chia nhỏ văn bản thành các đoạn gọi là chunk, có độ dài phù hợp với context window của mô hình.
 - Embedding: Sử dụng embedding model để mã hóa các chunk thành các vector số thực trong không gian ngữ nghĩa n-chiều.
- Output: Cơ sở dữ liệu Vector lưu trữ các vector kèm theo metadata tương ứng.

2. Retrieval: Đây là giai đoạn xử lý thời gian thực, được kích hoạt khi người dùng gửi câu hỏi.

- Hoạt động:
 1. Câu hỏi được đưa qua cùng một embedding model để tạo ra vector query (q).
 2. Hệ thống thực hiện tìm kiếm tương đồng giữa vector q và toàn bộ vector trong database.
- Output: Top- K chunk có độ tương đồng cao nhất, được kỳ vọng chứa thông tin trả lời cho câu hỏi.

3. Generation: Giai đoạn tổng hợp cuối cùng, nơi sức mạnh ngôn ngữ của LLM được phát huy.

- Hoạt động: Các chunk tìm được ở bước Retrieval được ghép nối lại để tạo thành context cùng với câu hỏi gốc được đưa vào một prompt theo cấu trúc định sẵn.
- Output: LLM xử lý prompt, tổng hợp thông tin từ context và sinh ra câu trả lời tự nhiên, chính xác và có căn cứ xác thực.

III. Advanced Indexing

Trong bài viết trước, chúng ta đã tiếp cận giai đoạn Indexing theo cách đơn giản nhất:

1. Fixed-size Chunking: Văn bản được cắt một cách rập khuôn cứ đủ 500 hoặc 1000 ký tự thì ngắt đoạn, kể cả khi câu văn hay ý tưởng đang diễn đạt dở dang.
2. Flat Indexing: Toàn bộ vector được lưu vào một danh sách dài. Khi tìm kiếm, hệ thống phải so sánh câu hỏi với từng vector một trong danh sách đó, còn gọi là Brute-force.

Cách làm này hoạt động tốt với vài trăm tài liệu. Tuy nhiên, khi hệ thống mở rộng lên gấp nhiều lần, hai vấn đề nghiêm trọng sẽ nảy sinh:

- Mất mát ngữ nghĩa: Việc chunking máy móc vô tình làm gãy mạch văn. Một ý tưởng trọn vẹn bị tách thành hai chunk vô nghĩa, khiến LLM không hiểu được ngữ cảnh.
- Độ trễ cao: Việc duyệt tuần tự qua hàng triệu vector quá chậm để đáp ứng thời gian thực.

Phần này sẽ giới thiệu hai kỹ thuật để giải quyết vấn đề trên: Semantic Chunking để cắt văn bản thông minh hơn và HNSW để tìm kiếm nhanh hơn.

III.1. Semantic Chunking

Vì chất lượng của dữ liệu đầu vào quyết định chất lượng đầu ra nên mục tiêu của chúng ta là chia văn bản thành các đoạn *đủ nhỏ* để tìm kiếm chính xác, nhưng vẫn phải *đủ lớn* để bao hàm trọn vẹn một ý tưởng.

Thay vì cắt theo số ký tự cố định, phương pháp Semantic Chunking đọc hiểu nội dung để quyết định điểm cắt.

Ý tưởng cốt lõi

Giả sử ta đang đọc một bài báo.

- Khi các câu văn liên tiếp cùng bàn về một chủ đề, vector của chúng sẽ nằm gần nhau trong không gian biểu diễn.
- Khi các câu văn hoặc nội dung chuyển sang chủ đề mới, vector sẽ đột ngột thay đổi hướng, tạo ra khoảng cách lớn.

→ Semantic Chunking phát hiện sự thay đổi này để thực hiện ngắt đoạn chính xác tại điểm giao giữa hai chủ đề.

Quy trình thực hiện:

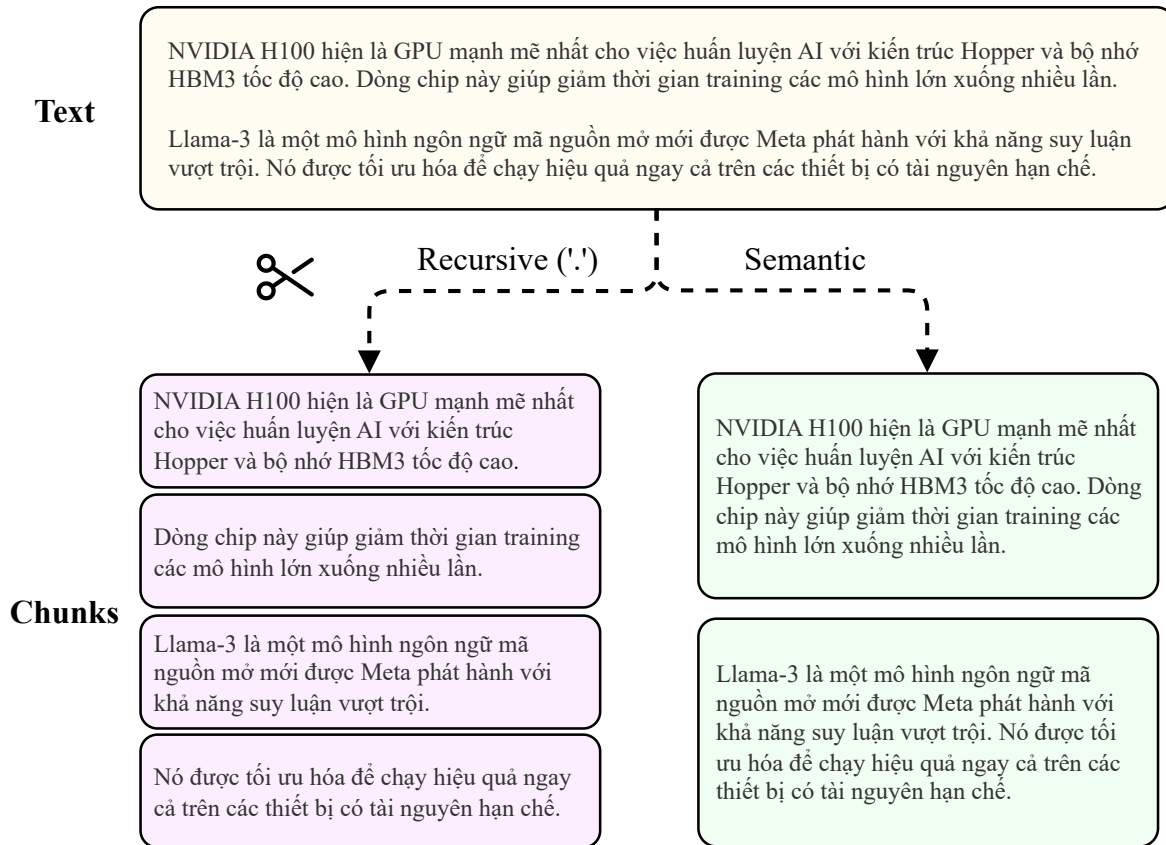
1. **Sentence Splitting:** Chia văn bản thành các câu hoàn chỉnh dựa trên dấu chấm câu.
2. **Similarity Calculation:** Tính toán độ tương đồng, ví dụ cosine similarity, giữa câu trước và câu sau.
3. **Đặt ngưỡng:**
 - Nếu độ tương đồng cao: Hai câu cùng chủ đề → Gộp chung vào một chunk.
 - Nếu độ tương đồng giảm sâu xuống dưới mức ngưỡng: Chủ đề đã thay đổi → Ngắt chunk tại đây và bắt đầu chunk mới.

III.1.1. So sánh: Recursive vs. Semantic Chunking

Để hình dung rõ hơn tác động của chiến lược chunking đến chất lượng dữ liệu, hãy xem xét một ví dụ cụ thể dưới đây.

Dữ liệu đầu vào: Một đoạn văn bản chứa hai luồng thông tin riêng biệt:

- Phần đầu: Giới thiệu về phần cứng (GPU NVIDIA H100).
- Phần sau: Giới thiệu về mô hình ngôn ngữ (Llama-3).



Hình 3: So sánh kết quả phân mảnh: Recursive cắt vụn thông tin (trái) trong khi Semantic bảo toàn ngữ nghĩa (phải).

Phân tích kết quả:

- **Recursive Chunking:** Do ngắt theo dấu chấm câu, văn bản được tách thành 4 chunk nhỏ.

Hệ quả: Thông tin bị đứt gãy. Ví dụ, ở Chunk 2 chứa thông tin “giúp giảm thời gian training”, nhưng chủ ngữ “NVIDIA H100” lại nằm ở Chunk 1. Nếu hệ thống chỉ tìm thấy Chunk 2, LLM sẽ không thể biết đối tượng đang được nhắc đến là gì, dẫn đến câu trả lời thiếu chính xác.

- **Semantic Chunking:** Thuật toán phát hiện được điểm gãy ngữ nghĩa và thực hiện cắt đúng lúc.

Hệ quả: Chúng ta thu được 2 chunk trọn vẹn. Chunk đầu chứa toàn bộ thông tin về H100, Chunk thứ hai chứa toàn bộ thông tin về Llama-3. Tính toàn vẹn của dữ liệu được đảm bảo.

Từ ví dụ trực quan trên, ta có thể so sánh ưu nhược điểm của hai phương pháp này trong bảng dưới đây:

Tiêu chí	Recursive Chunking	Semantic Chunking
Nguyên lý	Cắt dựa trên dấu câu (\n, dấu chấm) và số lượng ký tự cố định.	Cắt dựa trên sự thay đổi về ý nghĩa nội dung giữa các câu.
Ưu điểm	- Tốc độ cực nhanh, chi phí thấp. - Giữ nguyên cấu trúc văn bản gốc.	- Giữ trọn vẹn ý tưởng, không làm gãy mạch văn. - Tăng độ chính xác khi tìm kiếm.
Nhược điểm	- Dễ cắt ngang ý tưởng quan trọng. - Context có thể bị nhiễu vì chứa 2 nửa của 2 ý khác nhau.	- Tốn tài nguyên tính toán do phải chạy model để so sánh từng câu.
Phù hợp với	Tài liệu có cấu trúc rõ ràng, ví dụ hợp đồng, luật, sách giáo khoa ...	Tài liệu dạng tường thuật, ít cấu trúc như biên bản cuộc họp, transcript video ...

Bảng 1: Bảng so sánh chiến lược Chunking.

III.2. Tối ưu hóa Vector Database: HNSW Index

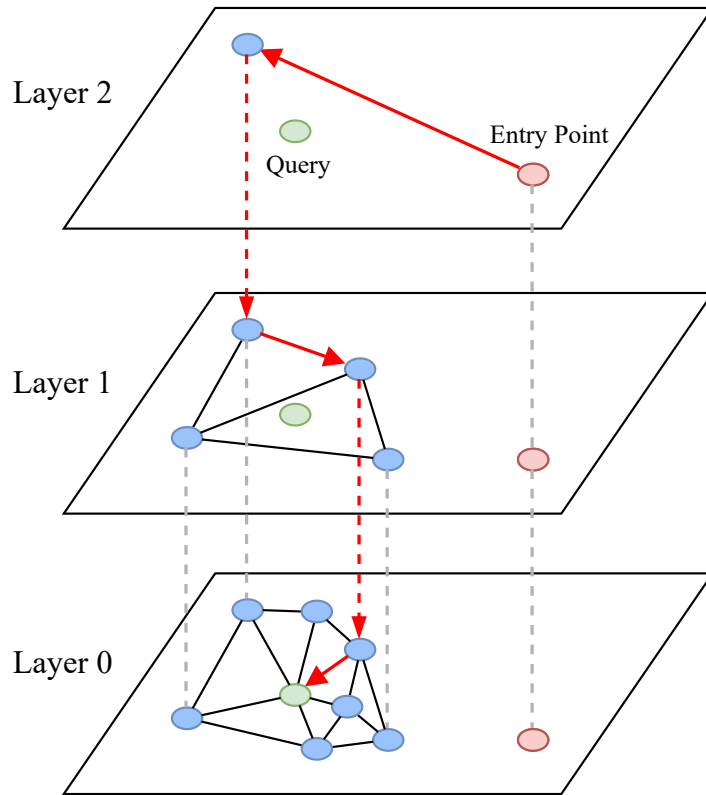
Khi làm việc với tập dữ liệu vector quy mô lớn, việc tìm kiếm vét cạn bằng cách tính khoảng cách tới từng điểm dữ liệu trở nên bất khả thi về mặt thời gian. Để giải quyết vấn đề này, giải pháp tiêu chuẩn là sử dụng các thuật toán tìm kiếm láng giềng gần nhất xấp xỉ thuộc nhóm approximate nearest neighbor (ANN).

Hiện nay, hierarchical navigable small world (HNSW) được xem là một trong các thuật toán hiệu quả, cân bằng tốt giữa tốc độ truy xuất và độ chính xác của kết quả.

III.2.1. Cơ chế hoạt động

HNSW tổ chức dữ liệu dưới dạng một cấu trúc đồ thị nhiều tầng. Thay vì duyệt tất cả các phần tử, thuật toán sử dụng cấu trúc phân cấp áp dụng trên đồ thị:

- Layer 0: Chứa toàn bộ các điểm dữ liệu, và các liên kết chi tiết nhất giữa chúng. Đây là tầng chứa thông tin đầy đủ nhất để tìm chính xác vector đích.
- Các layer cao hơn: Là các phiên bản rút gọn của layer 0. Các node tại đây thưa thớt hơn và các liên kết có độ dài lớn hơn, đóng vai trò như các đường dẫn tắt giúp thuật toán di chuyển nhanh qua không gian dữ liệu rộng lớn.



Quá trình tìm kiếm:

1. Thuật toán bắt đầu từ một điểm entry point ở layer cao nhất.
2. Tại mỗi tầng, nó di chuyển đến node láng giềng gần nhất với query vector cho đến khi đạt cực trị địa phương tại tầng đó.
3. Node tìm được sẽ trở thành entry point cho tầng tiếp theo bên dưới.
4. Quá trình lặp lại cho đến khi xuống tới Layer 0, nơi thuật toán thực hiện tìm kiếm cục bộ chính xác để trả về kết quả cuối cùng.

Hình 4: Cấu trúc đồ thị phân tầng của HNSW. Tìm kiếm bắt đầu từ các liên kết thừa ở tầng trên và tinh chỉnh dần độ chính xác tại tầng đáy.

III.2.2. Các tham số cấu hình quan trọng

Khi triển khai HNSW trên các cơ sở dữ liệu vector phổ biến như ChromaDB, Milvus hay Qdrant, hiệu năng của hệ thống phụ thuộc hoàn toàn vào việc cân chỉnh ba tham số cốt lõi. Việc hiểu rõ tác động của từng tham số sẽ giúp ta tối ưu hóa được sự cân bằng giữa tài nguyên phần cứng và chất lượng tìm kiếm.

1. Tham số M (Max Links per Node) quy định số lượng liên kết tối đa mà một node có thể tạo ra với các node láng giềng khác trong đồ thị.

- **Ý nghĩa:** Quyết định mật độ kết nối của đồ thị. Giá trị M càng lớn nghĩa là mạng lưới càng chằng chịt, các điểm dữ liệu có nhiều đường dẫn để đi tới nhau hơn.
- **Tác động:** Một giá trị M cao sẽ giúp thuật toán không bị mắc kẹt ở các cực trị địa phương, từ đó tăng độ chính xác của kết quả tìm kiếm. Tuy nhiên, lượng bộ nhớ RAM tiêu thụ sẽ tăng lên đáng kể để lưu trữ các liên kết này và thời gian để thêm mới dữ liệu cũng lâu hơn.

2. Tham số ef_construction (Construction Search Depth) kiểm soát kích thước của danh sách các ứng viên tiềm năng được thuật toán xem xét trong quá trình xây dựng chỉ mục.

- **Ý nghĩa:** Khi thêm một điểm dữ liệu mới vào đồ thị, thuật toán cần tìm xem những node nào đang tồn tại là láng giềng tốt nhất để tạo liên kết. `ef_construction` càng lớn nghĩa là thuật toán càng kỹ tính, nó sẽ duyệt qua nhiều ứng viên hơn để đảm bảo tìm được những liên kết tối ưu nhất.
- **Tác động:** Giá trị cao giúp tạo ra một cấu trúc đồ thị chất lượng cao, đảm bảo việc tìm kiếm sau này diễn ra thuận lợi và chính xác. Đổi lại, quá trình nạp dữ liệu sẽ diễn ra chậm hơn do thực hiện nhiều phép tính toán hơn cho mỗi điểm dữ liệu mới.

3. Tham số `ef_search` (Runtime Search Depth) Tương tự như tham số trên, nhưng được áp dụng tại thời điểm thực thi truy vấn. Nó quy định số lượng láng giềng gần nhất mà thuật toán sẽ lưu giữ trong hàng đợi ưu tiên khi tìm kiếm câu trả lời.

- **Ý nghĩa:** Tham số này quyết định phạm vi quét của thuật toán.
- **Tác động:** Đây là yếu tố quan trọng để đánh đổi trực tiếp giữa tốc độ và độ chính xác:
 - Tăng `ef_search` sẽ mở rộng phạm vi tìm kiếm, giúp hệ thống không bỏ sót các kết quả đúng, nhưng thời gian phản hồi sẽ chậm lại.
 - Giảm `ef_search` giúp hệ thống phản hồi cực nhanh, nhưng có rủi ro trả về các kết quả không sát với câu hỏi nhất.

i Chiến lược cấu hình thực tế

Không có một bộ tham số nào là hoàn hảo cho mọi bài toán. Chúng ta cần điều chỉnh dựa trên yêu cầu nghiệp vụ:

Trường hợp 1: Ứng dụng Chatbot thời gian thực

- Yêu cầu phản hồi nhanh, chấp nhận sai số nhỏ.
- Cấu hình: Giữ `ef_search` ở mức thấp, ví dụ: 50 - 100, để tối ưu độ trễ.

Trường hợp 2: Hệ thống tra cứu chuyên sâu

- Yêu cầu độ chính xác tuyệt đối, không được phép bỏ sót thông tin.
- Cấu hình: Thiết lập `M` cao, ví dụ: 64, và tăng `ef_search` lên mức cao, ví dụ: 400 - 500. Chấp nhận tốn RAM và thời gian phản hồi chậm hơn một chút để đảm bảo độ chính xác đạt mức tối đa.

IV. Advanced Retrieval Strategies

Trong phần trước, chúng ta đã tìm hiểu các cách tối ưu hóa lưu trữ dữ liệu. Tuy nhiên, dù dữ liệu được tổ chức tốt đến đâu, nếu phương pháp tìm kiếm không phù hợp, kết quả trả về vẫn có thể thiếu chính xác.

Ở kiến trúc RAG cơ bản, chúng ta phụ thuộc hoàn toàn vào *Vector Search*. Mặc dù rất mạnh trong việc nắm bắt ngữ nghĩa, phương pháp này lại bộc lộ điểm yếu khi gặp các truy vấn đòi hỏi độ chính xác tuyệt đối về mặt câu chữ, ví dụ như tìm kiếm tên riêng, mã sản phẩm hoặc các thuật ngữ chuyên ngành hiếm gặp.

Để khắc phục, chiến lược hiện đại là kết hợp sức mạnh của tìm kiếm ngữ nghĩa với tìm kiếm từ khóa truyền thống, tạo nên cơ chế *Hybrid Search*.

IV.1. Hạn chế của Vector Search và vai trò của BM25

Hãy xem xét một ví dụ đơn giản: Người dùng muốn tìm thông tin về lỗi “*Error 503 Service Unavailable*”.

- **Vector Search:** Có xu hướng tìm các đoạn văn có ý nghĩa tương đương như “*máy chủ bị quá tải*”, “*kết nối mạng thất bại*”. Đôi khi, nó bỏ qua chính xác con số “503” vì trong không gian vector, các con số này có thể không mang nhiều ý nghĩa ngữ nghĩa đặc trưng.
- **Keyword Search:** Sẽ tìm chính xác các văn bản có chứa chuỗi ký tự “Error” và “503”. Đây chính là lúc chúng ta cần đến các thuật toán tìm kiếm dựa trên tần suất từ khóa, mà tiêu biểu nhất là BM25.

IV.1.1. Thuật toán best matching 25 (BM25)

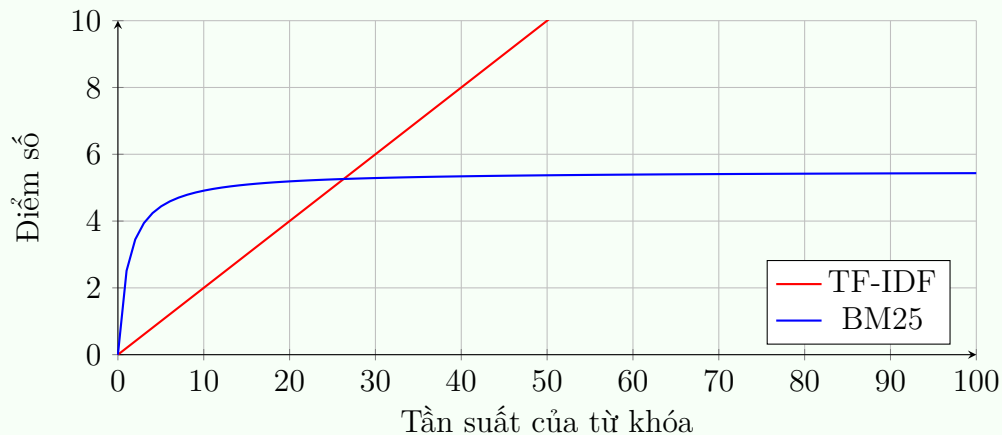
BM25 được xem là tiêu chuẩn vàng trong lĩnh vực truy hồi thông tin cổ điển. Dù hiện nay các mô hình Vector Search đang chiếm ưu thế, BM25 vẫn đóng vai trò không thể thay thế trong các hệ thống Hybrid Search nhờ khả năng khớp từ khóa chính xác.

Đây là phiên bản nâng cấp hoàn thiện của TF-IDF, được thiết kế để khắc phục các nhược điểm về spam từ khóa và độ dài văn bản.

Cơ chế của BM25

BM25 chấm điểm độ phù hợp của tài liệu dựa trên 3 nguyên lý thống kê cốt lõi:

1. **TF Saturation (Bão hòa tần suất):** Trong TF-IDF, nếu một từ xuất hiện càng nhiều, điểm số tăng tuyến tính vô hạn, ví dụ xuất hiện 100 lần thì điểm cao gấp 100 lần.



Hình 5: So sánh đồ thị điểm khi TF-IDF tăng mãi mãi, còn BM25 chỉ tiệm cận giới hạn nào đó.

Cải tiến của BM25: Áp dụng cơ chế bão hòa. Nếu từ khóa xuất hiện thêm lần thứ 2, thứ 3, điểm số sẽ tăng đáng kể. Nhưng nếu nó đã xuất hiện 100 lần, thì việc xuất hiện thêm lần thứ 101 hầu như không làm tăng thêm điểm số, như Hình 5. Điều này giúp chống lại việc spam từ khóa.

- IDF (Nghịch đảo tần suất):** Phạt nặng các từ phổ biến và thưởng lớn cho các từ hiếm.

Ví dụ: Trong câu truy vấn “*triệu chứng của bệnh viêm màng não*”, từ “*viêm màng não*” quan trọng hơn gấp nhiều lần so với từ “*của*”.

- Length Normalization:** Đây là điểm BM25 thông minh hơn hẳn. Nếu từ khóa xuất hiện 1 lần trong một đoạn văn ngắn, nó sẽ được đánh giá cao hơn là xuất hiện 1 lần trong một cuốn tiểu thuyết dài vì loãng thông tin.

💡 Ví dụ minh họa: Tại sao cần “Bão hòa”?

Giả sử người dùng tìm kiếm từ khóa: “Galaxy”.

Ta có hai tài liệu:

- Doc A:** “Galaxy Galaxy Galaxy Galaxy...” (Lặp lại 1000 lần nhằm spam để lên top).
- Doc B:** “Samsung vừa ra mắt điện thoại Galaxy mới...” (Từ Galaxy xuất hiện 5 lần trong ngữ cảnh hợp lý).

Cách chấm điểm:

- TF-IDF:** $Score(A) \gg Score(B)$. Doc A chiến thắng áp đảo vì có 1000 từ khóa. → *Kết quả sai lệch.*

- **BM25:** Do cơ chế bão hòa, sau khoảng 10-20 lần xuất hiện, điểm số của Doc A không tăng thêm nữa. Trong khi đó, Doc B được cộng điểm nhờ độ dài văn bản hợp lý.
→ BM25 đưa ra đánh giá công bằng hơn, loại bỏ ưu thế của việc spam từ khóa.

IV.2. Hybrid Search

Hybrid Search không phải là một thuật toán mới, mà là một quy trình kết hợp kết quả từ hai luồng tìm kiếm song song: *Sparse Retriever* và *Dense Retriever*.

Quy trình thực hiện:

1. **Parallel Execution:** Khi nhận được câu hỏi, hệ thống gửi truy vấn đồng thời đến cả hai bộ tìm kiếm.
 - BM25 trả về danh sách *A* khớp về từ khóa chính xác.
 - Vector Search trả về danh sách *B* khớp về ngữ cảnh và từ đồng nghĩa.
2. **Fusion:** Hợp nhất hai danh sách này lại thành một danh sách duy nhất.
3. **Ranking:** Sắp xếp lại thứ tự ưu tiên để đưa ra Top-K văn bản tốt nhất cho LLM.

IV.3. Reciprocal rank fusion (RRF)

Vấn đề khi thực hiện Hybrid Search là thang điểm của hai thuật toán hoàn toàn khác nhau:

- Vector Search thường dùng cosine similarity, điểm số nằm trong khoảng $[0, 1]$.
- BM25 dùng công thức thống kê, điểm số có thể là bất kỳ số dương nào.

Chúng ta không thể cộng trực tiếp điểm số này lại. Để giải quyết, ta sử dụng thuật toán reciprocal rank fusion (RRF).

Nguyên lí

Thay vì quan tâm đến *điểm số*, RRF chỉ quan tâm đến *thứ hạng*. Giả định rằng nếu một tài liệu xuất hiện ở thứ hạng cao trong cả hai danh sách, thì nó chắc chắn là tài liệu quan trọng. Công thức tính điểm RRF cho một tài liệu *d*:

$$Score(d) = \sum_i \frac{1}{k + rank_i(d)}$$

Trong đó:

- $rank_i(d)$: Là thứ hạng của tài liệu *d* trong danh sách *i*, bắt đầu từ 1.

- k : Là hằng số làm mượt, thường chọn $k = 60$. Hằng số này giúp giảm bớt sự chênh lệch điểm số giữa các thứ hạng quá cao, ví dụ Top 1 vs Top 2, đảm bảo tính công bằng.



Ví dụ minh họa: Sức mạnh của RRF

Giả sử hằng số $k = 60$. Ta xét hai tài liệu với thứ hạng khác nhau ở 2 bộ tìm kiếm:

- **Doc A:** Hạng 1 bên Vector vì rất đúng ngữ nghĩa nhưng hạng 10 bên BM25 vì thiếu từ khóa chính xác.
- **Doc B:** Hạng 2 bên Vector và hạng 3 bên BM25, khá tốt ở cả hai mặt.

1. Tính điểm Doc A:

$$Score(A) = \frac{1}{60 + 1} + \frac{1}{60 + 10} = 0.0164 + 0.0143 = \mathbf{0.0307}$$

2. Tính điểm Doc B:

$$Score(B) = \frac{1}{60 + 2} + \frac{1}{60 + 3} = 0.0161 + 0.0159 = \mathbf{0.0320}$$

→ **Kết quả: Doc B xếp trên Doc A** ($0.0320 > 0.0307$).

Mặc dù Doc A đứng Top 1 ở Vector Search, nhưng RRF ưu tiên Doc B hơn vì nó có sự đồng thuận cao từ cả hai thuật toán. RRF giúp hệ thống chọn ra tài liệu hài hòa nhất thay vì chỉ tin vào một phía.

Phương pháp	Ưu điểm	Nhược điểm
Vector Search	Hiểu ngữ nghĩa sâu, tìm được từ đồng nghĩa, đa ngôn ngữ.	Kém trong việc khớp chính xác từ khóa, khó giải thích kết quả.
BM25	Khớp chính xác từ khóa, hiệu quả với tên miền chuyên biệt, tốc độ nhanh.	Không hiểu ngữ cảnh, thất bại nếu người dùng dùng từ đồng nghĩa khác với văn bản.
Hybrid Search	Cân bằng: Đảm bảo không bỏ sót từ khóa quan trọng đồng thời vẫn hiểu được ngữ cảnh.	Phức tạp hơn khi triển khai, tốn tài nguyên hơn do phải chạy song song 2 luồng.

Bảng 2: So sánh các chiến lược tìm kiếm.

V. Query Transformations

Trong các phần trước, chúng ta đã giả định rằng câu hỏi của người dùng luôn rõ ràng, đầy đủ ngữ nghĩa và khớp với nội dung trong tài liệu. Tuy nhiên, thực tế hiếm khi hoàn hảo như vậy.

Người dùng thường có xu hướng đặt những câu hỏi ngắn, thiếu ngữ cảnh hoặc hỏi nhiều vấn đề cùng một lúc. Ví dụ: thay vì hỏi “Quy định về chế độ làm việc từ xa áp dụng từ ngày nào?”, họ có thể chỉ ghi “remote work”. Nếu đưa trực tiếp câu hỏi thô này vào hệ thống RAG, kết quả tìm kiếm thường rất kém vì vector của câu hỏi không khớp với vector của văn bản pháp quy chi tiết.

Để giải quyết vấn đề này, chúng ta sử dụng kỹ thuật *Query Transformation*. Ý tưởng cốt lõi là sử dụng một LLM để viết lại, mở rộng hoặc chia nhỏ câu hỏi của người dùng thành các phiên bản tốt hơn trước khi thực hiện tìm kiếm.

V.1. Hypothetical document embeddings (HyDE)

Một trong những thách thức lớn nhất của Vector Search là sự bất cân xứng về ngữ nghĩa giữa câu hỏi và câu trả lời. Câu hỏi thường ngắn và mang tính chất nghi vấn, trong khi tài liệu lại dài và mang tính chất khẳng định, mô tả.

HyDE là kỹ thuật khắc phục điều này bằng cách tận dụng khả năng sáng tạo của LLM. Thay vì tìm kiếm dựa trên câu hỏi, chúng ta yêu cầu LLM đưa ra một câu trả lời giả định, sau đó dùng câu trả lời giả định này để đi tìm tài liệu thật.

Cơ chế hoạt động

Quy trình HyDE diễn ra qua ba bước:

1. **Generate:** Hệ thống yêu cầu LLM viết một đoạn văn trả lời giả định cho câu hỏi của người dùng. Lưu ý rằng thông tin trong đoạn văn này có thể sai sự thật, nhưng văn phong và các từ vựng chuyên ngành được sử dụng sẽ giống với tài liệu thực tế.
2. **Encode:** Đưa đoạn văn giả định này qua embedding model để tạo ra vector.
3. **Retrieve:** Sử dụng vector này để tìm kiếm trong cơ sở dữ liệu. Vì vector của “câu trả lời giả” sẽ gần với vector của “câu trả lời thật” hơn là vector của “câu hỏi”, nên kết quả tìm kiếm thường chính xác hơn.

Ví dụ minh họa

User Question: “Cách xử lý lỗi màn hình xanh.”

Vấn đề: Câu hỏi quá ngắn, vector có thể khớp nhầm với các tài liệu mô tả màu sắc màn hình.

HyDE Generation (LLM viết nháp): “Để khắc phục lỗi Blue Screen of Death (BSOD) trên Windows, bạn cần khởi động lại máy, kiểm tra mã lỗi stop code, cập nhật driver card đồ họa hoặc vào chế độ Safe Mode để gỡ bỏ phần mềm xung đột...”

Kết quả: Hệ thống dùng đoạn văn nháp trên để tìm kiếm. Nhờ các từ khóa chuyên môn như “BSOD”, “driver”, “Safe Mode” xuất hiện trong đoạn nháp, hệ thống dễ dàng tìm thấy tài liệu hướng dẫn kỹ thuật chính xác trong database.

V.2. Query Decomposition

Kỹ thuật này đặc biệt hữu ích đối với các câu hỏi phức tạp mà một đoạn văn bản duy nhất không thể chứa đủ thông tin để trả lời.

Nếu người dùng hỏi một câu cần so sánh hoặc tổng hợp thông tin từ nhiều nguồn, việc tìm kiếm đơn thuần thường thất bại vì vector của câu hỏi sẽ nằm lưỡng lự giữa các chủ đề khác nhau. Query Decomposition giải quyết việc này bằng cách chia nhỏ vấn đề lớn thành các bài toán con đơn giản hơn.

Chiến lược

Hệ thống sử dụng LLM để phân tích câu hỏi gốc và tách nó thành một chuỗi các câu hỏi phụ độc lập.

1. **Breakdown:** Tách câu hỏi đa ý thành các câu hỏi đơn ý.
2. **Retrieval:** Thực hiện tìm kiếm tài liệu cho từng câu hỏi phụ riêng biệt. Điều này đảm bảo mỗi lần tìm kiếm đều có mục tiêu rõ ràng và độ chính xác cao.
3. **Synthesis:** Tổng hợp các đoạn văn bản tìm được từ tất cả các bước trên và đưa cho LLM để trả lời câu hỏi gốc ban đầu.

Ví dụ minh họa

User Question: “So sánh doanh thu của iPhone 15 và Samsung S24 trong quý 1 năm 2024.”

Vấn đề: Không có một tài liệu nào chứa sẵn bảng so sánh này. Thông tin nằm rải rác ở báo cáo tài chính của Apple và báo cáo của Samsung.

Decomposition Process:

- **Sub-query 1:** “Doanh thu iPhone 15 quý 1 năm 2024 là bao nhiêu?” → Tìm thấy trong Báo cáo Apple.

- **Sub-query 2:** “Doanh thu Samsung S24 quý 1 năm 2024 là bao nhiêu?” → Tìm thấy trong Báo cáo Samsung.

Final Generation: LLM nhận được cả hai con số từ hai lần tìm kiếm và tự tổng hợp thành câu trả lời so sánh hoàn chỉnh.

Tóm lại, Query Transformation đóng vai trò như một người biên tập viên thông minh, giúp chỉnh sửa và định hướng lại câu hỏi của người dùng trước khi gửi đến bộ phận tra cứu, đảm bảo rằng hệ thống luôn hiểu đúng ý định thực sự đằng sau những câu lệnh ngắn gọn.

VI. Re-ranking

Sau khi thực hiện các bước Indexing và Retrieval, chúng ta thu được một danh sách Top-K tài liệu tiềm năng. Tuy nhiên, danh sách này thường chưa đủ tối ưu để đưa trực tiếp vào LLM vì hai lý do chính:

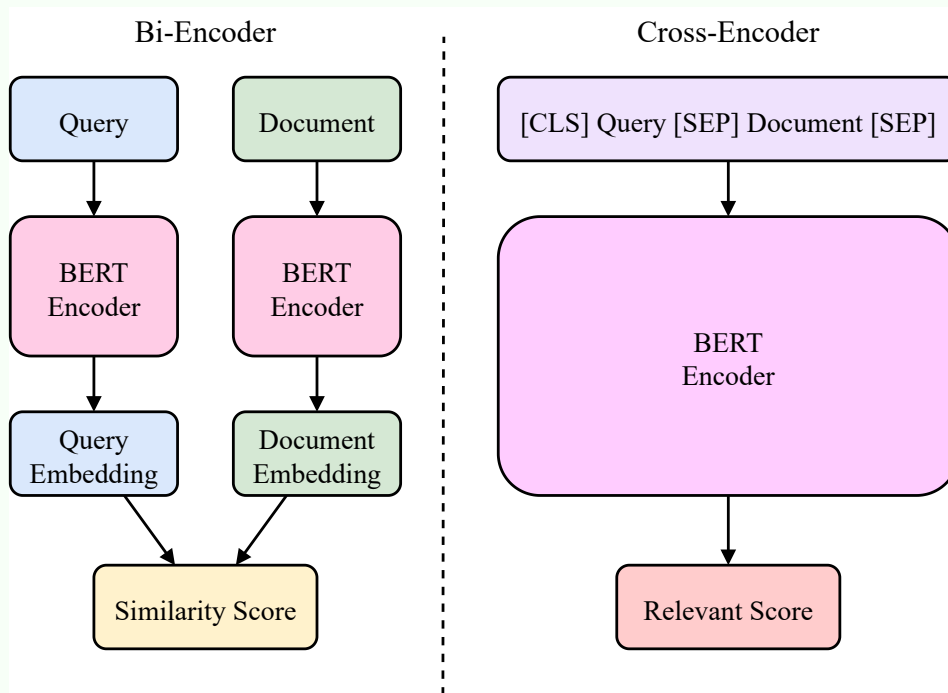
1. **Độ chính xác ngữ nghĩa hạn chế:** Các mô hình Embedding ưu tiên tối ưu hóa tốc độ truy xuất trên lượng dữ liệu lớn, do đó buộc phải đánh đổi khả năng thấu hiểu các mối quan hệ ngữ nghĩa phức tạp giữa câu hỏi và văn bản.
2. **Nhiều thông tin:** Các tài liệu được tìm thấy có thể chứa nhiều từ khóa khớp với câu hỏi nhưng lại sai lệch về ngữ cảnh hoặc ý định thực sự, dẫn đến việc cung cấp thông tin chưa chính xác cho LLM.

Re-ranking đóng vai trò như một bộ lọc ở bước cuối cùng. Chúng ta chấp nhận tốn thêm chút thời gian để “đọc kỹ” lại một tập nhỏ các ứng viên, ví dụ: 50 tài liệu, và chọn ra những tài liệu xuất sắc nhất, ví dụ: 5 tài liệu, gửi cho LLM.

VI.1. Cross-Encoder

Ở bước Retrieval, chúng ta đã sử dụng các mô hình Embedding để mã hóa câu hỏi và văn bản thành vector. Về mặt kiến trúc, phương pháp này được gọi là Bi-Encoder. Để hiểu tại sao chúng ta cần thêm Cross-Encoder ở bước này, hãy so sánh cơ chế hoạt động của chúng.

i Bi-Encoder vs. Cross-Encoder



Hình 6: So sánh sự khác biệt cơ bản giữa hai kiến trúc.

- **Bi-Encoder được dùng khi tìm kiếm:** Xử lý câu hỏi và tài liệu *riêng biệt*. Nó tạo ra vector cho câu hỏi, vector cho tài liệu, rồi tính khoảng cách.

Ưu điểm: Tốc độ nhanh, có thể thực hiện tính toán trước.

Nhược điểm: Mất đi các thông tin tương tác chi tiết giữa từng từ trong câu hỏi và từng từ trong tài liệu.

- **Cross-Encoder dùng trong Re-ranking:** Câu hỏi và tài liệu được ghép nối thành một chuỗi văn bản duy nhất như Hình 6. Chuỗi này được đưa vào mô hình để xử lý cùng lúc. Mô hình sẽ “đọc” song song và xem xét sự tương tác giữa mọi từ của câu hỏi với mọi từ của tài liệu thông qua cơ chế Self-Attention đầy đủ.

Ưu điểm: Độ chính xác cao, hiểu được sắc thái, phủ định và logic phức tạp.

Nhược điểm: Rất chậm và tốn tài nguyên, không thể dùng để tìm kiếm trên toàn bộ database.

Chiến lược hình phễu: Vì Cross-Encoder chậm, ta không áp dụng nó cho toàn bộ dữ liệu. Quy trình chuẩn trong thực tế là:

1. **Retrieve:** Dùng Bi-Encoder để lấy nhanh Top 50 tài liệu từ hàng triệu tài liệu.
2. **Re-rank:** Dùng Cross-Encoder để chấm điểm lại Top 50 tài liệu này.
3. **Select:** Lấy Top 5 tài liệu có điểm cao nhất sau khi chấm lại để đưa vào context cho LLM.

💡 Ví dụ về sức mạnh của Cross-Encoder

Query: “Python không ăn gì?”

Bi-Encoder: Có thể trả về các tài liệu chứa từ khóa “Python”, “ăn”, “thức ăn” như: “Cách cho trăn Python ăn chuột”, “Thức ăn của loài trăn”. Có thể tìm kiếm sai ý định vì chỉ bắt từ khóa.

Cross-Encoder: Khi đọc cả câu hỏi và tài liệu cùng lúc, nó nhận ra cấu trúc phủ định “không ăn” và ngữ cảnh sinh học, từ đó xếp hạng cao tài liệu: “Trăn Python có thể nhin ăn nhiều tháng...”.

VI.2. Maximal marginal relevance (MMR)

Đôi khi, việc tìm ra các tài liệu giống nhất lại không phải là điều tốt nhất.

Giả sử người dùng hỏi: “*Tiểu sử của Steve Jobs*”. Nếu chỉ dựa vào độ tương đồng, hệ thống có thể trả về 5 đoạn văn bản gần như y hệt nhau, đều nói về việc ông thành lập Apple năm 1976. Điều này làm lãng phí context window của LLM mà không cung cấp thêm thông tin mới.

MMR giải quyết vấn đề này bằng cách cân bằng giữa hai yếu tố:

- **Relevance:** Tài liệu phải liên quan đến câu hỏi.
- **Diversity:** Tài liệu phải khác biệt với các tài liệu đã được chọn trước đó.

📌 Nguyên lý hoạt động của MMR

1. Chọn tài liệu có độ tương đồng cao nhất với câu hỏi.
2. Tìm tài liệu tiếp theo sao cho nó vừa *giống câu hỏi*, lại vừa *ít giống nhất* với tài liệu đã chọn ở Bước 1.
3. Lặp lại cho đến khi đủ Top-K tài liệu.

Công thức đơn giản hóa:

$$Score = \lambda \cdot (Sim(Query, Doc)) - (1 - \lambda) \cdot (Sim(Doc, SelectedDocs))$$

Trong đó λ là tham số điều chỉnh, thường là 0.5. Nếu λ càng nhỏ, hệ thống càng ưu tiên sự đa dạng.

💡 Ví dụ minh họa MMR

Query: “Các tính năng của xe VinFast VF8.”

Kết quả nếu không có MMR:

- Doc 1: VF8 có động cơ điện mạnh mẽ 402 mã lực.
- Doc 2: Công suất tối đa của VF8 là 300kW (tương đương 402 mã lực).
- Doc 3: Động cơ VF8 cho khả năng tăng tốc ấn tượng.

→ Thông tin bị lặp lại hoàn toàn về động cơ.

Kết quả khi dùng MMR:

- Doc 1: VF8 có động cơ điện mạnh mẽ 402 mã lực. (Chọn vì giống câu hỏi nhất).
- Doc 2: Hệ thống trợ lái ADAS trên VF8 bao gồm cảnh báo lệch làn... (Chọn vì khác nội dung Doc 1).
- Doc 3: Chính sách bảo hành 10 năm cho pin của VF8. (Chọn vì khác Doc 1 và Doc 2).

→ LLM có đủ dữ liệu để trả lời toàn diện về động cơ, an toàn và hậu mãi.

Tóm lại, việc lựa chọn phương pháp Re-ranking phụ thuộc vào mục tiêu: Nếu bạn cần câu trả lời cực kỳ chính xác cho các câu hỏi khó, hãy dùng Cross-Encoder. Nếu bạn cần câu trả lời tổng quan, bao quát nhiều khía cạnh, hãy dùng MMR.

VII. Thực hành

Phần này sẽ tiến hành các thực nghiệm nhằm đánh giá hiệu quả thực tế của các kỹ thuật nâng cao được nhắc tới ở phần lý thuyết. Thông qua việc so sánh trực tiếp với mô hình RAG cơ bản, các thí nghiệm này sẽ làm rõ sự đánh đổi giữa chi phí tính toán và độ chính xác của từng phương pháp. Từ đó, chúng ta có thể đúc kết được những chiến lược cấu hình phù hợp nhất cho hệ thống thực tế.

VII.1. Định nghĩa bài toán

Mục tiêu của phần thực hành này là xây dựng và đánh giá hiệu năng của hệ thống RAG trong ngữ cảnh khai thác tri thức chuyên ngành. Cụ thể, bài toán tập trung vào việc tự động hóa quá trình trả lời câu hỏi dựa trên kho tài liệu học thuật của chương trình AI VIETNAM.

Bài toán được mô tả cụ thể như sau:

- **Dữ liệu đầu vào:** Tập hợp 45 tài liệu định dạng PDF chứa kiến thức chuyên sâu về Trí tuệ nhân tạo thuộc khóa học AIO.
- **Nhiệm vụ:** Với một câu truy vấn q từ người dùng và tập dữ liệu D , hệ thống cần truy xuất tập hợp các đoạn văn bản liên quan $C \subset D$ và sinh ra câu trả lời a chính xác, đầy đủ thông tin.
- **Mục tiêu đánh giá:** So sánh hiệu quả thực tế giữa mô hình RAG cơ bản và các kỹ thuật nâng cao. Quá trình này được thực hiện định lượng thông qua framework Ragas dựa trên 4 tiêu chí cốt lõi: Faithfulness, Answer Relevancy, Context Precision và Context Recall.

Việc định nghĩa rõ bài toán là tiền đề để xây dựng bộ dữ liệu kiểm thử chuẩn hóa và thiết lập các kịch bản thực nghiệm ở các phần tiếp theo.

VII.2. Bộ công cụ đánh giá – Ragas

Để có thể đánh giá một hệ thống RAG, ta cần một bộ công cụ đánh giá chuyên biệt. Một trong những ứng cử viên được lựa chọn là RAGAS, với công trình nghiên cứu tương ứng là “Ragas: Automated Evaluation of Retrieval Augmented Generation” vào năm 2024.

Ragas là framework đánh giá tự động được thiết kế riêng cho hệ thống RAG, giúp đo lường chất lượng của cả hai thành phần chính là retrieval và generation. Khác với các phương pháp đánh giá truyền thống đòi hỏi ground truth annotations do con người tạo ra, Ragas sử dụng các mô hình ngôn ngữ lớn như GPT-4 để tự động hóa quá trình đánh giá, giảm thiểu chi phí và thời gian cần thiết.

Framework hoạt động dựa trên nguyên tắc đánh giá đa chiều, trong đó mỗi khía cạnh của hệ thống RAG được đo lường thông qua các metric riêng biệt. Bốn metric chính được sử dụng trong tài liệu này bao gồm faithfulness, answer relevancy, context precision và context recall.

1. Faithfulness – Đo lường sự trung thực

Chỉ số *Faithfulness* đánh giá mức độ trung thực của câu trả lời so với *context* được truy xuất, đảm bảo không có hiện tượng *hallucination*. Một câu trả lời được coi là *faithful* nếu tất cả các phát biểu trong đó đều có thể được hỗ trợ bởi *context* đã truy xuất.

Công thức:

$$\text{Faithfulness} = \frac{|\text{Số phát biểu có thể suy ra từ context}|}{|\text{Tổng số phát biểu trong câu trả lời}|}$$

Quy trình tính toán:

1. **Phân tách:** Sử dụng LLM để chia câu trả lời thành các phát biểu (*claims*) riêng lẻ.
2. **Xác minh:** Kiểm tra từng phát biểu xem có thể suy ra từ *context* hay không.
3. **Tính điểm:** Áp dụng công thức để tính tỷ lệ phát biểu đúng.

💡 Ví dụ minh họa – Faithfulness

Question: Where and when was Einstein born?

Context: Albert Einstein (born 14 March 1879) was a German-born theoretical physicist, widely held to be one of the greatest and most influential scientists of all time.

Answer: Einstein was born in Germany on 20 March 1879.

Phân tích: LLM phân tách câu trả lời thành hai phát biểu:

- **Phát biểu 1:** “Einstein was born in Germany.”: Đúng, có thể suy ra từ *context* (“German-born”).
- **Phát biểu 2:** “Einstein was born on 20 March 1879.”: Sai, *context* ghi 14 March 1879 chứ không phải 20 March 1879.

Kết quả: Faithfulness = $\frac{1}{2} = 0.5$ do chỉ một trong hai phát biểu có thể được xác thực từ *context*.

📌 2. Answer Relevancy – Đo lường sự liên quan

Chỉ số *Answer Relevancy* đánh giá mức độ phù hợp giữa câu trả lời và câu hỏi ban đầu, nhằm xác nhận câu trả lời có giải quyết đúng vấn đề được hỏi hay không. Chỉ số này không đánh giá tính đúng-sai theo nghĩa *factuality*, mà tập trung vào mức độ đầy đủ và tránh thông tin dư thừa.

Công thức:

$$\text{Answer Relevancy} = \frac{1}{N} \sum_{i=1}^N \cos(E_{g_i}, E_o)$$

trong đó E_{g_i} là embedding của câu hỏi thứ i được sinh ra từ câu trả lời, E_o là embedding của câu hỏi gốc, và N là số lượng câu hỏi được sinh ra.

Quy trình tính toán:

1. **Reverse-engineer:** Yêu cầu LLM tạo ra N câu hỏi khác nhau từ câu trả lời đã cho.
2. **Embedding:** Chuyển đổi câu hỏi gốc và các câu hỏi sinh ra thành vector embedding.
3. **Tính độ tương đồng:** Tính trung bình cosine similarity giữa câu hỏi gốc và các câu hỏi sinh ra.

💡 Ví dụ minh họa – Answer Relevancy

Question: Where is France and what is its capital?

Low relevance answer: France is in western Europe.

High relevance answer: France is in western Europe and Paris is its capital.

Phân tích câu trả lời relevancy thấp: LLM có thể sinh ra các câu hỏi như “Where is France located?” hoặc “In which part of Europe is France situated?”. Các câu hỏi này chỉ

tương đồng một phần với câu hỏi gốc vì thiếu thông tin về thủ đô.

Phân tích câu trả lời relevancy cao: LLM có thể sinh ra câu hỏi “Where is France and what is its capital?” khớp với câu hỏi gốc, dẫn đến cosine similarity cao hơn.

Kết quả: Câu trả lời đầy đủ có điểm *Answer Relevancy* gần 1, trong khi câu trả lời thiếu sót có điểm thấp hơn đáng kể.

3. Context Precision – Đo lường độ chính xác retrieval

Chỉ số *Context Precision* đo lường độ chính xác của quá trình retrieval bằng cách đánh giá thứ hạng của các *context*. Chỉ số này kiểm tra xem các chunk liên quan có được xếp ở vị trí cao trong danh sách *retrieved contexts* hay không.

Công thức:

$$\text{Context Precision} = \frac{\sum_{k=1}^K (\text{Precision}@k \times v_k)}{\text{Tổng số context liên quan trong top } K}$$

trong đó K là tổng số chunk trong *retrieved contexts* và $v_k \in \{0, 1\}$ là chỉ số liên quan tại vị trí k .

Quy trình tính toán:

1. **Xác định tính liên quan:** Sử dụng LLM để đánh giá từng *context* có liên quan đến câu hỏi hay không.
2. **Tính Precision@k:** Với mỗi vị trí k , tính tỷ lệ *context* liên quan trong top k .
3. **Weighted average:** Tính trung bình có trọng số của Precision@k, chỉ tính cho các vị trí có *context* liên quan.

Ví dụ minh họa – Context Precision

Question: What are the health benefits of green tea?

Retrieved contexts theo thứ tự:

1. Green tea contains antioxidants that may reduce cancer risk. – Liên quan
2. Tea plantations are common in Asia, especially China and India. – Không liên quan
3. Green tea can boost metabolism and aid weight loss. – Liên quan
4. The history of tea dates back thousands of years. – Không liên quan
5. Green tea improves brain function and mental alertness. – Liên quan

Tính toán:

- $\text{Precision}@1 = \frac{1}{1} = 1.0, v_1 = 1$

- $\text{Precision@2} = \frac{1}{2} = 0.5, v_2 = 0$
- $\text{Precision@3} = \frac{2}{3} \approx 0.67, v_3 = 1$
- $\text{Precision@4} = \frac{2}{4} = 0.5, v_4 = 0$
- $\text{Precision@5} = \frac{3}{5} = 0.6, v_5 = 1$

Kết quả: Context Precision = $\frac{(1.0 \times 1 + 0.67 \times 1 + 0.6 \times 1)}{3} = \frac{2.27}{3} \approx 0.76$. Điểm số phản ánh việc có *context* không liên quan xen kẽ giữa các *context* hữu ích.

4. Context Recall – Đo lường độ bao phủ retrieval

Chỉ số *Context Recall* đánh giá khả năng bao quát của quá trình retrieval, đo lường có bao nhiêu thông tin cần thiết từ *reference answer* đã được tìm thấy trong *retrieved contexts*.

Công thức:

$$\text{Context Recall} = \frac{|\text{Số câu trong reference có thể gán cho retrieved contexts}|}{|\text{Tổng số câu trong reference}|}$$

Quy trình tính toán:

1. **Phân tách reference:** Chia *reference answer* thành các câu/*claims* riêng lẻ.
2. **Attribution:** Sử dụng LLM để kiểm tra từng *claim* có thể suy ra từ *retrieved contexts* hay không.
3. **Tính tỷ lệ:** Tính tỷ lệ *claims* được hỗ trợ bởi *contexts* trên tổng số *claims*.

Ví dụ minh họa – Context Recall

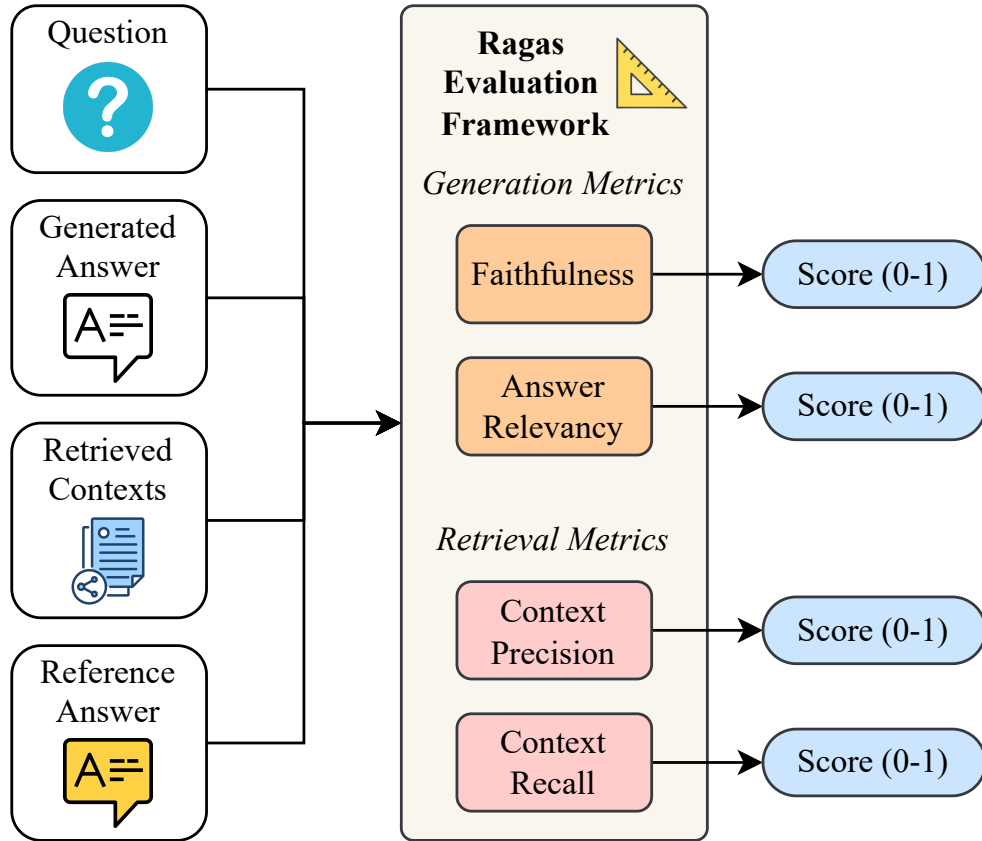
Question: Where is the Eiffel Tower located?

Reference answer: The Eiffel Tower is located in Paris.

Retrieved contexts: Paris is the capital of France.

Phân tích: *Reference answer* chứa *claim* chính: “The Eiffel Tower is located in Paris.” Tuy nhiên, *retrieved context* chỉ cung cấp thông tin “Paris is the capital of France” mà không đề cập vị trí của Eiffel Tower. Do đó, LLM không thể suy ra *claim* từ *reference* dựa trên *context* hiện có.

Kết quả: Context Recall = $\frac{0}{1} = 0$, cho thấy retriever đã thất bại trong việc tìm kiếm *context* chứa thông tin cần thiết để trả lời câu hỏi.



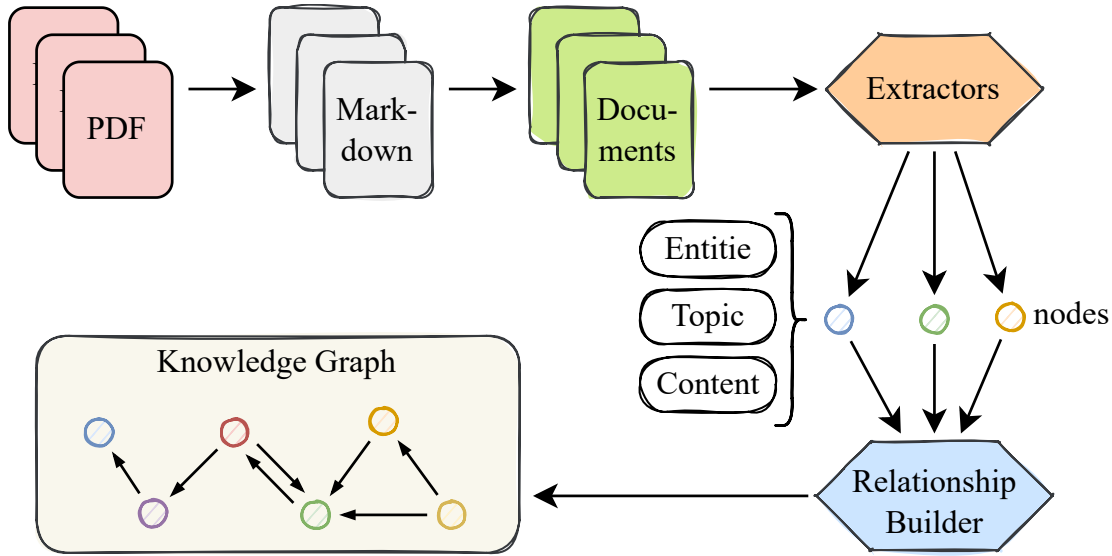
Hình 7: Hình minh hoạ cho các metrics của công cụ đánh giá Ragas.

Mỗi metric cho giá trị từ 0 đến 1, với giá trị cao hơn thể hiện chất lượng tốt hơn. Bốn metric này bổ sung cho nhau: *faithfulness* và *answer relevancy* đánh giá chất lượng generation, trong khi *context precision* và *context recall* đánh giá hiệu năng retrieval.

VII.3. Nguồn dữ liệu và bộ dữ liệu đánh giá

Ngoài bộ công cụ đánh giá, một bộ dữ liệu chuẩn hóa dùng riêng cho mục đích đánh giá là thành phần không thể thiếu trong quy trình xây dựng và kiểm thử hệ thống RAG. Nguồn dữ liệu đầu vào của các hệ thống RAG thường là tài liệu nội bộ hoặc dữ liệu nhạy cảm, do đó khó có thể sử dụng trực tiếp các bộ dữ liệu công khai để phản ánh đúng đặc thù nghiệp vụ. Vì vậy, việc thiết kế và xây dựng một bộ dữ liệu đánh giá riêng, bám sát ngữ cảnh triển khai thực tế, là yêu cầu bắt buộc.

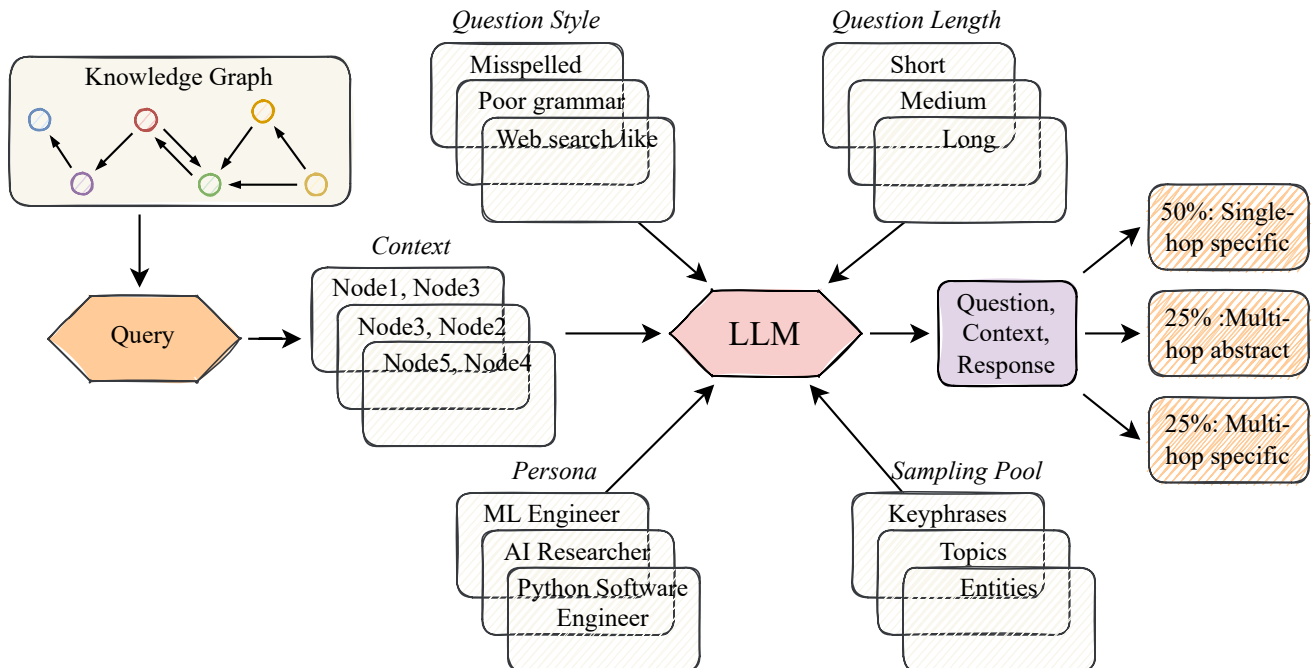
Đối với bài toán ở mục VII.1., nguồn dữ liệu gốc bao gồm 45 tài liệu AIO ở định dạng PDF được cung cấp trước, đây là corpus chính chứa kiến thức mà hệ thống RAG sẽ truy xuất thông tin. Từ corpus này, một bộ dữ liệu test chuyên biệt được tạo ra thông qua TestsetGenerator của Ragas để phục vụ cho việc đánh giá.



Hình 8: Minh hoạ quy trình tạo Knowledge Graph với nguồn dữ liệu là PDF bằng Ragas.

Quy trình tạo testset tự động bắt đầu với việc tiền xử lý tài liệu PDF, chuyển đổi sang định dạng markdown, sau đó phân tích để xây dựng knowledge graph thông qua embedding và trích xuất quan hệ giữa các đoạn văn bản, như hình minh hoạ Hình 9.

Tiếp theo, các query synthesizer sinh câu hỏi theo ba loại chính, gồm single-hop specific chiếm 50% là câu hỏi đơn giản chỉ cần một bước suy luận từ một nguồn duy nhất, multi-hop abstract chiếm 25% yêu cầu tổng hợp thông tin từ nhiều nguồn và trả lời mang tính phân tích tổng quan, cùng multi-hop specific chiếm 25% cần nhiều bước suy luận kết nối thông tin từ nhiều nguồn nhưng tập trung vào sự kiện cụ thể.



Hình 9: Minh hoạ quy trình từ Knowledge Graph thành bộ dữ liệu đánh giá thông qua Ragas.

Sau khi câu hỏi được sinh ra, LLM tạo reference answer và trích xuất `reference_contexts` phù hợp từ tài liệu gốc. Các thông tin này được tổ chức thành schema chuẩn với các trường chính bao gồm `user_input` là câu hỏi được sinh tự động, `reference_contexts` là danh sách đoạn văn bản ground truth từ tài liệu gốc, `reference` là câu trả lời chuẩn do LLM sinh ra, cùng các metadata như `persona_name` xác định vai trò người hỏi, `query_style` chỉ kiểu câu hỏi như `MISSPELLED`, `query_length` chỉ độ dài câu hỏi, và `synthesizer_name` xác định loại query synthesizer sử dụng, như hình minh hoạ Hình 9.

Khi đánh giá hệ thống RAG thực tế, cần bổ sung hai trường vào mỗi mẫu test là `response` là câu trả lời do hệ thống sinh ra và `retrieved_contexts` là danh sách context mà retriever truy xuất được, để có thể tính toán các metric như được đề cập ở VII.2..

VII.3.1. Triển khai

Phần này sẽ trình bày chi tiết quy trình triển khai xây dựng bộ dữ liệu đánh giá cho hệ thống RAG sử dụng Ragas. Quy trình bao gồm các bước từ thiết lập môi trường, tiền xử lý tài liệu PDF, chuyển đổi sang định dạng markdown để dễ dàng xử lý, đến việc sử dụng TestsetGenerator để tự động sinh các mẫu đánh giá có cấu trúc. Toàn bộ quá trình được thực hiện trên nền tảng Google Colab với mô hình `gpt-4o-mini` làm generator và `text-embedding-3-small` cho việc tạo embeddings.

VII.3.2. Thiết lập môi trường

Cài đặt thư viện

Bước đầu tiên là cài đặt các thư viện cần thiết cho quá trình sinh testset. Ragas phiên bản 0.2.0 trở lên cung cấp API mới cho testset generation. Các thư viện langchain hỗ trợ tích hợp với các mô hình ngôn ngữ lớn, trong khi `pymupdf4llm` giúp chuyển đổi PDF sang markdown với khả năng bảo toàn cấu trúc tài liệu.

```
1 !pip install -q ragas>=0.2.0 langchain langchain-community langchain-openai
2 !pip install -q openai pypdf rapidfuzz pandas pymupdf4llm
```

Nhập thư viện cần thiết

Đoạn code import các module cần thiết cho toàn bộ pipeline. Các thư viện Ragas cung cấp `LangchainLLMWrapper` và `TestsetGenerator` cho việc sinh testset tự động. `OpenAIEmbeddings` được sử dụng để tạo vector representation của văn bản, trong khi `DirectoryLoader` và `TextLoader` hỗ trợ đọc tài liệu markdown từ thư mục.

```
1 import os
2 import re
3 import json
4 import warnings
5 import pymupdf4llm
```

```

6 import pandas as pd
7 from pathlib import Path
8 from google.colab import userdata
9 from langchain_openai import ChatOpenAI
10 from ragas.llms import LangchainLLMWrapper
11 from ragas.testset import TestsetGenerator
12 from langchain_openai import OpenAIEmbeddings
13 from ragas.embeddings import LangchainEmbeddingsWrapper
14 from langchain_community.document_loaders import DirectoryLoader, TextLoader
15
16 warnings.filterwarnings('ignore')

```

Cấu hình API Key

API key của OpenAI được lấy từ Colab secrets để đảm bảo bảo mật. Key này cần thiết cho cả LLM generator và embedding model trong quá trình sinh testset.

```

1 os.environ["OPENAI_API_KEY"] = userdata.get("OPENAI_API_KEY")

```

VII.3.3. Dataset

Tải bộ dữ liệu

Bộ dữ liệu gồm 45 file PDF của AIO được tải xuống từ Google Drive thông qua gdown và giải nén vào thư mục làm việc. Đây là nguồn tài liệu gốc sẽ được chuyển đổi và sử dụng để sinh testset.

```

1 !gdown 19C6FVRVTn4bpe7HFoutClQ0-1VXn2L7w
2 !unzip -q data.zip

```

Chuyển đổi PDF sang Markdown

Hàm `convert_pdfs_to_markdown` sử dụng `pymupdf4llm` để chuyển đổi các file PDF sang định dạng markdown. Markdown giữ lại cấu trúc văn bản như heading, list, table mà vẫn dễ xử lý hơn PDF, giúp LLM hiểu rõ hơn cấu trúc và ngữ cảnh của tài liệu khi sinh câu hỏi. Output là các file .md tương ứng được lưu trong thư mục markdown.

```

1 def convert_pdfs_to_markdown(pdf_dir, output_dir=None):
2     pdf_path = Path(pdf_dir)
3     output_path = Path(output_dir) if output_dir else pdf_path / "markdown"
4     output_path.mkdir(exist_ok=True)
5     pdf_files = list(pdf_path.glob("*.pdf"))
6     print(f"Found {len(pdf_files)} PDF files")

```

```

7     converted = 0
8     for pdf_file in pdf_files:
9         try:
10             print(f"Converting: {pdf_file.name}")
11             md_text = pymupdf4llm.to_markdown(str(pdf_file))
12             md_file = output_path / f"{pdf_file.stem}.md"
13             with open(md_file, 'w', encoding='utf-8') as f:
14                 f.write(md_text)
15             converted += 1
16         except Exception as e:
17             print(f"Error: {e}")
18     print(f"\nConverted {converted}/{len(pdf_files)} files")
19     print(f"Output: {output_path}")
20     return str(output_path)
21
22 PDF_DIR = "./AIO_Docs"
23 md_dir = convert_pdfs_to_markdown(PDF_DIR)

```

Đọc dữ liệu markdown

DirectoryLoader được cấu hình để đọc tất cả file markdown trong thư mục, sử dụng TextLoader với encoding UTF-8 để xử lý tiếng Việt. Kết quả là một danh sách các document objects của Langchain, mỗi object chứa nội dung và metadata của một file.

```

1 DATA_DIR = "./AIO_Docs/markdown"
2 loader = DirectoryLoader(
3     DATA_DIR,
4     glob="**/*.md",
5     loader_cls=TextLoader,
6     loader_kwargs={"encoding": "utf-8"},
7     show_progress=True
8 )
9 docs = loader.load()
10 print(f"\nLoaded {len(docs)} Markdown documents")

```

VII.3.4. Thiết lập các mô hình

Hai thành phần chính được khởi tạo gồm `generator_llm` sử dụng `gpt-4o-mini` với `max_completion_tokens` là 8192 để sinh câu hỏi và câu trả lời, và `generator_embeddings` sử dụng `text-embedding-3-small` để tạo vector representation cho similarity search trong quá trình xây dựng knowledge graph.

```

1 generator_llm = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o-mini",
2                                               max_completion_tokens=8192))
3 generator_embeddings = LangchainEmbeddingsWrapper(
4     OpenAIEmbeddings(model="text-embedding-3-small")

```

```
4 )
```

VII.3.5. Tạo bộ dữ liệu đánh giá

TestsetGenerator nhận vào LLM và embedding model đã được cấu hình, sau đó gọi phương thức `generate_with_langchain_docs` với tham số `TESTSET_SIZE` để sinh số lượng mẫu mong muốn. Quá trình này bao gồm việc xây dựng knowledge graph từ documents, trích xuất entities và relationships, sau đó sinh các câu hỏi theo phân phối mặc định gồm single-hop specific, multi-hop abstract và multi-hop specific queries.

```
1 TESTSET_SIZE = 50
2
3 generator = TestsetGenerator(
4     llm=generator_llm,
5     embedding_model=generator_embeddings
6 )
7 dataset = generator.generate_with_langchain_docs(
8     docs,
9     testset_size=TESTSET_SIZE
10 )
11
12 print(f"Generating {TESTSET_SIZE} samples from {len(docs)} documents...")
13 print(f"\nGenerated {len(dataset)} samples!")
```

Lưu thành file json

Testset được chuyển đổi thành dictionary records và lưu dưới dạng JSON với encoding UTF-8 để hỗ trợ tiếng Việt. Tham số `ensure_ascii=False` đảm bảo các ký tự Unicode được giữ nguyên, trong khi `indent=2` giúp file dễ đọc hơn.

```
1 records = df.to_dict(orient="records")
2 with open("ragas_testset.json", 'w', encoding="utf-8") as f:
3     json.dump(records, f, ensure_ascii=False, indent=2, default=str)
```

Cấu trúc file JSON đầu ra

File JSON đầu ra chứa danh sách các sample, mỗi sample là một object với các trường thông tin cần thiết cho việc đánh giá hệ thống RAG. Bảng dưới đây mô tả chi tiết các thuộc tính trong mỗi sample:

Cấu trúc dữ liệu trong Ragas Testset

Thuộc tính	Mô tả
<code>user_input</code>	Câu hỏi được sinh ra tự động từ nội dung tài liệu, đại diện cho query của người dùng.
<code>reference_contexts</code>	Danh sách các đoạn văn bản từ tài liệu gốc có liên quan đến câu hỏi, dùng làm ground truth contexts để tính context precision và context recall.
<code>reference</code>	Câu trả lời tham chiếu được sinh bởi LLM dựa trên <code>reference_contexts</code> , dùng để tính context recall và đánh giá chất lượng câu trả lời của hệ thống.
<code>persona_name</code>	Vai trò hoặc persona của người hỏi như AI Research Scientist hay Machine Learning Engineer, giúp tạo câu hỏi phù hợp với ngữ cảnh sử dụng.
<code>query_style</code>	Kiểu câu hỏi như MISSPELLED có lỗi chính tả để kiểm tra độ robust của hệ thống.
<code>query_length</code>	Độ dài câu hỏi như MEDIUM hay LONG, giúp kiểm tra khả năng xử lý câu hỏi phức tạp.
<code>synthesizer_name</code>	Tên của query synthesizer được sử dụng như <code>single_hop_specific_query_synthesizer</code> , xác định loại câu hỏi được tạo.

Các trường `user_input`, `reference_contexts` và `reference` là bắt buộc để tính toán các metric đánh giá. Khi đánh giá hệ thống RAG thực tế, cần bổ sung thêm hai trường: `response` chứa câu trả lời do hệ thống sinh ra và `retrieved_contexts` chứa danh sách context mà retriever của hệ thống truy xuất được. Với cấu trúc đầy đủ này, có thể tính toán faithfulness bằng cách so sánh `response` với `retrieved_contexts`, answer relevancy bằng cách so sánh `response` với `user_input`, context precision và context recall bằng cách so sánh `retrieved_contexts` với `reference_contexts`.

VII.4. Baseline hệ thống RAG cơ bản

Trước khi so sánh, ta cần thiết lập baseline gồm các thành phần tiêu chuẩn của RAG, gồm sử dụng chunking kích thước cố định, tìm kiếm bằng vector thuần túy và chưa áp dụng các phương pháp tối ưu hóa như Query Transformation hay Re-ranking.

Đây là cơ sở để thực hiện các thực nghiệm ở phần sau, giúp ta quan sát rõ sự thay đổi hiệu năng khi lần lượt thay thế từng thành phần trong hệ thống.

VII.4.1. Môi trường thực nghiệm

Ở bước đầu tiên, chúng ta thiết lập môi trường làm việc cho toàn bộ hệ thống, đảm bảo các thành phần trong pipeline RAG có thể được triển khai và vận hành một cách thống nhất. Việc chuẩn bị môi trường đóng vai trò nền tảng, giúp quá trình xây dựng và mở rộng hệ thống trong các bước tiếp theo diễn ra thuận lợi và có hệ thống.

Cài đặt thư viện

4.1 Cài đặt thư viện

```
1 !pip install -q \  
2 "langchain>=0.2.0" \  
3 "langchain-core>=0.2.0" \  
4 "langchain-community>=0.1.0" \  
5 "langchain-text-splitters>=0.2.0" \  
6 "sentence-transformers>=2.2.0" \  
7 "langchain-huggingface>=0.0.1" \  
8 "chromadb>=0.5.0" \  
9 "langchain-chroma>=0.2.0" \  
10 "pypdf" \  
11 "ragas>=0.2.0" \  
12 "datasets>=2.14.0" \  
13 "pandas" \  
14 "matplotlib" \  
15 "seaborn" \  
16 "openai>=1.0.0" \  
17 "langchain-openai>=0.1.0" \  
18 "gdown" \  
19 "tiktoken" \  
20 "nest_asyncio"
```

Khai báo thư viện

Sau khi hoàn tất việc thiết lập môi trường, chúng ta tiến hành khởi tạo không gian làm việc bằng cách nạp các thư viện và thành phần cần thiết. Bước này đảm bảo toàn bộ các chức năng phục vụ xử lý dữ liệu, truy xuất và sinh nội dung được sẵn sàng trước khi xây dựng pipeline RAG ở các bước tiếp theo.

4.1 Khai báo thư viện

```
1 import os  
2 import re  
3 import json  
4 import glob  
5 import time  
6 import gdown  
7 import random  
8 import asyncio
```

```

9 import unicodedata
10 import numpy as np
11 import pandas as pd
12 import nest_asyncio
13 import seaborn as sns
14 from tqdm import tqdm
15 from typing import List
16 import matplotlib.pyplot as plt
17 from google.colab import userdata
18 from matplotlib.backends.backend_pdf import PdfPages
19
20 from openai import OpenAI
21 from datasets import Dataset
22 from langchain_chroma import Chroma
23
24 from langchain_core.prompts import PromptTemplate
25 from langchain_core.runnables import RunnablePassthrough
26 from langchain_core.output_parsers import StrOutputParser
27 from langchain_openai import ChatOpenAI, OpenAIEmbeddings
28 from langchain_community.document_loaders import PyPDFLoader
29 from langchain_text_splitters import RecursiveCharacterTextSplitter
30
31 from ragas import evaluate
32 from ragas.run_config import RunConfig
33 from ragas.metrics import (
34     faithfulness,
35     answer_relevancy,
36     context_precision,
37     context_recall
38 )
39 from ragas.embeddings import LangchainEmbeddingsWrapper
40
41 nest_asyncio.apply()

```

Thiết lập tham số ngẫu nhiên

Để đảm bảo tính tái lập của thực nghiệm, chúng ta cố định các nguồn ngẫu nhiên được sử dụng trong quá trình thực thi hệ thống. Bước này giúp giảm thiểu sự biến thiên ngoài ý muốn giữa các lần chạy, từ đó đảm bảo rằng các kết quả thu được phản ánh đúng tác động của cấu hình và phương pháp được sử dụng.

```

1 SEED = 42
2
3 def set_global_seed(seed_value: int = 42):
4     random.seed(seed_value)
5     np.random.seed(seed_value)
6     os.environ["PYTHONHASHSEED"] = str(seed_value)
7     print(f"Global seed set to {seed_value}")
8
9 set_global_seed(SEED)

```


Thiết lập API Key

Hệ thống cần cấu hình API key để xác thực và gửi yêu cầu tới hạ tầng của OpenAI. API key đóng vai trò như chứng thực quyền truy cập, cho phép chúng ta khai thác sức mạnh của các mô hình như GPT-4o hay text embedding trong quá trình xử lý dữ liệu của pipeline.

4.1 Thiết lập API Key

```
1 os.environ["OPENAI_API_KEY"] = userdata.get("OPENAI_API_KEY")
2 OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
```

Cấu hình tham số và mô hình

Ở bước này, chúng ta thiết lập các tham số điều khiển quá trình thực thi của hệ thống. Những cấu hình này ảnh hưởng trực tiếp đến cách pipeline được vận hành, bao gồm chiến lược xử lý theo lô, lựa chọn mô hình, và các thiết lập phục vụ quá trình đánh giá. Việc tách riêng bước cấu hình giúp đảm bảo hệ thống có thể được điều chỉnh linh hoạt cho các kịch bản thực nghiệm khác nhau.

4.1 Cấu hình tham số và mô hình

```
1 BATCH_SIZE = 32
2 EMBEDDING_BATCH_SIZE = 100
3
4 LLM_MODEL = "gpt-4o-mini"
5 EMBEDDING_MODEL = "text-embedding-3-small"
6
7 EVAL_SAMPLE_SIZE = None
8 EVAL_MAX_WORKERS = 64
9 EVAL_TIMEOUT = 180
```

VII.4.2. Dữ liệu thực nghiệm

Tải bộ dữ liệu gốc

Đầu tiên, chúng ta tải về bộ dữ liệu gốc được đóng gói dưới định dạng nén zip. Đây là tập hợp các file tài liệu PDF public của AI VIET NAM mà hệ thống RAG sẽ sử dụng để indexing. Sau khi tải xong, đoạn mã sẽ tự động giải nén và lưu vào thư mục RAG_DATA để thuận tiện cho việc truy cập ở các bước sau.

4.2 Tải bộ dữ liệu gốc

```
1 file_id = "1Ebcv-0kq6_eF6QYfIMSL6ZXQUeTnWKW"
2 zip_path = "/content/data.zip"
3 gdown.download(f"https://drive.google.com/uc?id={file_id}", zip_path, quiet=False)
4
```

```

5 !unzip -o /content/data.zip
6 DATA_DIR = "RAG_DATA"

```

Tải bộ dữ liệu đánh giá

Khác với các thực nghiệm cơ bản chỉ dừng lại ở việc quan sát bằng mắt thường, trong bài nâng cao này, chúng ta cần một thước đo định lượng chính xác. Do đó, đoạn mã dưới đây sẽ tải về file `test.json` chứa thông tin của tập `test`.

4.2 Tải bộ dữ liệu đánh giá

```

1 file_id = "1a5RL7ex88w_5tZ1g6R31QTrfibvxqT_Q"
2 gdown.download(f"https://drive.google.com/uc?id={file_id}", quiet=False)

```

Đây là bộ dữ liệu đã được chuẩn bị sẵn, chứa danh sách các câu hỏi, câu trả lời và ngữ cảnh tham chiếu tương ứng. File này sẽ đóng vai trò làm đề bài để chấm điểm hệ thống RAG thông qua các chỉ số của Ragas ở phần sau.

VII.4.3. Khởi tạo mô hình và các thành phần xử lý

Sau khi đã có dữ liệu, bước tiếp theo ta sẽ thiết lập các thành phần tính toán cốt lõi. Trong phần baseline này, ta sẽ sử dụng các dịch vụ API của OpenAI để đảm bảo chất lượng đầu ra ổn định nhất nhằm mục đích làm quy chuẩn so sánh.

Cấu hình LLM và embedding model

Chúng ta xây dựng hai hàm tiện ích `get_llm` và `get_embeddings` để khởi tạo mô hình. Đối với mô hình LLM, ta sẽ sử dụng `gpt-4o-mini` với `temperature=0.0`, với mục đích tối đa hóa tính xác định của câu trả lời. Và ta sử dụng mô hình `text-embedding-3-small`, một mô hình embedding tốt giữa cân bằng hiệu năng và tối ưu chi phí.

4.3 Cấu hình LLM và Embedding Model

```

1 def get_llm(
2     model_name: str = LLM_MODEL,
3     temperature: float = 0.0,
4     max_tokens: int = 8192,
5     seed: int = SEED,
6 ):
7     llm = ChatOpenAI(
8         model=model_name,
9         temperature=temperature,
10        max_tokens=max_tokens,
11        seed=seed,
12        api_key=OPENAI_API_KEY,

```

```

13     )
14     return llm
15
16
17 def get_embeddings(
18     model_name: str = EMBEDDING_MODEL,
19     chunk_size: int = EMBEDDING_BATCH_SIZE,
20 ):
21     embeddings = OpenAIEmbeddings(
22         model=model_name,
23         api_key=OPENAI_API_KEY,
24         chunk_size=chunk_size,
25     )
26     return embeddings

```

Tiền xử lý văn bản tiếng Việt

Dữ liệu văn bản tiếng Việt trích xuất từ PDF thường gặp các vấn đề về lỗi font hoặc mã hóa Unicode không đồng nhất. Do đó, hàm `clean_vietnamese_text` được thiết kế để chuẩn hóa văn bản về dạng NFC và loại bỏ các ký tự nhiễu.

4.3 Tiền xử lý văn bản

```

1 def clean_vietnamese_text(text: str) -> str:
2     text = unicodedata.normalize("NFC", text)
3     text = "".join(
4         char for char in text
5         if not unicodedata.category(char).startswith("C") or char in "\n\t"
6     )
7     text = re.sub(r"\s+", " ", text)
8     text = re.sub(r"\n\s*\n", "\n", text)
9     return text.strip()

```

Đọc và chia nhỏ tài liệu

Tiếp theo, chúng ta định nghĩa hai lớp `SimpleLoader` để đọc dữ liệu và `TextSplitter` để chia nhỏ văn bản. Lưu ý rằng ở baseline này, ta sử dụng chiến lược Fixed-size Chunking tiêu chuẩn với kích thước `chunk_size=1024` và `chunk_overlap=128`.

4.3 Đọc và chia nhỏ tài liệu

```

1 class SimpleLoader:
2     def load_pdf(self, pdf_file: str):
3         docs = PyPDFLoader(pdf_file, extract_images=True).load()
4         for doc in docs:
5             doc.page_content = clean_vietnamese_text(doc.page_content)
6         return docs

```

```

7
8     def load_dir(self, dir_path: str) -> List:
9         pdf_files = glob.glob(f"{dir_path}/*.pdf")
10        if not pdf_files:
11            raise ValueError(f"No PDF files found in {dir_path}")
12
13        all_docs = []
14        for pdf_file in tqdm(pdf_files, desc="Loading PDFs"):
15            try:
16                all_docs.extend(self.load_pdf(pdf_file))
17            except Exception as e:
18                pass
19        return all_docs
20
21
22    class TextSplitter:
23        def __init__(
24            self,
25            chunk_size: int = 1024,
26            chunk_overlap: int = 128,
27        ):
28            self.splitter = RecursiveCharacterTextSplitter(
29                separators=["\n\n", "\n", " ", ""],
30                chunk_size=chunk_size,
31                chunk_overlap=chunk_overlap,
32                length_function=len,
33            )
34
35        def split(self, documents):
36            return self.splitter.split_documents(documents)

```

Xây dựng Vector Database

Cuối cùng, class `VectorDB` sẽ đóng vai trò trung tâm quản lý kho dữ liệu vector. Nơi chịu trách nhiệm nhận các chunks văn bản, chuyển đổi chúng thành vector thông qua embedding model đã khởi tạo ở trên, và sau cùng lưu trữ vào ChromaDB. Hàm `get_retriever` trong class này giúp chuyển đổi database thành công cụ tìm kiếm, giúp tìm kiếm nhanh thông tin từ tài liệu trong không gian vector.

4.3 Xây dựng Vector Database

```

1    class VectorDB:
2        def __init__(
3            self,
4            documents=None,
5            embedding=None,
6            collection_name: str = "vietnamese_docs",
7            persist_dir: str = "/content/chroma_data",
8        ):
9            self.persist_dir = persist_dir

```

```

10     self.collection_name = collection_name
11     self.embedding = embedding or get_embeddings()
12     self.db = self._build_db(documents)
13
14     def _build_db(self, documents):
15         if documents is None or len(documents) == 0:
16             db = Chroma(
17                 collection_name=self.collection_name,
18                 embedding_function=self.embedding,
19                 persist_directory=self.persist_dir,
20             )
21         else:
22             db = Chroma.from_documents(
23                 documents=documents,
24                 embedding=self.embedding,
25                 collection_name=self.collection_name,
26                 persist_directory=self.persist_dir,
27             )
28         return db
29
30     def get_retriever(self, search_kwargs: dict = None):
31         if search_kwargs is None:
32             search_kwargs = {"k": 3}
33
34         return self.db.as_retriever(
35             search_type="similarity",
36             search_kwargs=search_kwargs,
37         )

```

VII.4.4. Xây dựng luồng xử lý cho RAG

Để khắc phục vấn đề độ trễ lớn khi mở rộng quy mô đánh giá lên hàng trăm mẫu kiểm thử, ta xây dựng class `BatchRAG` nhằm tối ưu hóa quy trình thông qua xử lý theo batch. Class này quản lý việc kết nối Retriever với LLM và điều phối luồng dữ liệu, giúp tận dụng tối đa băng thông API và giảm đáng kể tổng thời gian thực nghiệm so với phương pháp gọi tuần tự.

Song song đó, class `FocusedAnswerParser` được tích hợp để hậu xử lý kết quả bằng cách loại bỏ các cụm từ dẫn dắt dư thừa (như “Dựa trên tài liệu...”, “Câu trả lời là...”). Việc làm sạch này giúp giữ lại nội dung cốt lõi, đảm bảo tính nhất quán và độ chính xác cao hơn cho quá trình chấm điểm tự động ở bước đánh giá.

4.4 Xây dựng luồng xử lý cho RAG

```

1 class FocusedAnswerParser(StrOutputParser):
2     def parse(self, text: str) -> str:
3         text = (text or "").strip()
4         if "[TRẢ LỜI]:" in text:
5             answer = text.split("[TRẢ LỜI]:")[-1].strip()
6         else:
7             answer = text

```

```

8
9     answer = re.sub(r"^s*[\u2022\-\]*\s*", "", answer, flags=re.MULTILINE)
10    answer = re.sub(r"\n+", " ", answer).strip()
11    return answer
12
13
14    class BatchRAG:
15        def __init__(self, llm, batch_size: int = BATCH_SIZE):
16            self.llm = llm
17            self.batch_size = batch_size
18            self.prompt_template = """Bạn là trợ lý AI phân tích tài liệu tiếng Việt.
19
20    [TÀI LIỆU]:
21    {context}
22
23    [CÂU HỎI]:
24    {question}
25
26    Hãy trả lời dựa trên tài liệu. Nếu tài liệu không có thông tin, nói rõ "Không có thông
27                                     tin".
28    [TRẢ LỜI]:"""
29
30    self.prompt = PromptTemplate.from_template(self.prompt_template)
31    self.answer_parser = FocusedAnswerParser()
32
33    def _format_docs(self, docs):
34        formatted = []
35        seen = set()
36        for doc in docs:
37            content = (doc.page_content or "").strip()
38            if content and len(content) > 40 and content not in seen:
39                formatted.append(content)
40                seen.add(content)
41        return "\n\n".join(formatted)
42
43    def get_chain(self, retriever):
44        def format_docs(docs):
45            return self._format_docs(docs)
46
47        rag_chain = (
48            {"context": retriever | format_docs, "question": RunnablePassthrough()}
49            | self.prompt
50            | self.llm
51            | self.answer_parser
52        )
53        return rag_chain
54
55    def batch_retrieve(self, questions: List[str], retriever) -> List[dict]:
56        all_contexts = []
57        for question in tqdm(questions, desc="Retrieving documents"):
58            docs = retriever.invoke(question)
59            contexts = [(doc.page_content or "") for doc in docs]
60            formatted_context = self._format_docs(docs)
61            all_contexts.append({

```

```

60         "question": question,
61         "contexts": contexts,
62         "formatted_context": formatted_context
63     })
64     return all_contexts
65
66     def batch_generate(self, prompts: List[str]) -> List[str]:
67         from langchain_core.messages import HumanMessage
68
69         all_answers = []
70         for i in tqdm(
71             range(0, len(prompts), self.batch_size), desc="Generating answers"
72         ):
73             batch = prompts[i:i + self.batch_size]
74
75             messages_batch = [[HumanMessage(content=p)] for p in batch]
76             batch_results = self.llm.batch(messages_batch)
77
78             for result in batch_results:
79                 answer = self.answer_parser.parse(result.content)
80                 all_answers.append(answer)
81
82         return all_answers
83
84     def answer_with_contexts_batch(self, questions: List[str], retriever):
85         retrieved_data = self.batch_retrieve(questions, retriever)
86         prompts = [
87             self.prompt.format(
88                 context=data["formatted_context"],
89                 question=data["question"]
90             )
91             for data in retrieved_data
92         ]
93         answers = self.batch_generate(prompts)
94
95         results = []
96         for data, answer in zip(retrieved_data, answers):
97             results.append({
98                 "answer": answer,
99                 "contexts": data["contexts"]
100             })
101
102         return results

```

VII.4.5. Khởi tạo RAG

Sau khi đã định nghĩa đầy đủ các class, đây là lúc chúng ta hiện thực hóa hệ thống. Đoạn mã dưới đây thực hiện quy trình "lắp ráp" toàn bộ pipeline từ đầu đến cuối như sau:

1. **Thiết lập Seed:** Đảm bảo mọi yếu tố ngẫu nhiên đều được cố định, giúp kết quả thực nghiệm có tính nhất quán.

2. **Nạp và Xử lý dữ liệu:** Gọi SimpleLoader để đọc PDF và TextSplitter để chia nhỏ văn bản.
3. **Indexing:** Đưa các chunks vào VectorDB để tạo chỉ mục tìm kiếm.
4. **Chain Creation:** Cuối cùng, khởi tạo đối tượng BatchRAG và kết nối với retriever để tạo thành một chuỗi xử lý hoàn chỉnh.

4.5 Khởi tạo RAG

```

1 set_global_seed(SEED)
2
3 llm = get_llm(model_name=LLM_MODEL, seed=SEED)
4 embeddings = get_embeddings(
5     model_name=EMBEDDING_MODEL,
6     chunk_size=EMBEDDING_BATCH_SIZE
7 )
8
9 loader = SimpleLoader()
10 text_splitter = TextSplitter(chunk_size=1024, chunk_overlap=128)
11
12 raw_docs = loader.load_dir(DATA_DIR)
13 split_docs = text_splitter.split(raw_docs)
14
15 vdb = VectorDB(documents=split_docs, embedding=embeddings)
16 retriever = vdb.get_retriever(search_kwargs={"k": 3})
17
18 rag = BatchRAG(llm, batch_size=BATCH_SIZE)
19 rag_chain = rag.get_chain(retriever)

```

Khi chạy xong bước này, hệ thống Baseline RAG đã sẵn sàng để tiếp nhận câu hỏi.

VII.4.6. Thực thi RAG trên bộ câu hỏi kiểm thử

Trước khi có thể chấm điểm, hệ thống cần phải "làm bài thi" trước. Tại bước này, trước tiên chúng ta sẽ nạp bộ câu hỏi từ file `testset.json` đã tải về ở trên vào một DataFrame.

Đọc bộ đánh giá

4.6 Đọc bộ đánh giá

```

1 testset_path = "testset.json"
2
3 with open(testset_path, "r", encoding="utf-8") as f:
4     testset_data = json.load(f)
5
6 test_df = pd.DataFrame([
7     {
8         "user_input": item["user_input"],
9         "reference": item["reference"],

```



```

10         "reference_contexts": item["reference_contexts"]
11     }
12     for item in testset_data
13 ]))
14
15 display(test_df.head())

```

Thực thi RAG

Tại bước này, hệ thống sẽ thực hiện inference (tạm dịch: suy luận) dựa trên toàn bộ danh sách câu hỏi từ tập test. Nhờ cơ chế Batch Inference đã thiết lập trong class `BatchRAG`, quá trình này sẽ diễn ra nhanh chóng hơn nhiều so với việc chạy kết quả tuần tự. Kết quả trả về bao gồm câu trả lời của mô hình (response), và các đoạn văn bản mà mô hình đã tìm kiếm được (retrieved_context), được lưu lại nhằm phục vụ cho bước đánh giá tiếp theo.

4.6 Thực thi RAG

```

1 set_global_seed(SEED)
2
3 questions = test_df["user_input"].tolist()
4
5 start_time = time.time()
6 batch_results = rag.answer_with_contexts_batch(questions, retriever)
7 elapsed_time = time.time() - start_time
8
9 results = []
10 for idx, (result, (_, row)) in enumerate(zip(batch_results, test_df.iterrows())):
11     results.append({
12         "user_input": row["user_input"],
13         "response": result["answer"],
14         "retrieved_contexts": result["contexts"],
15         "reference": row.get("reference", ""),
16         "reference_contexts": row.get("reference_contexts", [])
17     })
18
19 eval_df = pd.DataFrame(results)
20 display(eval_df.head())

```

VII.4.7. Đánh giá RAG với Ragas

Đây là bước quan trọng nhất để định lượng hiệu năng của Baseline. Chúng ta sử dụng thư viện Ragas để chấm điểm dựa trên 4 tiêu chí cốt lõi như sau:

- **Faithfulness (tạm dịch: độ trung thực):** Đánh giá tính nhất quán của câu trả lời, đảm bảo nội dung được sinh ra hoàn toàn dựa trên ngữ cảnh được cung cấp và không bịa đặt.
- **Answer Relevancy (tạm dịch: độ liên quan câu trả lời):** Đo lường mức độ phù hợp của câu trả lời đối với câu hỏi đặt ra, loại bỏ các câu trả lời dài dòng hoặc lạc đề.

- **Context Precision (tạm dịch: độ chính xác ngữ cảnh):** Đánh giá tỷ lệ các chunks thực sự liên quan trong danh sách tìm kiếm được. Chỉ số này càng cao nghĩa là hệ thống ít bị nhiễu bởi thông tin rác.
- **Context Recall (tạm dịch: độ bao phủ ngữ cảnh):** Đánh giá khả năng tìm kiếm thông tin đầy đủ. Kiểm tra xem ngữ cảnh tìm được có bao chứa toàn bộ thông tin cần thiết so với Ground Truth để trả lời câu hỏi hay không.

4.7 Đánh giá RAG

```

1 if EVAL_SAMPLE_SIZE is not None:
2     eval_subset = eval_df.sample(
3         n=min(EVAL_SAMPLE_SIZE, len(eval_df)), random_state=SEED
4     )
5 else:
6     eval_subset = eval_df
7
8 eval_dataset = Dataset.from_pandas(eval_subset.reset_index(drop=True))
9
10 wrapped_embeddings = LangchainEmbeddingsWrapper(
11     OpenAIEmbeddings(model="text-embedding-3-small")
12 )
13
14 run_config = RunConfig(
15     max_workers=EVAL_MAX_WORKERS,
16     timeout=EVAL_TIMEOUT,
17     max_retries=1,
18     max_wait=60,
19 )
20
21 ragas_results = evaluate(
22     dataset=eval_dataset,
23     metrics=[
24         faithfulness,
25         answer_relevancy,
26         context_precision,
27         context_recall
28     ],
29     embeddings=wrapped_embeddings,
30     run_config=run_config,
31     raise_exceptions=False,
32 )
33
34 print("=== RAGAS SCORES ===")
35 print(ragas_results)

```

Trực quan hóa kết quả

Nhằm hỗ trợ quá trình phân tích định lượng, kết quả đánh giá sẽ được chuyển đổi thành các biểu đồ trực quan và lưu trữ dạng PDF.

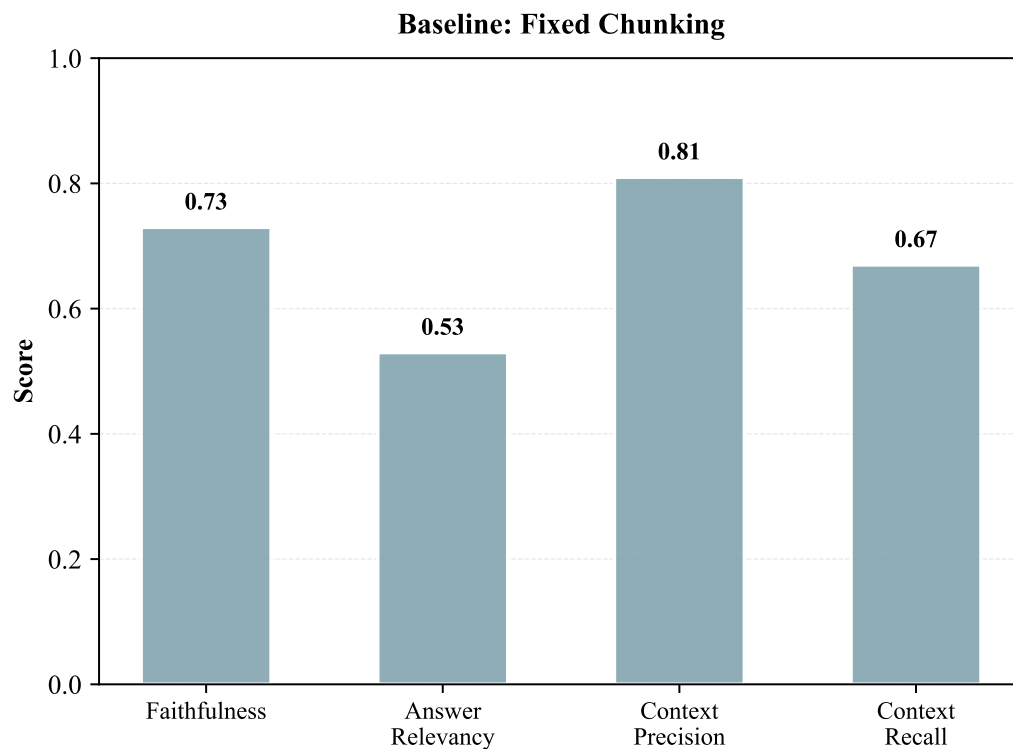
4.7 Trực quan hoá kết quả

```

1 results_df = ragas_results.to_pandas()
2 pdf_filename = "RAGAS_Evaluation_Report_API.pdf"
3
4 with PdfPages(pdf_filename) as pdf:
5     sns.set_style("whitegrid")
6     fig = plt.figure(figsize=(8, 8))
7
8     metric_cols = [col for col in ["faithfulness", "answer_relevancy", "
9                                     context_precision", "context_recall"]
10                     if col in results_df.columns]
11
12     if metric_cols:
13         scores = {col: results_df[col].mean() for col in metric_cols}
14         plt.bar(
15             scores.keys(), scores.values(), color="skyblue",
16             edgecolor="navy", alpha=0.7)
17         plt.title(f"RAGAS Scores (API: {LLM_MODEL})", fontsize=14, fontweight="bold")
18         plt.ylabel("Score (0-1)")
19         plt.ylim(0, 1)
20         pdf.savefig(fig)
21 print(f"Saved: {pdf_filename}")

```

Kết quả thực nghiệm với Baseline



Hình 10: Kết quả đánh giá các chỉ số RAGAS trên hệ thống baseline.

Quan sát Hình 10, ta có thể rút ra những nhận xét quan trọng về hiện trạng của hệ thống Baseline cơ bản như sau:

- **Faithfulness (0.73):** Chỉ số này ở mức khá, cho thấy LLM `gpt-4o-mini` đang tuân thủ tốt nguyên tắc chỉ trả lời dựa trên ngữ cảnh, ít bị ảo giác.
- **Answer Relevancy (0.53):** Đây là điểm yếu nhất của hệ thống Baseline. Mặc dù câu trả lời có tính trung thực cao, nhưng điểm Relevancy thấp cho thấy câu trả lời có thể đang quá dài dòng, lặp lại hoặc chưa đi thẳng vào trọng tâm câu hỏi của người dùng.
- **Context Precision (0.81):** Đây là chỉ số cao nhất trong 4 tiêu chí. Điều này cho thấy các chunks mà hệ thống tìm được có tỷ lệ liên quan cao, chứng tỏ Vector Search cơ bản hoạt động tốt trong việc lọc nhiễu ở top đầu kết quả.
- **Context Recall (0.67):** Chỉ số này ở mức trung bình. Việc chỉ đạt 0.67 cho thấy hệ thống đang bỏ sót khoảng 33% thông tin quan trọng cần thiết để trả lời câu hỏi. Đây có thể là hệ quả của việc Fixed-size Chunking làm đứt gãy mạch văn, khiến Retriever không tìm thấy đủ thông tin.

Kết luận: Baseline hoạt động ổn định nhưng chưa tối ưu về khả năng bao phủ thông tin và độ súc tích của câu trả lời. Đây là động lực chính để chúng ta triển khai các kỹ thuật nâng cao ở các phần tiếp theo.

VII.5. Thực nghiệm

Từ baseline bên trên, phần này tiến hành các thực nghiệm bằng cách cài đặt từng thành phần cải tiến được nhắc đến tại mục lý thuyết III., IV., V. và VI.. Dựa vào kết quả của các thực nghiệm này, ta biết được các thành phần có ảnh hưởng như thế nào tới một hệ thống RAG. Cụ thể, các thực nghiệm bao gồm:

- Thực nghiệm 1: Indexing.
 1. Recursive Chunking.
 2. Semantic Chunking.
- Thực nghiệm 2: Retrieval Strategies.
 1. BM25.
 2. Hybrid kết hợp Interleaving
 3. Hybrid với RRF.
- Thực nghiệm 3: Query Transformations.
 1. HyDE.
 2. Decompostion.
- Thực nghiệm 4: Re-ranking.

1. Cross-Encoder.
2. MMR.

Lưu ý: Trong từng thực nghiệm, nếu bất kỳ thành phần nào không được đề cập, thì sẽ giữ nguyên như trong baseline VII.4..

VII.5.1. Thực nghiệm 1: Indexing

Trong thực nghiệm đầu tiên, chúng ta sẽ đi sâu vào kỹ thuật Chunking. Đây là bước xử lý tiền đề có tác động trực tiếp đến chất lượng tìm kiếm. Dựa trên lý thuyết tại III., chúng ta sẽ so sánh chiến lược Recursive Chunking được sử dụng làm Baseline và chiến lược thử nghiệm Semantic Chunking nhằm đánh giá khả năng khắc phục tình trạng phân mảnh thông tin và tác động của nó đến chất lượng câu trả lời cuối cùng:

- **Recursive Chunking:** Chia văn bản theo kích thước cố định dựa trên ký tự phân tách như xuống dòng, dấu chấm. Đặc điểm là nhanh, dễ triển khai nhưng lại dễ cắt mạch văn, gây hiện tượng đứt gãy ngữ cảnh.
- **Semantic Chunking:** Sử dụng embedding để tính mức độ tương đồng giữa các câu, chỉ ngắt đoạn khi phát hiện sự thay đổi về chủ đề. Với cách làm này, ta thấy được nó sẽ giúp giữ trọn vẹn ngữ nghĩa đoạn văn, đảm bảo tính toàn vẹn thông tin nhưng sẽ tốn tài nguyên máy nhiều hơn.

Việc chuyển dịch từ Recursive sang Semantic Chunking xuất phát từ nhu cầu khắc phục sự rời rạc của dữ liệu đầu vào. Chúng ta kỳ vọng rằng khi các chunk chứa đựng thông tin trọn vẹn hơn, Retriever sẽ tìm kiếm chính xác hơn, tăng chỉ số Context Recall, từ đó tạo tiền đề để LLM trả lời đúng trọng tâm hơn.

Triển khai kỹ thuật

Quá trình chuyển đổi từ Baseline sang Semantic Chunking đòi hỏi sự thay đổi đồng bộ trong mã nguồn. Chúng ta sẽ tiến hành nâng cấp pipeline bắt đầu từ việc bổ sung các thư viện xử lý ngữ nghĩa, thiết lập ngưỡng nhảy cảm cho thuật toán, tái định nghĩa logic phân đoạn cốt lõi và cuối cùng là tích hợp bộ chia mới vào luồng khởi tạo hệ thống.

Cài đặt thư viện: Trước tiên, môi trường thực nghiệm cần được bổ sung thư viện `underthesea`. Đây là công cụ xử lý ngôn ngữ tự nhiên tối ưu cho tiếng Việt, đóng vai trò cốt lõi trong việc nhận diện chính xác ranh giới các câu, điều mà các bộ tách từ mặc định của tiếng Anh thường xử lý sai.

4.1 Cài đặt thư viện

```
1 # ... như baseline và bổ sung:  
2 !pip install -q "underthesea"
```

Cập nhật thư viện hỗ trợ: Tiếp theo, tại phần khai báo thư viện, chúng ta import module `sent_tokenize` từ `underthesea` để thực hiện tách câu và class `Document` từ `LangChain` để đóng gói kết quả sau khi cắt.

4.1 Khai báo thư viện

```
1 # ... như baseline và bổ sung:
2 from underthesea import sent_tokenize
3 from langchain_core.documents import Document
4 from langchain_huggingface import HuggingFaceEmbeddings
```

Thiết lập tham số cấu hình mới: Khác với việc cắt cứng nhắc theo ký tự, Semantic Chunking dựa vào độ tương đồng ngữ nghĩa. Chúng ta cần định nghĩa các tham số ngưỡng để kiểm soát độ "nhảy" của thuật toán.

- **SEMANTIC_BREAKPOINT_THRESHOLD:** Ngưỡng quyết định điểm gãy mạch văn. Nếu độ tương đồng giữa hai câu liên tiếp thấp hơn giá trị này (0.5), hệ thống sẽ ngắt chunk.
- **MIN_CHUNK_SIZE** và **MAX_CHUNK_SIZE:** Giới hạn kích thước chunk để tránh trường hợp chunk quá vụn vặt hoặc quá khổ.
- **CHUNK_OVERLAP:** Đảm bảo tính liên mạch thông tin giữa các đoạn cắt.

4.1 Cấu hình tham số phân đoạn ngữ nghĩa

```
1 # ... như baseline và bổ sung:
2 SEMANTIC_BREAKPOINT_THRESHOLD = 0.5
3 MIN_CHUNK_SIZE = 600
4 MAX_CHUNK_SIZE = 1024
5 CHUNK_OVERLAP = 128
```

Xây dựng bộ chia Semantic Chunker: Đây là bước thay đổi quan trọng nhất. Chúng ta xây dựng class `SemanticChunker` tùy chỉnh với logic xử lý được tối ưu hóa:

1. **Batch Processing:** Tính toán embedding cho toàn bộ các câu trong văn bản cùng một lúc để tăng tốc độ xử lý.
2. **Adaptive Splitting:** So sánh câu hiện tại (i) với câu liền trước ($i - 1$). Nếu độ tương đồng < 0.5 , hệ thống hiểu rằng chủ đề đã thay đổi và tiến hành ngắt đoạn.
3. **Overlap Handling:** Tự động thêm phần đuôi của chunk trước vào đầu chunk sau để giữ ngữ cảnh.

4.3 Triển khai thuật toán Semantic Chunking

```

1 class SemanticChunker:
2     def __init__(
3         self,
4         embedding_model: str = EMBEDDING_MODEL,
5         breakpoint_threshold: float = SEMANTIC_BREAKPOINT_THRESHOLD,
6         overlap_size: int = CHUNK_OVERLAP,
7     ):
8         self.breakpoint_threshold = breakpoint_threshold
9         self.overlap_size = overlap_size
10
11         print(f"Loading embedding model: {embedding_model}...")
12         self.embeddings = OpenAIEmbeddings(
13             model=embedding_model,
14             api_key=OPENAI_API_KEY
15         )
16
17     def _split_into_sentences(self, text: str) -> List[str]:
18         sentences = sent_tokenize(text)
19         return [s.strip() for s in sentences if s.strip() and len(s) > 20]
20
21     def _calculate_cosine_similarity(self, emb1, emb2):
22         return np.dot(emb1, emb2) / (np.linalg.norm(emb1) * np.linalg.norm(emb2))
23
24     def _chunk_by_semantic_similarity(self, sentences: List[str]) -> List[str]:
25         if not sentences:
26             return []
27
28         sentence_embeddings = self.embeddings.embed_documents(sentences)
29
30         chunks = []
31         current_chunk = [sentences[0]]
32
33         for i in range(1, len(sentences)):
34             prev_emb = sentence_embeddings[i-1]
35             curr_emb = sentence_embeddings[i]
36             similarity = self._calculate_cosine_similarity(prev_emb, curr_emb)
37
38             chunk_text = " ".join(current_chunk)
39             chunk_len = len(chunk_text)
40
41             if chunk_len >= MAX_CHUNK_SIZE:
42                 chunks.append(chunk_text)
43                 current_chunk = [sentences[i]]
44             elif similarity >= self.breakpoint_threshold or chunk_len < MIN_CHUNK_SIZE:
45                 current_chunk.append(sentences[i])
46             else:
47                 chunks.append(chunk_text)
48                 current_chunk = [sentences[i]]
49
50         if current_chunk:
51             chunks.append(" ".join(current_chunk))
52

```

```

53     return chunks
54
55     def split(self, documents) -> List:
56         all_chunks = []
57         for doc in tqdm(documents, desc="Semantic chunking"):
58             sentences = self._split_into_sentences(doc.page_content)
59             if not sentences: continue
60
61             chunks = self._chunk_by_semantic_similarity(sentences)
62
63             for idx, chunk_text in enumerate(chunks):
64                 if not chunk_text.strip(): continue
65
66                 if idx > 0 and self.overlap_size > 0:
67                     prev_chunk = chunks[idx-1]
68                     if len(prev_chunk) >= self.overlap_size:
69                         overlap = prev_chunk[-self.overlap_size:]
70                         chunk_text = overlap + " " + chunk_text
71
72                 chunk_doc = Document(
73                     page_content=chunk_text,
74                     metadata=doc.metadata.copy()
75                 )
76                 all_chunks.append(chunk_doc)
77
78     return all_chunks

```

Khởi tạo lại hệ thống: Cuối cùng, tại bước khởi tạo Pipeline, chúng ta thay thế `TextSplitter` cũ bằng `SemanticChunker` mới được khởi tạo với các tham số đã cấu hình. Các thành phần khác như LLM và VectorDB được giữ nguyên để đảm bảo tính công bằng khi so sánh.

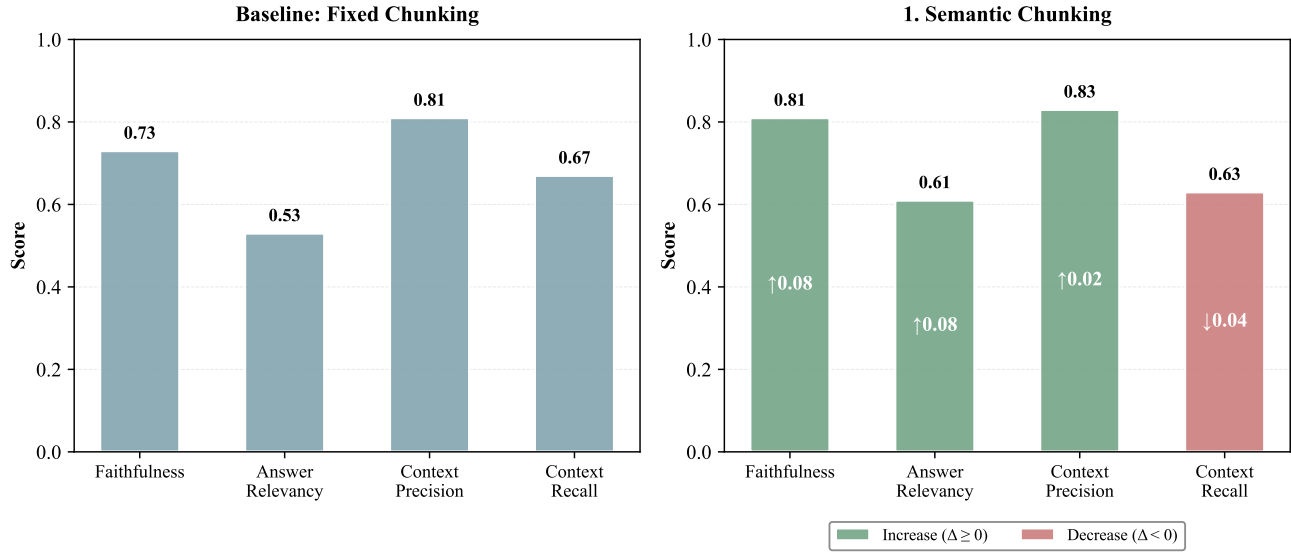
4.5 Khởi tạo RAG

```

1  # ... như baseline và thay đổi hàm Splitter thành SemanticChunker:
2  chunker = SemanticChunker(
3      embedding_model=EMBEDDING_MODEL,
4      breakpoint_threshold=SEMANTIC_BREAKPOINT_THRESHOLD,
5      overlap_size=CHUNK_OVERLAP
6  )

```

Kết quả thực nghiệm với Semantic Chunking



Hình 11: Kết quả đánh giá RAGAS với Semantic Chunking.

Quan sát Hình 11, khi so sánh với kết quả Baseline, ta nhận thấy sự thay đổi rõ rệt về chất lượng:

- **Cải thiện chất lượng sinh:** Chỉ số Faithfulness tăng mạnh từ 0.73 → 0.81 và Answer Relevancy cải thiện đáng kể, từ 0.53 → 0.61. Điều này xác nhận giả thuyết ban đầu: khi ngữ cảnh được giữ trọn vẹn và không bị cắt vụn, LLM hiểu vấn đề tốt hơn, từ đó đưa ra câu trả lời sát thực tế và đúng trọng tâm hơn.
- **Thay đổi về chỉ số độ bao phủ:** Tuy nhiên, chỉ số Context Recall lại giảm nhẹ từ 0.67 → 0.63. Nguyên nhân có thể do thuật toán gom nhóm ngữ nghĩa quá chặt chẽ, khiến một số thông tin liên quan nhưng nằm rải rác hoặc có ngữ nghĩa không quá gần nhau bị loại bỏ khỏi top-k kết quả tìm kiếm.

Kết luận: Từ thực nghiệm trên, ta thấy rằng Semantic Chunking giúp hệ thống “thông minh” hơn trong việc trả lời, nhưng đòi hỏi phải tinh chỉnh kỹ lưỡng ngưỡng threshold để không làm mất mát các thông tin quan trọng.

VII.5.2. Thực nghiệm 2: Retrieval Strategies

Sau khi tối ưu hóa giai đoạn Indexing, chúng ta chuyển sang nâng cấp thành phần cốt lõi tiếp theo: Retrieval. Hệ thống Baseline hiện tại đang sử dụng thuần Dense Retrieval. Mặc dù phương pháp này vượt trội trong việc nắm bắt ngữ nghĩa và các từ đồng nghĩa, tuy nhiên nó vẫn thường bộc lộ hạn chế khi xử lý các truy vấn đòi hỏi độ chính xác tuyệt đối về mặt từ khóa, ví dụ như: tên riêng, mã số sản phẩm, hoặc các thuật ngữ chuyên ngành hiếm gặp. Để khắc phục điểm yếu này, chúng ta sẽ lần lượt triển khai và đánh giá ba chiến lược tìm kiếm nâng cao: Sparse Retrieval với BM25, Hybrid Search với Interleaving, và Hybrid Search kết hợp Reciprocal Rank Fusion.

1. Sparse Retrieval với BM25 Dựa trên cơ sở lý thuyết tại mục IV., BM25 được xem là chuẩn mực cho các thuật toán tìm kiếm dựa trên từ khóa. Tuy nhiên, khi áp dụng cho tiếng Việt, đặc thù ngôn ngữ đòi hỏi một quy trình Tokenization chuyên biệt. Việc tách từ theo khoảng trắng mặc định sẽ phá vỡ cấu trúc từ ghép, biến chúng thành các đơn vị rời rạc và làm sai lệch ngữ nghĩa. Do đó, yêu cầu bắt buộc trong thực nghiệm này là tích hợp kỹ thuật Word Tokenization để bảo toàn đơn vị ngữ nghĩa của tiếng Việt.

Cài đặt thư viện: Cài đặt thư viện `rank-bm25` để thực thi thuật toán và `underthesea` phục vụ xử lý ngôn ngữ tự nhiên tiếng Việt.

4.1 Cài đặt thư viện

```
1 # ... như baseline và bổ sung:
2 !pip install -q \
3     "rank-bm25" \
4     "underthesea"
```

Cập nhật thư viện hỗ trợ: Tại đây, ta khai báo các module cần thiết, bao gồm thuật toán BM25Okapi và hàm `word_tokenize` để hỗ trợ tách từ.

4.1 Cập nhật thư viện

```
1 # ... như baseline và bổ sung:
2 from rank_bm25 import BM25Okapi
3 from underthesea import word_tokenize
4 from langchain_core.retrievers import BaseRetriever
5 from langchain_core.documents import Document
6 from pydantic import Field
7 from typing import List, Any
```

Xây dựng class BM25Retriever: Tại thực nghiệm này, chúng ta sẽ thay thế class `VectorDB` của baseline bằng class `BM25Retriever`. Class này kế thừa từ `BaseRetriever`, tích hợp sẵn logic tách từ tiếng Việt vào quá trình xử lý văn bản và truy vấn.

4.3 Triển khai Sparse Retrieval với BM25

```
1 class BM25Retriever(BaseRetriever):
2     documents: List[Any] = Field(description="List of documents")
3     bm25: Any = Field(description="BM25 index")
4     k: int = Field(default=5, description="Number of documents to retrieve")
5
6     @classmethod
7     def from_documents(cls, documents: List, k: int = 5):
8         tokenized_docs = [
9             word_tokenize(doc.page_content.lower())
10            for doc in documents
11        ]
12        bm25 = BM25Okapi(tokenized_docs)
13        return cls(documents=documents, bm25=bm25, k=k)
14
```

```

15 def _get_relevant_documents(self, query: str, *, run_manager=None) -> List[Document
    ]:
16     tokenized_query = word_tokenize(query.lower())
17     bm25_scores = self.bm25.get_scores(tokenized_query)
18     top_k_indices = np.argsort(bm25_scores)[: -1][:self.k]
19     return [self.documents[i] for i in top_k_indices]

```

Khởi tạo hệ thống với BM25: Tại bước khởi tạo pipeline, chúng ta loại bỏ thành phần Embedding và VectorDB, thay thế trực tiếp bằng BM25Retriever để thiết lập mốc so sánh cho phương pháp tìm kiếm từ khóa.

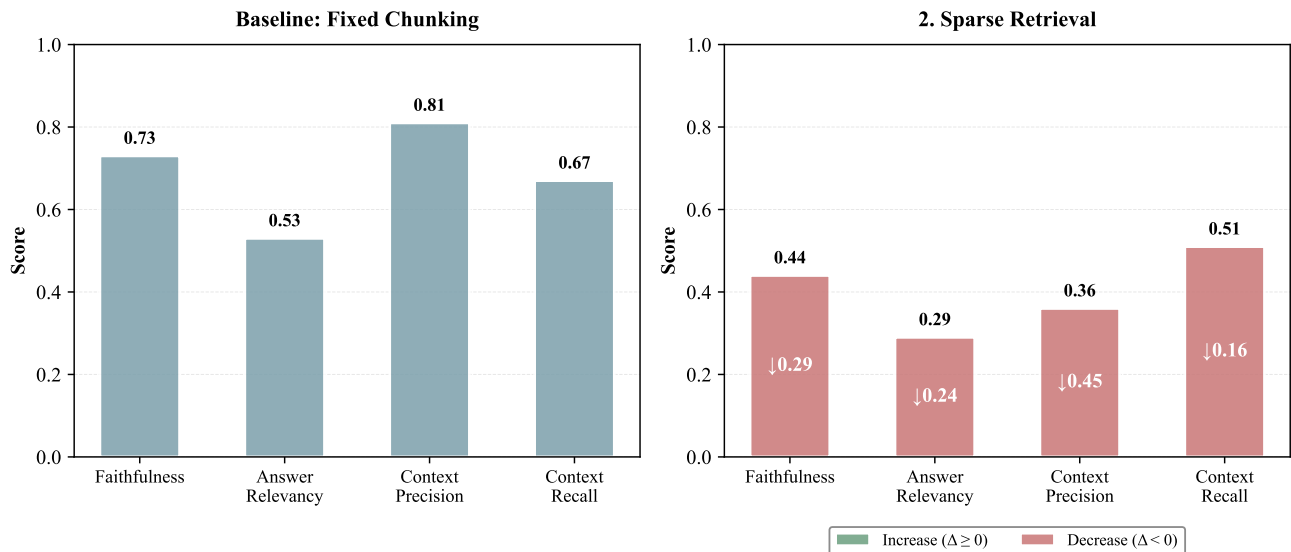
4.5 Cài đặt thư viện

```

1 # ... như baseline và cập nhật hàm khởi tạo retriever:
2 retriever = BM25Retriever.from_documents(documents=split_docs, k=5)

```

Kết quả thực nghiệm với BM25 Retrieval



Hình 12: Kết quả đánh giá RAGAS với chiến lược BM25.

Quan sát Hình 12, ta thấy rằng hiệu năng bị giảm sút nghiêm trọng so với Baseline ở mọi chỉ số:

- **Hiệu suất thấp:** Điểm số ở cả 4 metric đều rất thấp. Context Precision chỉ đạt 0.36 và Answer Relevancy rớt xuống 0.29.
- **Nguyên nhân:** BM25 hoạt động dựa trên nguyên tắc khớp từ khóa chính xác. Trong khi đó, các câu hỏi trong tập testset thường sử dụng ngôn ngữ tự nhiên, từ đồng nghĩa hoặc mô tả gián tiếp. Vì không nắm bắt được ngữ nghĩa sâu sắc như Vector Search, BM25 đã thất bại trong việc tìm kiếm các văn bản phù hợp, dẫn đến câu trả lời của LLM bị sai lệch hoặc thiếu thông tin.

Kết luận: Từ thực nghiệm, ta có thể kết luận rằng BM25 đơn lẻ không đủ mạnh để xử lý các truy vấn ngữ nghĩa phức tạp, nhưng nó vẫn là mảnh ghép cần thiết để bù đắp cho Vector Search trong các trường hợp tìm kiếm các từ khóa định danh cụ thể.

2. Hybrid Search kết hợp Interleaving Mỗi phương pháp truy xuất đều sở hữu ưu thế riêng: BM25 tối ưu cho sự khớp chính xác các từ khóa, trong khi Dense Retrieval mạnh về sự tương đồng ngữ nghĩa. Vì thế, Hybrid Search được ra đời nhằm trung hòa sức mạnh của cả hai phương pháp.

Trong chiến lược này, chúng ta áp dụng phương pháp Interleaving. Cơ chế này hoạt động bằng cách lấy xen kẽ kết quả từ hai danh sách theo thứ tự ưu tiên: Top 1 BM25 → Top 1 Vector → Top 2 BM25... Phương pháp này đảm bảo tính công bằng và đặc biệt hữu ích khi muốn tránh sự phức tạp của việc chuẩn hóa hai thang điểm hoàn toàn khác biệt vì điểm BM25 thường > 0 và không có giới hạn trên, trong khi Cosine Similarity nằm trong khoảng $[0, 1]$.

Xây dựng HybridRetriever: Định nghĩa class `HybridRetriever`, thực hiện gửi truy vấn song song đến cả hai bộ tìm kiếm và hợp nhất danh sách kết quả theo cơ chế Interleaving, đồng thời loại bỏ các văn bản trùng lặp.

4.3 Xây dựng HybridRetriever

```

1 class HybridRetriever(BaseRetriever):
2     bm25_retriever: Any = Field(description="BM25 retriever instance")
3     vector_retriever: Any = Field(description="Vector retriever instance")
4     top_k: int = Field(default=5, description="Final number of documents")
5
6     @classmethod
7     def from_retrievers(cls, bm25_retriever, vector_retriever, top_k: int = 5):
8         return cls(
9             bm25_retriever=bm25_retriever,
10            vector_retriever=vector_retriever,
11            top_k=top_k
12        )
13
14     def _get_relevant_documents(self, query: str, *, run_manager=None) -> List[Document]:
15         bm25_docs = self.bm25_retriever.invoke(query)
16         vector_docs = self.vector_retriever.invoke(query)
17
18         merged_docs = []
19         seen_contents = set()
20
21         max_len = max(len(bm25_docs), len(vector_docs))
22         for i in range(max_len):
23             if i < len(bm25_docs):
24                 doc = bm25_docs[i]
25                 if doc.page_content not in seen_contents:
26                     merged_docs.append(doc)
27                     seen_contents.add(doc.page_content)
28
29             if i < len(vector_docs):

```

```

30         doc = vector_docs[i]
31         if doc.page_content not in seen_contents:
32             merged_docs.append(doc)
33             seen_contents.add(doc.page_content)
34
35         if len(merged_docs) >= self.top_k: break
36
37     return merged_docs[:self.top_k]

```

Khởi tạo hệ thống Hybrid: Khởi tạo hệ thống Hybrid bằng cách thiết lập đồng thời BM25Retriever và VectorRetriever, sau đó kết nối chúng vào pipeline thông qua HybridRetriever.

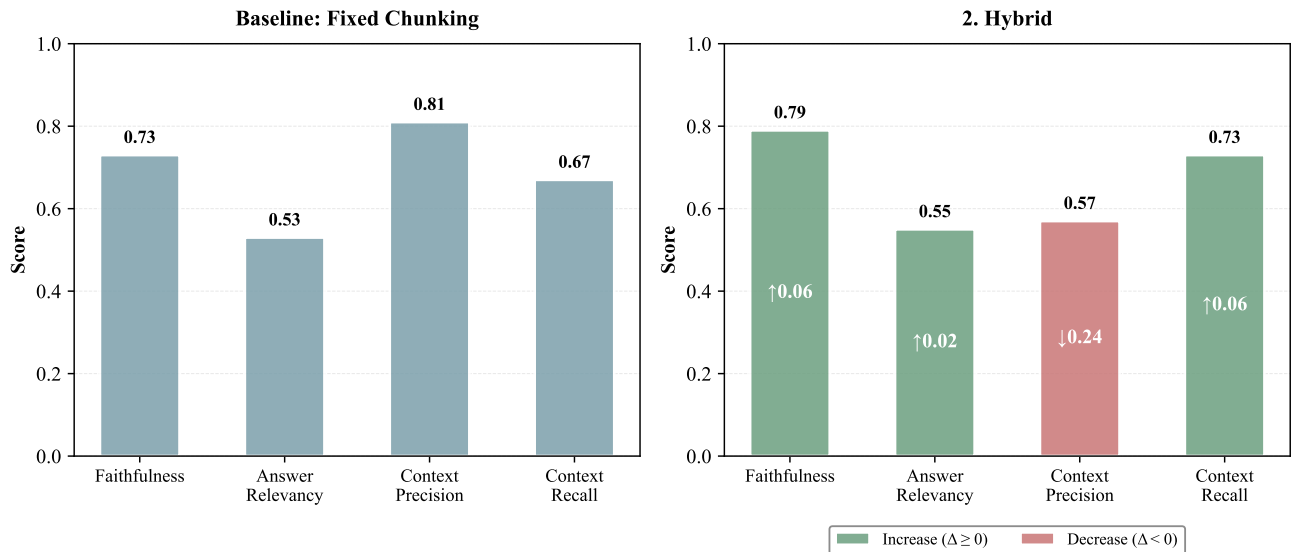
4.5 Khởi tạo RAG với Hybrid Search

```

1  # ... như baseline và cập nhật hàm khởi tạo retriever, vector database:
2  bm25_retriever = BM25Retriever.from_documents(documents=split_docs, k=5)
3
4  vdb = VectorDB(documents=split_docs, embedding=embeddings)
5  vector_retriever = vdb.get_retriever(search_kwargs={"k": 5})
6
7  retriever = HybridRetriever.from_retrievers(
8      bm25_retriever=bm25_retriever,
9      vector_retriever=vector_retriever,
10     top_k=5
11 )

```

Kết quả thực nghiệm với Hybrid Search Interleaving



Hình 13: Kết quả đánh giá RAGAS với chiến lược Hybrid Search Interleaving.

Quan sát Hình 13 và đối chiếu với Baseline, ta thấy một sự đánh đổi rõ rệt giữa độ bao phủ và

độ chính xác:

- **Cải thiện đáng kể về Recall:** Chỉ số Context Recall tăng từ 0.67 lên 0.73. Việc kết hợp hai luồng tìm kiếm giúp hệ thống “quét” được rộng hơn, không bỏ sót các văn bản chứa từ khóa quan trọng nhờ BM25 mà Vector Search có thể đã lướt qua.
- **Sự sụt giảm về Precision:** Ta thấy rằng, Context Precision lại giảm mạnh từ 0.81 xuống 0.57. Nguyên nhân là do cơ chế Interleaving hoạt động khá máy móc, nó buộc phải lấy xen kẽ kết quả từ BM25, cái vốn có độ chính xác thấp, chỉ 0.36 như đã thấy ở thí nghiệm trước, sau đó trộn lẫn với kết quả Vector. Điều này vô tình đưa nhiều thông tin nhiễu vào trong context cuối cùng gửi cho LLM.

Kết luận: Interleaving thành công trong việc mở rộng phạm vi tìm kiếm, tuy nhiên lại làm loãng mật độ thông tin hữu ích. Điều này đặt ra nhu cầu cần một cơ chế xếp hạng thông minh hơn để chỉ giữ lại những kết quả tốt nhất từ cả hai phía thay vì lấy xen kẽ một cách mù quáng, đây chính là lý do ta cần đến thuật toán RRF ở phần tiếp theo.

3. Hybrid Search với Reciprocal Rank Fusion Mặc dù Interleaving giải quyết được vấn đề chênh lệch thang điểm, nó lại vô tình đánh đồng thứ hạng giữa các thuật toán và bỏ qua sự “đồng thuận”. Về mặt lý thuyết, nếu một tài liệu xuất hiện ở vị trí cao trong cả hai danh sách truy xuất, tài liệu đó nên được đánh giá quan trọng hơn hẳn so với việc chỉ xuất hiện ở một bên.

Để hiện thực hóa điều này, ta sử dụng thuật toán Reciprocal Rank Fusion (RRF). RRF tính điểm dựa trên thứ hạng nghịch đảo, giúp lọc bỏ nhiễu và đưa các tài liệu có độ tin cậy cao nhất lên đầu danh sách kết quả.

Cấu hình tham số Thiết lập tham số HYBRID_K nhằm biểu thị số lượng kết quả đầu ra mong muốn và RRF_K là hằng số làm mượt mẫu số trong công thức RRF.

4.1 Cấu hình tham số RRF

```
1 HYBRID_K = 7
2 RRF_K = 40
```

Xây dựng HybridVectorDB: Class HybridVectorDB đóng vai trò wrapper, quản lý việc khởi tạo và lưu trữ đồng thời cả Vector Index ChromaDB và Keyword Index BM25 từ bộ dữ liệu đầu vào.

4.3 Xây dựng lớp HybridVectorDB

```
1 class HybridVectorDB:
2     def __init__(
3         self,
4         documents=None,
5         embedding=None,
6         collection_name: str = "vietnamese_docs",
7         persist_dir: str = "/content/chroma_data",
8     ):
```

```

9         self.persist_dir = persist_dir
10        self.collection_name = collection_name
11        self.embedding = embedding or get_embeddings()
12        self.documents = documents or []
13
14        self.vector_db = self._build_vector_db(documents)
15
16        self.bm25 = self._build_bm25_index(documents)
17
18    def _build_vector_db(self, documents):
19        if documents is None or len(documents) == 0:
20            db = Chroma(
21                collection_name=self.collection_name,
22                embedding_function=self.embedding,
23                persist_directory=self.persist_dir,
24            )
25        else:
26            db = Chroma.from_documents(
27                documents=documents,
28                embedding=self.embedding,
29                collection_name=self.collection_name,
30                persist_directory=self.persist_dir,
31            )
32        return db
33
34    def _build_bm25_index(self, documents):
35        if documents is None or len(documents) == 0:
36            print("No documents for BM25 index")
37            return None
38
39        tokenized_docs = [
40            word_tokenize(doc.page_content.lower())
41            for doc in documents
42        ]
43        bm25 = BM25Okapi(tokenized_docs)
44        return bm25
45
46    def get_retriever(self, search_kwargs: dict = None):
47        if search_kwargs is None:
48            search_kwargs = {"k": HYBRID_K}
49
50        return HybridRetriever(
51            vector_db=self.vector_db,
52            bm25=self.bm25,
53            documents=self.documents,
54            k=search_kwargs.get("k", HYBRID_K),
55            rrf_k=RRF_K
56        )

```

Xây dựng HybridRetriever với RRF Logic: Cài đặt logic thuật toán RRF bên trong HybridRetriever. Đoạn code tính toán lại điểm số cho từng tài liệu dựa trên vị trí xếp hạng của nó trong các danh sách thành phần và sắp xếp lại kết quả cuối cùng.

4.3 Cập nhật thuật toán RRF cho HybridRetriever

```

1 class HybridRetriever(BaseRetriever):
2     vector_db: Chroma
3     bm25: object
4     documents: List
5     k: int = HYBRID_K
6     rrf_k: int = RRF_K
7
8     class Config:
9         arbitrary_types_allowed = True
10
11     def _get_relevant_documents(
12         self, query: str, *, run_manager: CallbackManagerForRetrieverRun = None
13     ) -> List[LangchainDocument]:
14         results_list = []
15
16         vector_results = self.vector_db.similarity_search(query, k=self.k)
17         results_list.append(vector_results)
18
19         if self.bm25 is not None:
20             tokenized_query = word_tokenize(query.lower())
21             bm25_scores = self.bm25.get_scores(tokenized_query)
22
23             top_k_indices = np.argsort(bm25_scores)[::-1][:self.k]
24             bm25_results = [self.documents[i] for i in top_k_indices]
25             results_list.append(bm25_results)
26
27         fused_results = self._reciprocal_rank_fusion(results_list, k=self.rrf_k)
28
29         return fused_results[:self.k]
30
31     def _reciprocal_rank_fusion(self, results_list: List[List], k: int) -> List[
32                                     LangchainDocument]:
33         from collections import defaultdict
34
35         rrf_scores = defaultdict(float)
36         doc_content_map = {}
37
38         for results in results_list:
39             for rank, doc in enumerate(results, start=1):
40                 doc_id = doc.page_content
41                 rrf_scores[doc_id] += 1 / (k + rank)
42                 doc_content_map[doc_id] = doc
43
44         sorted_docs = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)
45         return [doc_content_map[doc_id] for doc_id, score in sorted_docs]

```

Khởi tạo hệ thống Hybrid RRF Khi lắp ráp toàn bộ các thành phần vào pipeline RAG cuối cùng. Hệ thống lúc này đạt được sự cân bằng tối ưu: vừa bắt đúng từ khóa quan trọng, vừa hiểu ngữ cảnh, và có cơ chế xếp hạng thông minh.

4.5 Khởi tạo RAG với Hybrid RRF

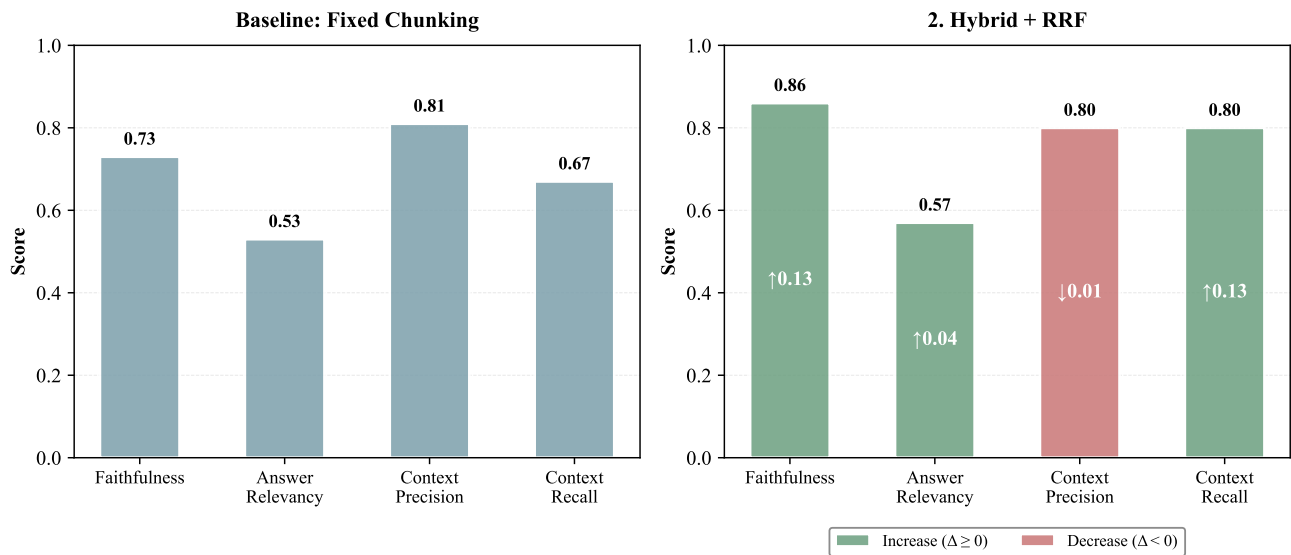
```

1 # ... như baseline và cập nhật hàm khởi tạo Vector Database và Retriever
2 vdb = HybridVectorDB(documents=split_docs, embedding=embeddings)
3 retriever = vdb.get_retriever(search_kwargs={"k": HYBRID_K})

```

Với cấu hình này, hệ thống RAG đạt được sự cân bằng tối ưu: vừa đảm bảo khả năng bắt đúng từ khóa quan trọng nhờ BM25, vừa duy trì sự phù hợp về mặt ngữ nghĩa nhờ Vector Search, đồng thời cơ chế RRF giúp lọc bỏ nhiễu và đưa các tài liệu có độ đồng thuận cao lên đầu danh sách.

Kết quả thực nghiệm với Hybrid Search + RRF



Hình 14: Kết quả đánh giá RAGAS với chiến lược Hybrid Search kết hợp RRF Fusion.

Hình 14 cho thấy được sự vượt trội toàn diện của chiến lược RRF khi so với các phương pháp trước đó khi:

- **Cân bằng hoàn hảo giữa Precision và Recall:** Cả hai chỉ số Context Precision và Context Recall đều đồng loạt đạt mức rất cao là 0.80. Điều này chứng minh rằng thuật toán RRF đã thành công trong việc lọc bỏ các tài liệu nhiễu, khắc phục được điểm yếu của Interleaving đồng thời vẫn giữ lại đầy đủ thông tin quan trọng từ cả hai nguồn từ khóa và vector.
- **Chất lượng sinh văn bản tối ưu:** Chỉ số Faithfulness đạt đỉnh điểm ở mức 0.86, cao nhất trong tất cả các thực nghiệm tính đến hiện tại. Khi đầu vào có chất lượng cao và được xếp hạng chính xác, LLM sẽ có cơ sở vững chắc để đưa ra câu trả lời trung thực và đáng tin cậy.
- **Answer Relevancy:** Mặc dù vẫn giữ ở mức trung bình là 0.57, nhưng sự ổn định này cho thấy vấn đề về độ súc tích của câu trả lời có thể nằm ở khâu Prompting hoặc bản chất của mô hình sinh, chứ không còn do thiếu hụt thông tin đầu vào.

Kết luận: Hybrid Search kết hợp RRF là chiến lược Retrieval mạnh mẽ trong chuỗi thực nghiệm này. Nó giúp giải quyết triệt để bài toán đánh đổi giữa “tìm đủ” và “tìm đúng”, xứng đáng là một trong các giải pháp tối ưu được ưu tiên hàng đầu cho các bài toán RAG cần độ chính xác cao.

VII.5.3. Thực nghiệm 3: Query Transformations

- Hypothetical Document Embedding (HyDE).** Đoạn code dưới đây xây dựng lại luồng xử lý RAG theo batch, nhưng thay vì truy vấn trực tiếp bằng câu hỏi gốc, hệ thống trước hết sinh ra một đoạn văn giả định mang tính chuyên ngành nhờ LLM, sau đó dùng đoạn văn này để truy vấn vào Vector Database. Cách làm này giúp embedding của truy vấn gần hơn với không gian ngữ nghĩa của tài liệu, từ đó cải thiện chất lượng retrieval cho các câu hỏi ngắn hoặc thiếu ngữ cảnh.

4.4 Xây dựng luồng xử lý cho RAG

```

1 class FocusedAnswerParser(StrOutputParser):
2     # ... như baseline
3
4 class HyDEQueryTransformer:
5     def __init__(self, llm):
6         self.llm = llm
7         self.prompt = PromptTemplate.from_template("""Hãy viết một đoạn văn ngắn (khoảng
8             100-150 từ) trả lời câu hỏi sau.
9             Viết như thể đây là một đoạn trích từ tài liệu chuyên ngành hoặc sách giáo khoa.
10            Đoạn văn nên chứa các thông tin, thuật ngữ kỹ thuật liên quan đến chủ đề.
11            Không cần bắt đầu bằng "Theo tài liệu" hay các cụm từ tương tự.
12            Câu hỏi: {question}
13
14            Đoạn văn: """)
15
16     def generate_hypothetical_doc(self, question: str) -> str:
17         response = self.llm.invoke(self.prompt.format(question=question))
18         return response.content.strip()
19
20 class HyDEBatchRAG:
21     """RAG với HyDE - Sử dụng Hypothetical Document để cải thiện retrieval"""
22
23     def __init__(self, llm, vector_db, batch_size: int = BATCH_SIZE):
24         self.llm = llm
25         self.vector_db = vector_db
26         self.batch_size = batch_size
27         self.hyde_transformer = HyDEQueryTransformer(llm)
28         self.prompt_template = """Bạn là trợ lý AI phân tích tài liệu tiếng Việt.
29
30         [TÀI LIỆU]:
31         {context}
32
33         [CÂU HỎI]:
34         {question}

```

```

35
36 Hãy trả lời dựa trên tài liệu. Nếu tài liệu không có thông tin, nói rõ "Không có thông
    tin".
37 [TRẢ LỜI]:"""
38     self.prompt = PromptTemplate.from_template(self.prompt_template)
39     self.answer_parser = FocusedAnswerParser()
40
41     def _format_docs(self, docs):
42         # ... như baseline
43
44     def hyde_retrieve(self, question: str, k: int = 3) -> dict:
45         # Tạo hypothetical document
46         hypothetical_doc = self.hyde_transformer.generate_hypothetical_doc(question)
47
48         # Sử dụng hypothetical document để retrieve
49         docs = self.vector_db.similarity_search(hypothetical_doc, k=k)
50
51         return {
52             "question": question,
53             "hypothetical_doc": hypothetical_doc,
54             "docs": docs,
55             "contexts": [doc.page_content for doc in docs],
56             "formatted_context": self._format_docs(docs)
57         }
58
59     def batch_retrieve(self, questions: List[str], k: int = 3) -> List[dict]:
60         all_data = []
61         for question in tqdm(questions, desc="HyDE Retrieving"):
62             data = self.hyde_retrieve(question, k=k)
63             all_data.append(data)
64         return all_data
65
66     def batch_generate(self, prompts: List[str]) -> List[str]:
67         # ... như baseline
68
69     def answer_with_contexts_batch(self, questions: List[str], k: int = 3):
70         print(f"Processing {len(questions)} questions with HyDE...")
71         print(f"Batch size: {self.batch_size}")
72
73         retrieved_data = self.batch_retrieve(questions, k=k)
74
75         prompts = [
76             self.prompt.format(
77                 context=data["formatted_context"],
78                 question=data["question"]
79             )
80             for data in retrieved_data
81         ]
82         answers = self.batch_generate(prompts)
83
84         results = []
85         for data, answer in zip(retrieved_data, answers):
86             results.append({

```

```

87         "answer": answer,
88         "contexts": data["contexts"],
89         "hypothetical_doc": data["hypothetical_doc"]
90     })
91
92     return results

```

Khởi tạo RAG theo lớp HyDEBatchRAG thay cho batch RAG thông thường. Luồng xử lý phía sau vẫn giữ nguyên, chỉ thay đổi chiến lược truy vấn ở bước retrieval.

4.5. Khởi tạo RAG

```

1 # ...
2 rag = HyDEBatchRAG(llm, vector_db=vdb, batch_size=BATCH_SIZE)
3 # ...

```

Đoạn code dưới đây thực thi suy luận batch cho toàn bộ tập câu hỏi trong testset, đo thời gian chạy, sau đó ghép kết quả trả lời với thông tin ground truth để tạo DataFrame `eval_df` dùng cho đánh giá Ragas.

4.5 Thực thi RAG với HyDE

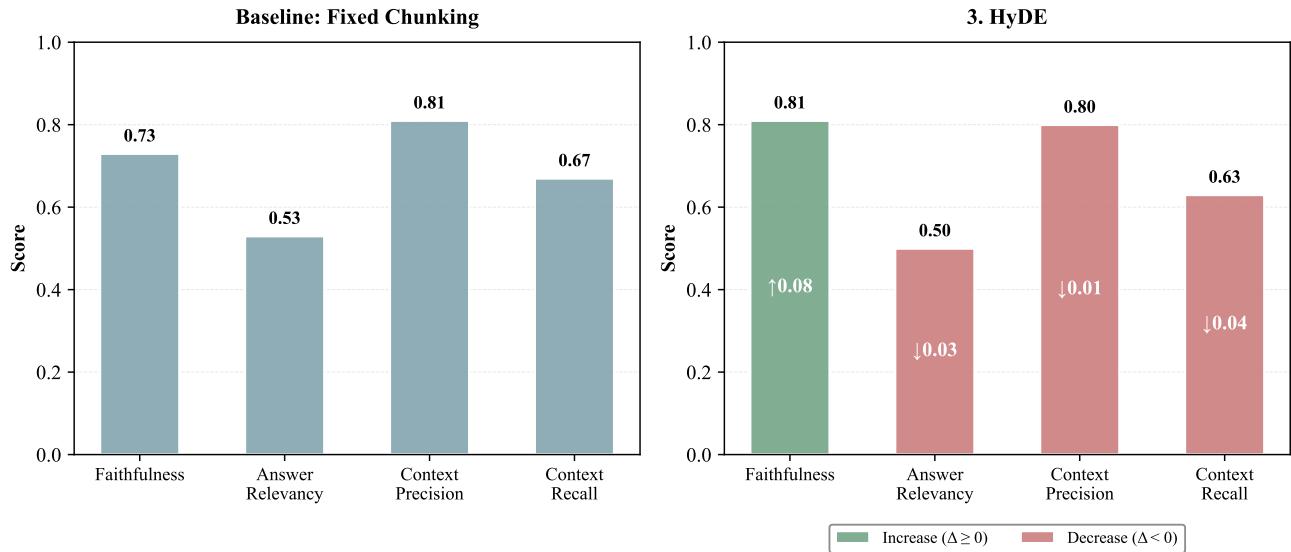
```

1 set_global_seed(SEED)
2
3 print(f"Starting HyDE RAG inference...")
4 print(f"LLM: {LLM_MODEL}")
5 print(f"Batch size: {BATCH_SIZE}")
6
7 questions = test_df["user_input"].tolist()
8
9 start_time = time.time()
10 batch_results = rag.answer_with_contexts_batch(questions, k=3)
11 elapsed_time = time.time() - start_time
12
13 print(f"Inference completed in {elapsed_time:.2f} seconds")
14 print(f"Average time per question: {elapsed_time/len(questions):.2f} seconds")
15
16 results = []
17 for idx, (result, (_, row)) in enumerate(zip(batch_results, test_df.iterrows())):
18     results.append({
19         "user_input": row["user_input"],
20         "response": result["answer"],
21         "retrieved_contexts": result["contexts"],
22         "reference": row.get("reference", ""),
23         "reference_contexts": row.get("reference_contexts", []),
24         "hypothetical_doc": result.get("hypothetical_doc", "")
25     })
26
27 eval_df = pd.DataFrame(results)
28 print(f"Collected {len(eval_df)} predictions")

```

```
29 display(eval_df.head())
```

Kết quả thực nghiệm với HyDE



Hình 15: Kết quả đánh giá RAGAS với chiến lược HyDE.

Quan sát Hình 15, ta thấy chiến lược Hypothetical Document Embedding mang lại những thay đổi so với Baseline như sau:

- **Cải thiện tính trung thực:** Chỉ số Faithfulness tăng 0.08 điểm, đạt mức 0.81. Việc tìm kiếm dựa trên văn bản giả định giúp mô hình nắm bắt tốt hơn các thuật ngữ chuyên ngành, tạo tiền đề ngữ cảnh tốt cho quá trình sinh câu trả lời.
- **Sự sụt giảm ở các chỉ số khác:** Answer Relevancy giảm nhẹ 0.03 điểm xuống còn 0.50. Tương tự, Context Recall giảm xuống 0.63 (giảm 0.04 điểm so với baseline).
- **Nguyên nhân:** Do HyDE dựa vào khả năng sáng tạo của LLM để viết câu trả lời giả định. Nếu đoạn văn này chứa thông tin sai lệch, nó sẽ dẫn dắt bộ truy xuất tìm kiếm sai hướng, làm loãng độ chính xác của ngữ cảnh thực tế trong cơ sở dữ liệu.

2. Decomposition. Với chiến lược Query Decomposition, thay vì hỏi trực tiếp một câu dài hoặc phức tạp, hệ thống nhờ LLM phân tách câu hỏi gốc thành một tập các câu hỏi con đơn giản hơn, sau đó truy vấn từng câu hỏi con để thu thập thêm ngữ cảnh trước khi tổng hợp trả lời. Cách làm này thường giúp cải thiện chất lượng truy vấn đối với các câu hỏi đa bước hoặc chứa nhiều ý nhỏ.

4.4 Xây dựng luồng xử lý cho RAG

```

1 class FocusedAnswerParser(StrOutputParser):
2     # ... như baseline
3
4 class DecompositionQueryTransformer:
5     def __init__(self, llm):
6         self.llm = llm
7         self.prompt = PromptTemplate.from_template("""Hãy phân tách câu hỏi sau thành c
8             ác câu hỏi con đơn giản hơn.
9             Mỗi câu hỏi con nên tập trung vào một khía cạnh cụ thể của câu hỏi gốc.
10            Trả về tối đa 3 câu hỏi con, mỗi câu trên một dòng.
11            Nếu câu hỏi đã đơn giản, chỉ trả về chính câu hỏi đó.
12            Câu hỏi gốc: {question}
13
14            Các câu hỏi con: """)
15
16    def decompose(self, question: str) -> List[str]:
17        response = self.llm.invoke(self.prompt.format(question=question))
18        sub_questions = [q.strip() for q in response.content.strip().split("\n") if q.
19            strip()]
20
21        cleaned = []
22        for q in sub_questions:
23            cleaned_q = re.sub(r"^\d+\.\s*", "", q).strip()
24            if cleaned_q:
25                cleaned.append(cleaned_q)
26        return cleaned[:3]
27
28 class DecompositionBatchRAG:
29     def __init__(self, llm, batch_size: int = BATCH_SIZE):
30         self.llm = llm
31         self.batch_size = batch_size
32         self.decomposer = DecompositionQueryTransformer(llm)
33         self.prompt_template = """Bạn là trợ lý AI phân tích tài liệu tiếng Việt.
34
35        [TÀI LIỆU]:
36        {context}
37
38        [CÂU HỎI GỐC]:
39        {question}
40
41        [CÁC CÂU HỎI CON ĐÃ PHÂN TÁCH]:
42        {sub_questions}
43
44        Hãy trả lời câu hỏi gốc dựa trên tài liệu, sử dụng thông tin từ các câu hỏi con làm hướ
45            ng dẫn.
46        Nếu tài liệu không có thông tin, nói rõ "Không có thông tin".
47        [TRẢ LỜI]: """)
48
49    def _format_docs(self, docs):
50        self.prompt = PromptTemplate.from_template(self.prompt_template)
51        self.answer_parser = FocusedAnswerParser()
52
53    def _format_docs(self, docs):

```

```

50     # ... như baseline
51
52     def decompose_and_retrieve(self, question: str, retriever) -> dict:
53         sub_questions = self.decomposer.decompose(question)
54
55         all_docs = []
56         seen_contents = set()
57
58         for sub_q in sub_questions:
59             docs = retriever.invoke(sub_q)
60             for doc in docs:
61                 if doc.page_content not in seen_contents:
62                     all_docs.append(doc)
63                     seen_contents.add(doc.page_content)
64
65         original_docs = retriever.invoke(question)
66         for doc in original_docs:
67             if doc.page_content not in seen_contents:
68                 all_docs.append(doc)
69                 seen_contents.add(doc.page_content)
70
71         return {
72             "question": question,
73             "sub_questions": sub_questions,
74             "docs": all_docs,
75             "contexts": [doc.page_content for doc in all_docs],
76             "formatted_context": self._format_docs(all_docs)
77         }
78
79     def batch_retrieve(self, questions: List[str], retriever) -> List[dict]:
80         all_data = []
81         for question in tqdm(questions, desc="Decomposing & Retrieving"):
82             data = self.decompose_and_retrieve(question, retriever)
83             all_data.append(data)
84         return all_data
85
86     def batch_generate(self, prompts: List[str]) -> List[str]:
87         # ... như baseline
88
89     def answer_with_contexts_batch(self, questions: List[str], retriever):
90         print(f"Processing {len(questions)} questions with Query Decomposition...")
91         print(f"Batch size: {self.batch_size}")
92
93         retrieved_data = self.batch_retrieve(questions, retriever)
94
95         prompts = [
96             self.prompt.format(
97                 context=data["formatted_context"],
98                 question=data["question"],
99                 sub_questions="\n".join(f"- {sq}" for sq in data["sub_questions"])
100             )
101             for data in retrieved_data
102         ]
103         answers = self.batch_generate(prompts)

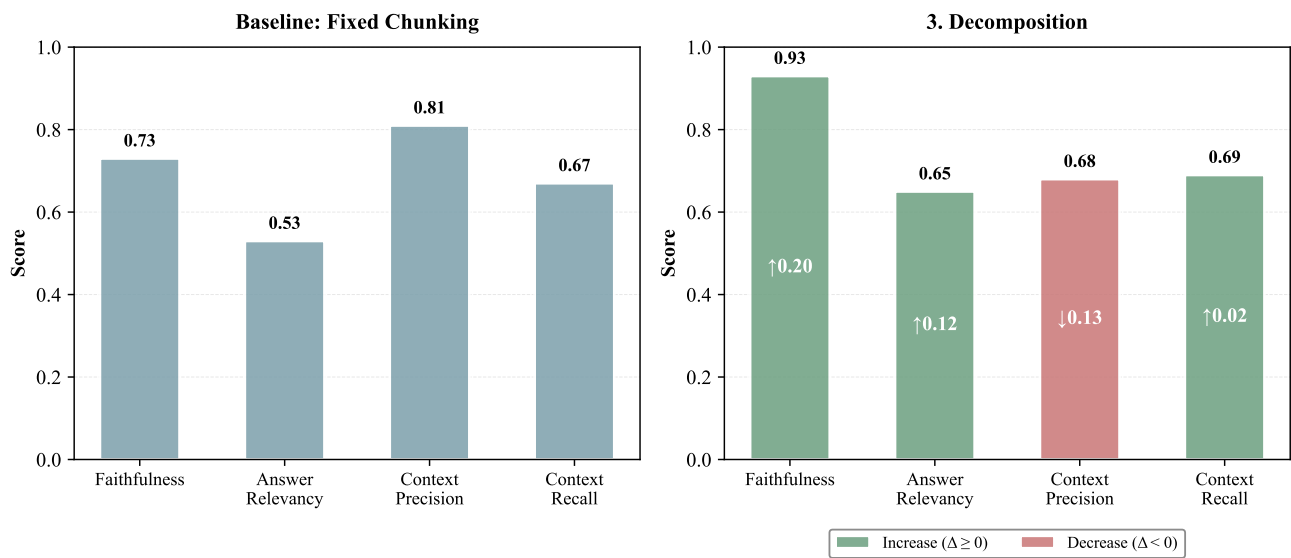
```

```

04
05 results = []
06 for data, answer in zip(retrieved_data, answers):
07     results.append({
08         "answer": answer,
09         "contexts": data["contexts"],
10         "sub_questions": data["sub_questions"]
11     })
12
13 return results

```

Kết quả thực nghiệm với Decompostion



Hình 16: Kết quả đánh giá RAGAS với chiến lược Decompostion.

Chiến lược phân tách câu hỏi mang lại kết quả ấn tượng nhất về chất lượng nội dung sinh ra trong toàn bộ chuỗi thực nghiệm, như kết quả ở Hình 16:

- **Đột phá về độ trung thực:** Faithfulness tăng mạnh 0.20 điểm, đạt mức 0.93. Đây là mức điểm cao nhất đạt được, cho thấy việc chia nhỏ câu hỏi giúp mô hình tập trung vào các mảnh thông tin xác thực một cách tối ưu.
- **Cải thiện độ liên quan:** Answer Relevancy tăng lên 0.65 (tăng 0.12 điểm). Việc trả lời dựa trên các câu hỏi phụ giúp cấu trúc nội dung đầy đủ và logic hơn.
- **Thách thức về lọc nhiễu:** Context Precision giảm 0.13 điểm xuống còn 0.68.
- **Nguyên nhân:** Do hệ thống phải truy xuất cho nhiều câu hỏi con đồng thời, lượng văn bản đổ về quá lớn khiến mật độ thông tin hữu ích bị loãng bởi các đoạn văn không liên quan trực tiếp đến câu hỏi chính.

VII.5.4. Thực nghiệm 4: Re-ranking

1. **Cross-Encoder.** Đầu tiên cần cài đặt thêm các thư viện phục vụ cho Cross-Encoder, bao gồm transformers, torch và accelerate để tải và chạy mô hình reranker ở dạng causal LM.

4.1 Cài đặt thư viện

```
1 !pip install -q \
2 # ... như baseline và thêm:
3 "transformers>=4.51.0" \
4 "torch" \
5 "accelerate"
```

Sau đó bổ sung các import cần thiết cho Cross-Encoder, bao gồm torch và hai lớp AutoModelForCausalLM, AutoTokenizer từ HuggingFace Transformers.

4.1 Khai báo thư viện

```
1 # ... như baseline và thêm:
2 import torch
3 from transformers import AutoModelForCausalLM, AutoTokenizer
```

Các tham số bên dưới cấu hình tên mô hình reranker, số lượng tài liệu được truy xuất ban đầu và số lượng tài liệu được giữ lại sau khi rerank. Việc dùng RETRIEVE_K lớn hơn RERANK_TOP_K cho phép retriever lấy nhiều ứng viên, sau đó Cross-Encoder chọn lại các đoạn liên quan nhất.

4.1 Cấu hình tham số và mô hình

```
1 # ... như baseline và thêm:
2 RERANKER_MODEL = "Qwen/Qwen3-Reranker-0.6B"
3 RETRIEVE_K = 10 # số lượng doc được query trước khi rerank
4 RERANK_TOP_K = 3 # số lượng doc được giữ lại sau khi rerank
```

Hàm get_retriever được cập nhật để luôn truy xuất RETRIEVE_K tài liệu trước khi đưa qua Cross-Encoder. Điều này giúp reranker có đủ ứng viên để đánh giá lại mức độ phù hợp dựa trên mô hình ngôn ngữ.

4.3 Xây dựng Vector Database

```
1 # ... (như base line) và cập nhật hàm bên dưới:
2 def get_retriever(self, search_kwargs: dict = None):
3     if search_kwargs is None:
4         search_kwargs = {"k": RETRIEVE_K} # thêm số lượng doc khi query
5
6     return self.db.as_retriever(
7         search_type="similarity",
8         search_kwargs=search_kwargs,
9     )
```

Lớp CrossEncoderReranker hiện thực chiến lược reranking bằng mô hình Qwen3-Reranker. Mỗi cặp truy vấn là tài liệu được mã hóa dưới dạng đoạn hội thoại ngắn, mô hình sinh ra phân phối xác suất yes/no thể hiện mức độ liên quan, sau đó các tài liệu được sắp xếp lại theo điểm này và cắt xuống top_k.

4.4 Xây dựng luồng xử lý cho RAG

```

1 class FocusedAnswerParser(StrOutputParser):
2     # ... như baseline
3
4 class CrossEncoderReranker:
5     def __init__(self, model_name: str = RERANKER_MODEL, top_k: int = RERANK_TOP_K):
6         self.top_k = top_k
7         self.model_name = model_name
8
9         print(f"Loading Cross-Encoder Reranker: {model_name}...")
10        self.tokenizer = AutoTokenizer.from_pretrained(model_name, padding_side="left")
11        self.model = AutoModelForCausalLM.from_pretrained(
12            model_name,
13            torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32
14        ).eval()
15
16        if torch.cuda.is_available():
17            self.model = self.model.cuda()
18            print(f"Reranker loaded on GPU")
19        else:
20            print(f"Reranker loaded on CPU")
21
22        self.token_false_id = self.tokenizer.convert_tokens_to_ids("no")
23        self.token_true_id = self.tokenizer.convert_tokens_to_ids("yes")
24        self.max_length = 8192
25
26        self.prefix = "<|im_start|>system\nJudge whether the Document meets the
                                requirements based on the Query and the
                                Instruct provided. Note that the answer can
                                only be \"yes\" or \"no\".<|im_end|>\n<|
                                im_start|>user\n"
27        self.suffix = "<|im_end|>\n<|im_start|>assistant\n<think>\n\n</think>\n\n"
28        self.prefix_tokens = self.tokenizer.encode(self.prefix, add_special_tokens=
                                False)
29        self.suffix_tokens = self.tokenizer.encode(self.suffix, add_special_tokens=
                                False)
30
31    def _format_instruction(self, query: str, doc: str) -> str:
32        instruction = "Given a web search query, retrieve relevant passages that answer
                                the query"
33        return f"<Instruct>: {instruction}\n<Query>: {query}\n<Document>: {doc}"
34
35    def _process_inputs(self, pairs: List[str]):
36        inputs = self.tokenizer(
37            pairs,
38            padding=False,
39            truncation="longest_first",

```

```

40         return_attention_mask=False,
41         max_length=self.max_length - len(self.prefix_tokens) - len(self.
                                     suffix_tokens)
42     )
43     for i, ele in enumerate(inputs["input_ids"]):
44         inputs["input_ids"][i] = self.prefix_tokens + ele + self.suffix_tokens
45     inputs = self.tokenizer.pad(inputs, padding=True, return_tensors="pt",
                                   max_length=self.max_length)
46     for key in inputs:
47         inputs[key] = inputs[key].to(self.model.device)
48     return inputs
49
50 @torch.no_grad()
51 def _compute_scores(self, inputs) -> List[float]:
52     batch_scores = self.model(**inputs).logits[:, -1, :]
53     true_vector = batch_scores[:, self.token_true_id]
54     false_vector = batch_scores[:, self.token_false_id]
55     batch_scores = torch.stack([false_vector, true_vector], dim=1)
56     batch_scores = torch.nn.functional.log_softmax(batch_scores, dim=1)
57     scores = batch_scores[:, 1].exp().tolist()
58     return scores
59
60 def rerank(self, query: str, documents: List) -> List:
61     if len(documents) == 0:
62         return documents
63
64     pairs = [self._format_instruction(query, doc.page_content) for doc in documents
65             ]
66
67     inputs = self._process_inputs(pairs)
68     scores = self._compute_scores(inputs)
69
70     doc_score_pairs = list(zip(documents, scores))
71     doc_score_pairs.sort(key=lambda x: x[1], reverse=True)
72
73     return [doc for doc, score in doc_score_pairs[:self.top_k]]
74
75 class BatchRAG:
76     def __init__(self, llm, reranker=None, batch_size: int = BATCH_SIZE):
77         self.llm = llm
78         self.reranker = reranker # thêm Cross-Encoder Reranker
79         self.batch_size = batch_size
80         self.prompt_template = """Bạn là trợ lý AI phân tích tài liệu tiếng Việt.
81
82 [TÀI LIỆU]:
83 {context}
84
85 [CÂU HỎI]:
86 {question}
87
88 Hãy trả lời dựa trên tài liệu. Nếu tài liệu không có thông tin, nói rõ "Không có thông
                                     tin".

```

```

89 [TRẢ LỜI]: """
90     self.prompt = PromptTemplate.from_template(self.prompt_template)
91     self.answer_parser = FocusedAnswerParser()
92
93     def _format_docs(self, docs):
94         #... (như baseline)
95
96     def get_chain(self, retriever):
97         #... (như baseline)
98
99     def batch_retrieve(self, questions: List[str], retriever) -> List[dict]:
100         all_contexts = []
101         for question in tqdm(questions, desc="Retrieving and reranking documents"):
102             docs = retriever.invoke(question)
103
104             if self.reranker is not None: # áp dụng Reranking
105                 docs = self.reranker.rerank(question, docs)
106
107             contexts = [(doc.page_content or "") for doc in docs]
108             formatted_context = self._format_docs(contexts)
109             all_contexts.append({
110                 "question": question,
111                 "contexts": contexts,
112                 "formatted_context": formatted_context
113             })
114         return all_contexts
115
116     def batch_generate(self, prompts: List[str]) -> List[str]:
117         #... (như baseline)
118
119     def answer_with_contexts_batch(self, questions: List[str], retriever):
120         #... (như baseline)

```

Cuối cùng, RAG được khởi tạo lại để gắn thêm `CrossEncoderReranker`. Retriever vẫn sử dụng vector database như trước, nhưng kết quả trả về sẽ được reranker sắp xếp lại trước khi đưa vào LLM sinh câu trả lời.

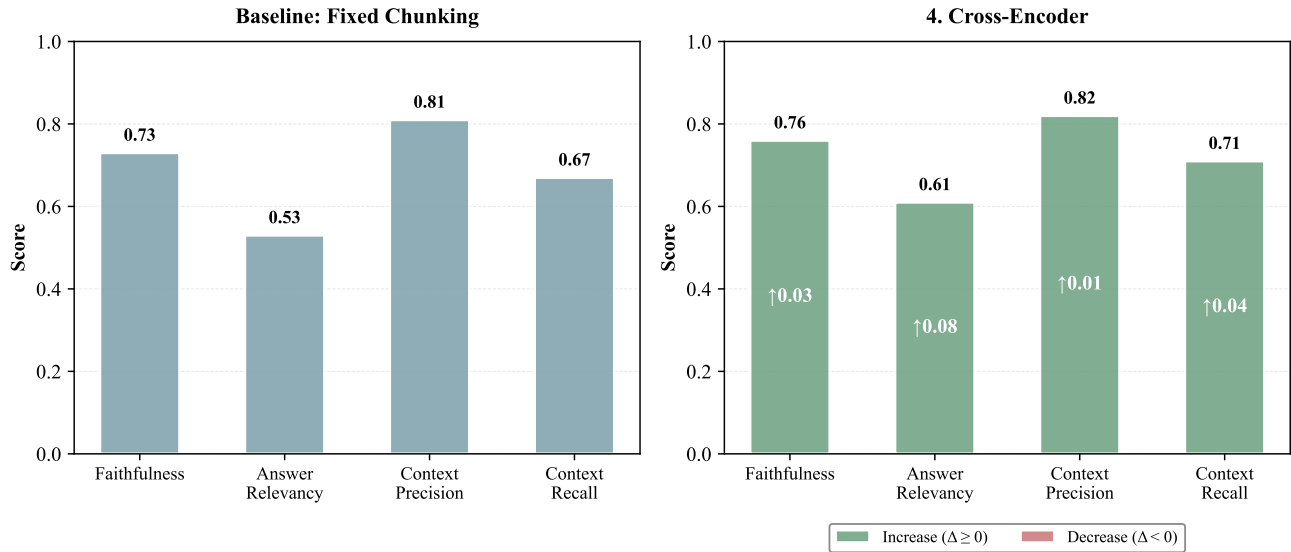
4.5 Khởi tạo RAG

```

1 # ... như baseline và thêm:
2 reranker = CrossEncoderReranker(model_name=RERANKER_MODEL, top_k=RERANK_TOP_K)
3
4 # cập nhật hàm khởi tạo RAG với reranker:
5 rag = BatchRAG(llm, reranker=reranker, batch_size=BATCH_SIZE)
6 rag_chain = rag.get_chain(retriever)

```

Kết quả thực nghiệm với Cross-Encoder



Hình 17: Kết quả đánh giá RAGAS với chiến lược CrossEncoder.

Thực nghiệm Re-ranking cho thấy sự cải thiện đồng đều ở tất cả các chỉ số so với mô hình baseline, như ở Hình 17:

- **Tăng cường chất lượng sinh:** Faithfulness tăng lên 0.76 và Answer Relevancy đạt 0.61 (tăng 0.08 điểm).
- **Nâng cao độ bao phủ:** Context Recall tăng 0.04 điểm, đạt mức 0.71.
- **Nguyên nhân:** Khác với Bi-Encoder chỉ bắt từ khóa, mô hình Cross-Encoder thực hiện Self-Attention toàn phần giữa câu hỏi và tài liệu. Điều này giúp hệ thống hiểu được các sắc thái phức tạp, từ đó xếp hạng cao các tài liệu thực sự có giá trị về mặt logic thay vì chỉ dựa trên độ tương đồng vector.

2. Maximal Marginal Relevance (MMR). Với chiến lược MMR, thay vì chỉ tối ưu độ tương đồng giữa truy vấn và tài liệu, retriever cân bằng giữa hai yếu tố, bao gồm mức độ liên quan với truy vấn và tính đa dạng giữa các tài liệu được chọn. Ba tham số bên dưới điều khiển số lượng tài liệu cuối cùng, số lượng tài liệu lấy ban đầu và hệ số cân bằng lambda.

4.1 Cấu hình tham số và mô hình

```

1 # ... như baseline và thêm các tham số cho MMR:
2 MMR_K = 3 # số lượng doc cuối cùng
3 MMR_FETCH_K = 10 # số lượng doc được lấy trước khi MMR
4 MMR_LAMBDA_MULT = 0.5 # tham số Lambda từ 0.0 tới 1.0, thường là 0.5 để cân bằng

```

Hàm `get_retriever` được cập nhật để sử dụng `search_type` là `mmr` thay vì `similarity`. Retriever trước hết lấy `MMR_FETCH_K` tài liệu ứng viên, sau đó áp dụng Maximal Marginal Relevance để chọn ra `MMR_K` tài liệu vừa liên quan, vừa không trùng lặp quá nhiều về nội dung.

4.3 Xây dựng Vector Database

```

1 def get_retriever(self, search_kwargs: dict = None):
2     if search_kwargs is None:
3         search_kwargs = {
4             "k": MMR_K,
5             "fetch_k": MMR_FETCH_K,
6             "lambda_mult": MMR_LAMBDA_MULT
7         }
8
9     return self.db.as_retriever(
10         search_type="mmr", # Đổi từ "similarity" thành "mmr"
11         search_kwargs=search_kwargs,
12     )

```

Khi khởi tạo retriever cho RAG, chỉ cần truyền lại đúng bộ tham số MMR đã cấu hình. Phần còn lại của pipeline giữ nguyên, nên có thể so sánh trực tiếp hiệu quả giữa MMR, Cross-Encoder và baseline similarity.

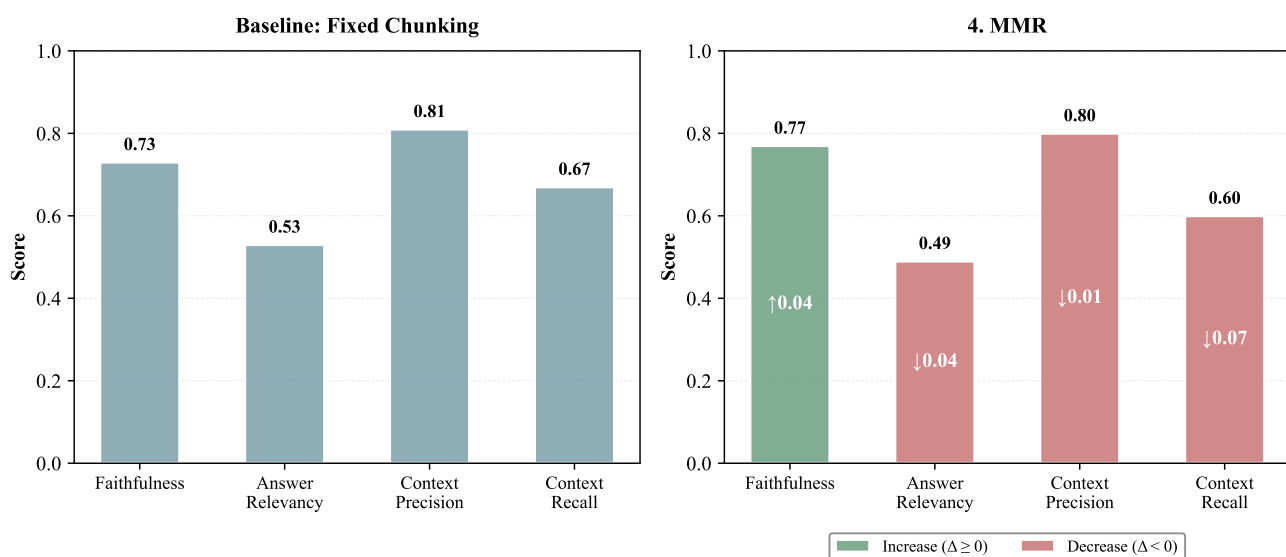
4.5 Khởi tạo RAG

```

1 # ... như baseline và cập nhật hàm khởi tạo retriever:
2 retriever = vdb.get_retriever(search_kwargs={
3     "k": MMR_K,
4     "fetch_k": MMR_FETCH_K,
5     "lambda_mult": MMR_LAMBDA_MULT
6 })

```

Kết quả thực nghiệm với MMR



Hình 18: Kết quả đánh giá RAGAS với chiến lược MMR.

Chiến lược tối ưu hóa sự đa dạng mang lại kết quả đặc thù cho bài toán tra cứu tổng quan, như ở Hình 18:

- **Duy trì tính xác thực:** Faithfulness tăng nhẹ 0.04 điểm lên mức 0.77.
- **Sự sụt giảm về độ bao phủ:** Context Recall giảm mạnh 0.07 điểm xuống mức 0.60. Đồng thời Answer Relevancy cũng giảm xuống 0.49.
- **Nguyên nhân:** MMR trừng phạt các tài liệu có nội dung tương đồng để tránh lặp lại. Trong các tài liệu học thuật chuyên sâu, nhiều chi tiết quan trọng có thể nằm trong các đoạn văn có ngữ nghĩa gần giống nhau. Việc ưu tiên sự đa dạng quá mức vô tình loại bỏ các thông tin bổ trợ cần thiết, khiến mô hình thiếu dữ liệu để trả lời đầy đủ ý.

VII.6. Tổng kết thực nghiệm

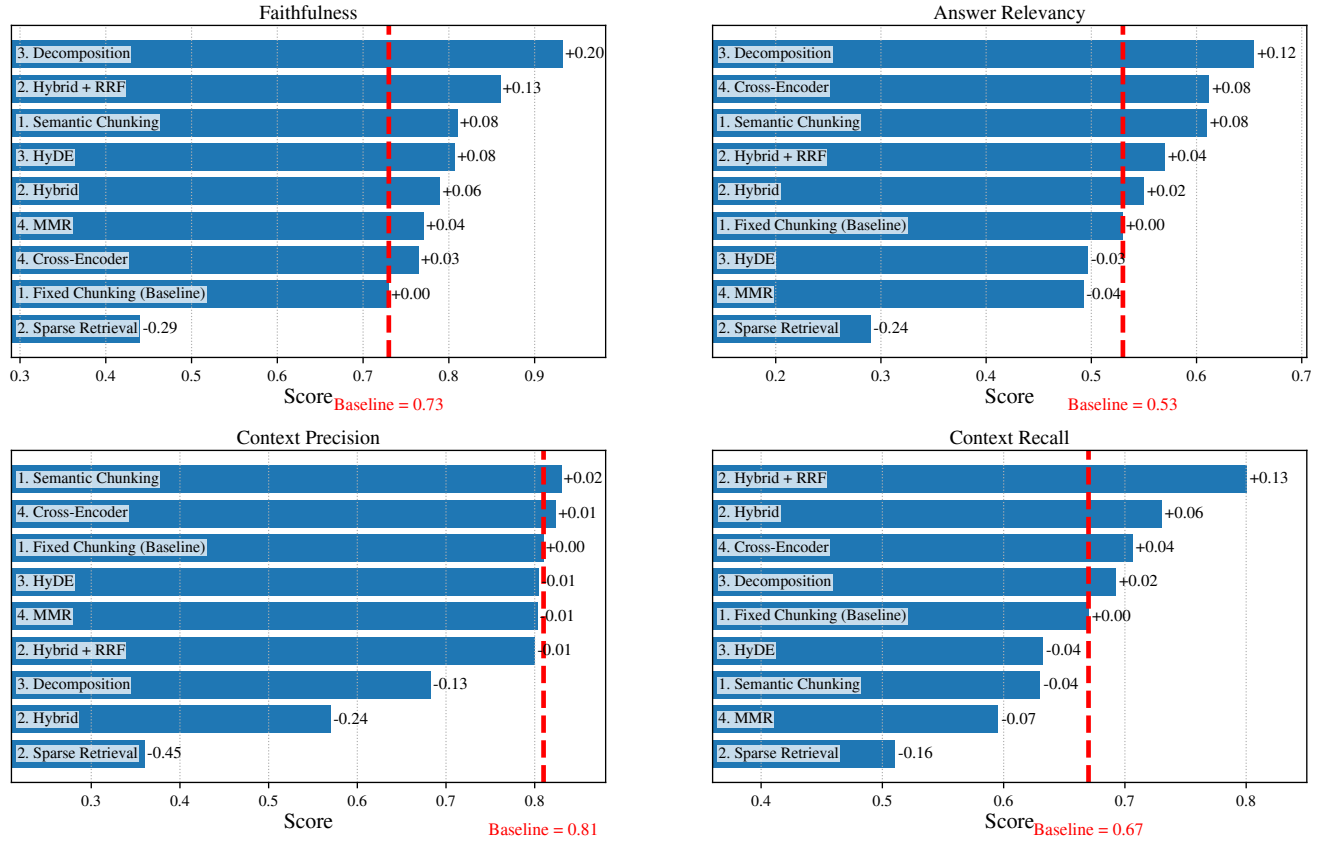
Bảng 3: Kết quả RAGAS và độ chênh so với Baseline (Δ).

Thực nghiệm	Faithfulness		Answer Relevancy		Context Precision		Context Recall	
	Score	Δ	Score	Δ	Score	Δ	Score	Δ
1. Fixed Chunking (Baseline)	0.73	0.00	0.53	0.00	0.81	0.00	0.67	0.00
1. Semantic Chunking	0.81	0.08	0.61	0.08	0.83	0.02	0.63	-0.04
2. Sparse Retrieval	0.44	-0.29	0.29	-0.24	0.36	-0.45	0.51	-0.16
2. Hybrid	0.79	0.06	0.55	0.02	0.57	-0.24	0.73	0.06
2. Hybrid + RRF	0.86	0.13	0.57	0.04	0.80	-0.01	0.80	0.13
3. HyDE	0.81	0.08	0.50	-0.03	0.80	-0.01	0.63	-0.04
3. Decomposition	0.93	0.20	0.65	0.12	0.68	-0.13	0.69	0.02
4. Cross-Encoder	0.76	0.03	0.61	0.08	0.82	0.01	0.71	0.04
4. MMR	0.77	0.04	0.49	-0.04	0.80	-0.01	0.60	-0.07

Dựa trên kết quả tổng hợp từ Bảng 3 và Hình 19, chúng ta có thể đưa ra các đánh giá tổng quát về hiệu năng của các kỹ thuật nâng cao so với hệ thống Baseline như sau:

VII.6.1. Đánh giá về chất lượng sinh văn bản

- **Độ trung thực (Faithfulness):** Chiến lược Decomposition đạt kết quả cao nhất với điểm số 0.93, tăng 0.20 điểm so với baseline. Tiếp theo là Hybrid + RRF với mức tăng 0.13 điểm. Điều này cho thấy việc chia nhỏ câu hỏi giúp mô hình tập trung vào dữ liệu xác thực tốt hơn.
- **Độ liên quan câu trả lời (Answer Relevancy):** Decomposition tiếp tục dẫn đầu với mức tăng 0.12 điểm. Các kỹ thuật như Cross-Encoder và Semantic Chunking cũng cho thấy sự cải thiện đồng đều khi cùng tăng 0.08 điểm so với baseline. Ngược lại, Sparse Retrieval làm giảm chỉ số này nghiêm trọng nhất, giảm 0.24 điểm.



Hình 19: Tổng hợp chênh lệch giữa các thực nghiệm so với baseline.

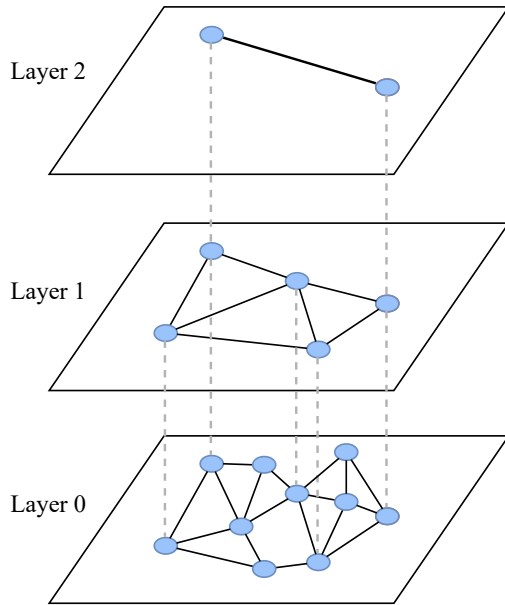
VII.6.2. Đánh giá về hiệu năng truy xuất (Retrieval)

- **Độ chính xác ngữ cảnh (Context Precision):** Phần lớn các kỹ thuật nâng cao đều gây ra sự sụt giảm ở chỉ số này do lượng dữ liệu lấy về nhiều hơn làm loãng thông tin. Semantic Chunking là kỹ thuật duy nhất giúp tăng nhẹ 0.02 điểm, trong khi Decomposition và Hybrid Search làm giảm lần lượt 0.13 và 0.24 điểm.
- **Độ bao phủ ngữ cảnh (Context Recall):** Hybrid + RRF cho thấy hiệu quả vượt trội trong việc tìm kiếm đầy đủ thông tin với mức tăng 0.13 điểm. Kỹ thuật Cross-Encoder cũng giúp cải thiện 0.04 điểm. Tuy nhiên, Sparse Retrieval thất bại trong việc bao phủ thông tin ngữ nghĩa khi làm giảm 0.16 điểm so với baseline.

VII.6.3. Kết luận chung

- **Lựa chọn chiến lược:** Hybrid Search kết hợp RRF là giải pháp cân bằng hiệu năng toàn diện nhất, trong khi Decomposition là lựa chọn hàng đầu cho độ chính xác tuyệt đối dù chi phí tính toán cao hơn.
- **Tối ưu hóa thành phần:** Thực nghiệm cho thấy BM25 đơn lẻ kém hiệu quả với truy vấn phức tạp, khẳng định sự cần thiết của bước Re-ranking (Cross-Encoder) để lọc nhiễu chuyên sâu trước khi gửi dữ liệu vào LLM.

VIII. Câu hỏi trắc nghiệm



1. Trong cấu trúc đồ thị phân tầng của HNSW, vai trò chính của các layer cao là gì?
 - (a) Lưu trữ toàn bộ dữ liệu chi tiết để đảm bảo không mất mát thông tin.
 - (b) Đóng vai trò như các đường cao tốc với các liên kết thưa, giúp thuật toán nhảy nhanh đến vùng dữ liệu tiềm năng trước khi tìm kiếm chi tiết.
 - (c) Thực hiện tính toán chính xác khoảng cách Cosine giữa tất cả các cặp điểm.
 - (d) Lưu trữ metadata của các vector để phục vụ việc filtering.

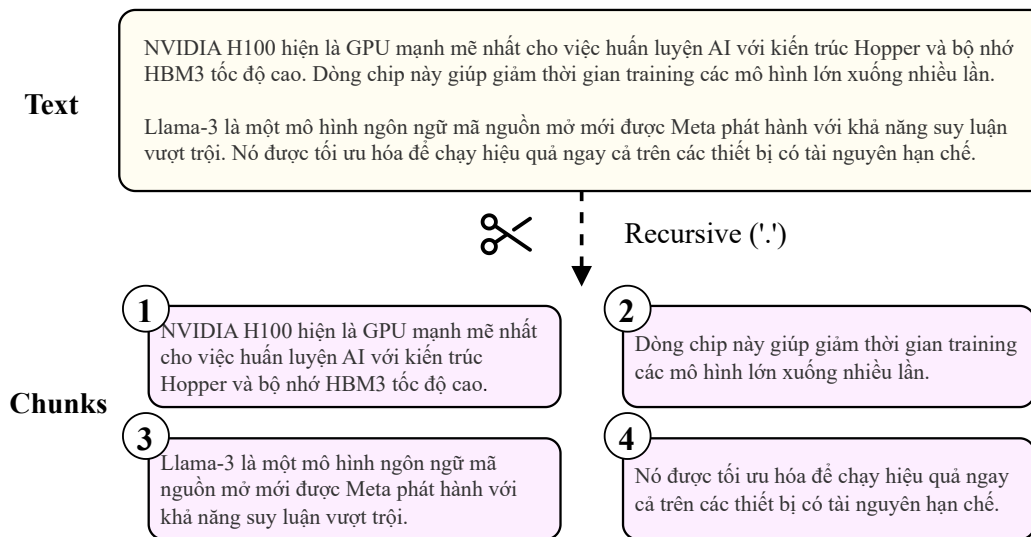
Hình 20: Minh họa các layer trong HNSW.

2. Giả sử bạn đang vận hành một hệ thống RAG sử dụng HNSW Index cho ứng dụng Chatbot hỗ trợ khách hàng thời gian thực với yêu cầu độ trễ thấp. Tuy nhiên, hệ thống hiện tại đang phản hồi quá chậm. Dựa trên cơ chế của HNSW, thay đổi nào sau đây là hợp lý nhất để cải thiện tốc độ mà ít ảnh hưởng đến chất lượng nhất?
 - (a) Tăng tham số `M` để đồ thị dày đặc hơn, giúp tìm đường nhanh hơn.
 - (b) Tăng tham số `ef_construction` để xây dựng đồ thị kỹ lưỡng hơn ngay từ đầu.
 - (c) Giảm tham số `ef_search` để thu hẹp phạm vi quét các láng giềng ứng viên tại thời điểm truy vấn.
 - (d) Chuyển toàn bộ dữ liệu từ Layer 0 lên Layer cao nhất để giảm số bước nhảy.
3. Một hệ thống RAG pháp lý cần tìm kiếm thông tin về “Điều 244 Bộ luật Hình sự”.
 - Kết quả Vector Search trả về các văn bản mô tả chung về “tội phạm kinh tế” nhưng không có số hiệu điều luật cụ thể, Top 1 có score = 0.88.
 - Kết quả BM25 trả về chính xác văn bản chứa chuỗi “Điều 244”, Top 1 có score = 15.5.

Tại sao thuật toán RRF lại là giải pháp phù hợp để xếp hạng trường hợp này thay vì dùng Weighted Sum?

- (a) Vì Weighted Sum sẽ yêu cầu chuẩn hóa hai thang điểm về cùng miền giá trị $[0, 1]$, quá trình này rất nhạy cảm với các giá trị ngoại lai của BM25, dẫn đến kết quả không ổn định.
- (b) Vì BM25 không hỗ trợ tính toán trên GPU nên không thể thực hiện phép cộng ma trận với Vector Search.

- (c) Vì RRF ưu tiên tuyệt đối cho kết quả từ BM25 bất kể thứ hạng của Vector Search như thế nào.
- (d) Vì Vector Search luôn trả về kết quả sai khi gặp các con số, nên RRF sẽ tự động loại bỏ danh sách từ Vector Search.
4. Hãy quan sát chunk thứ 4 trong Hình 21, nếu hệ thống Retrieval chỉ tìm thấy chunk này và đưa vào context cho LLM, hiện tượng nào sau đây sẽ xảy ra?



Hình 21: Ví dụ về Recursive Chunking bằng dấu chấm.

- (a) LLM sẽ không xác định được đại từ “Nó” đang ám chỉ một đối tượng nào.
- (b) Chunk này quá ngắn nên sẽ gây lỗi khi padding vector, khiến LLM từ chối trả lời.
- (c) LLM sẽ tự động hiểu “Nó” là NVIDIA H100 vì H100 được nhắc đến ở đầu văn bản gốc.
- (d) LLM hiện đại có khả năng suy luận ngược để tìm lại văn bản gốc trong dữ liệu huấn luyện của nó.
5. Trong trường hợp người dùng đặt câu hỏi so sánh phức tạp: “So sánh doanh thu của iPhone 15 và Samsung S24 trong quý 1 năm 2024”, tại sao phương pháp Query Decomposition lại hiệu quả hơn việc tìm kiếm trực tiếp?
- (a) Vì nó giúp tách câu hỏi thành hai luồng tìm kiếm riêng biệt vào hai tài liệu báo cáo tài chính khác nhau, tránh việc vector câu hỏi bị nằm lửng lơ giữa hai chủ đề.
- (b) Vì nó giúp LLM tự động tính toán con số chênh lệch doanh thu trước khi tìm kiếm tài liệu.
- (c) Vì phương pháp này sử dụng BM25 để tìm chính xác từ khóa “so sánh” trong cơ sở dữ liệu.
- (d) Vì nó gộp chung báo cáo của Apple và Samsung thành một văn bản duy nhất để dễ xử lý.
6. Trong pipeline tạo bộ dữ liệu đánh giá bằng Ragas, vai trò chính của `TestsetGenerator.generate_with_langchain_docs()` là gì?

- (a) Sinh embedding cho toàn bộ corpus và lưu trực tiếp vào vector database.
 - (b) Xây dựng knowledge graph từ các document, sau đó sinh câu hỏi và reference answer theo các kiểu truy vấn khác nhau.
 - (c) Chỉ dùng để chuyển đổi tài liệu PDF sang markdown.
 - (d) Đánh giá trực tiếp hệ thống RAG bằng các metric Faithfulness và Context Recall.
7. Trong thuật toán MMR, nếu ta giảm tham số λ xuống thấp, ví dụ: 0.2, kết quả trả về cho câu truy vấn “Tính năng xe VinFast VF8” sẽ thay đổi như thế nào?
- (a) Hệ thống sẽ trả về các tài liệu hoàn toàn không liên quan đến VinFast VF8.
 - (b) Hệ thống sẽ ưu tiên trả về các tài liệu có nội dung rất giống nhau để khẳng định độ chính xác.
 - (c) Hệ thống sẽ ưu tiên sự đa dạng, trả về các khía cạnh khác nhau thay vì lặp lại thông tin về động cơ.
 - (d) Hệ thống sẽ chỉ trả về đúng 1 tài liệu duy nhất có điểm số cao nhất.
8. Hãy xem xét đoạn code logic phân chia văn bản trong class `SemanticChunker`:

```

1 similarity = self._calculate_cosine_similarity(prev_emb, curr_emb)
2
3 if similarity >= self.breakpoint_threshold:
4     current_chunk.append(sentences[i])
5 else:
6     chunks.append(chunk_text)
7     current_chunk = [sentences[i]]
8

```

Giả sử bạn thay đổi tham số `breakpoint_threshold` từ mặc định 0.5 lên mức rất cao là 0.95. Hiện tượng nào sẽ xảy ra với dữ liệu sau khi chunking?

- (a) Các chunk sẽ trở nên rất dài vì thuật toán gộp hầu hết các câu lại với nhau.
 - (b) Văn bản sẽ bị cắt vụn thành rất nhiều chunk nhỏ, vì rất khó để hai câu liên tiếp đạt độ giống nhau tới 95%.
 - (c) Thuật toán sẽ bỏ qua ngữ nghĩa và chỉ cắt dựa trên `MAX_CHUNK_SIZE`.
 - (d) Hệ thống sẽ báo lỗi vì threshold không được vượt quá 0.5.
9. Đây là đoạn code cài đặt thuật toán Reciprocal Rank Fusion (RRF) trong class `HybridRetriever`:

```

1 def _reciprocal_rank_fusion(self, results_list, k):
2     rrf_scores = defaultdict(float)
3
4     for results in results_list:
5         for rank, doc in enumerate(results, start=1):
6             rrf_scores[doc.page_content] += 1 / (k + rank)
7
8     return sorted_docs

```

9

Dòng tính điểm $1 / (k + \text{rank})$, vai trò của hằng số k (thường thiết lập là 60) là gì?

- (a) Để giới hạn chỉ lấy top k tài liệu tốt nhất trả về cho người dùng.
- (b) Để chuyển đổi thứ hạng từ thang đo logarit sang thang đo tuyến tính.
- (c) Để “làm mượt” điểm số, ngăn không cho các tài liệu ở Top 1 có điểm số quá áp đảo so với các tài liệu ở hạng 2, 3.
- (d) Để đảm bảo mẫu số không bao giờ bằng 0 trong trường hợp $\text{rank} = 0$.

10. Hãy xem xét phương thức `decompose_and_retrieve` trong class `DecompositionBatchRAG`:

```

1 for sub_q in sub_questions:
2     docs = retriever.invoke(sub_q)
3     # ...code here...
4
5 original_docs = retriever.invoke(question)
6 for doc in original_docs:
7     # ...code here...
8

```

Tại sao code lại thực hiện retrieval cho cả `sub_questions` lẫn `question` thay vì chỉ tìm kiếm cho các câu hỏi con?

- (a) Để đảm bảo tính dự phòng, nếu việc phân tách câu hỏi thất bại, hệ thống vẫn còn kết quả từ câu hỏi gốc.
- (b) Để tối ưu hóa tốc độ, hỗ trợ tìm kiếm câu hỏi gốc nhanh hơn tìm kiếm nhiều câu hỏi con.
- (c) Để đảm bảo tính toàn diện, câu hỏi con giúp đi sâu vào chi tiết cụ thể, trong khi câu hỏi gốc giúp lấy được các ngữ cảnh tổng quan mà việc phân tách có thể đã vô tình bỏ sót.
- (d) Để so sánh đối chiếu, hệ thống sẽ tự động chọn kết quả nào tốt hơn giữa gốc và phân tách để trả về.

Phụ lục

1. **Datasets:** Các file dataset được đề cập trong bài có thể được tải tại [đây](#).
2. **Solution:** Các file code cài đặt thực nghiệm có thể được tải tại [đây](#).
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,6K thành viên



Hình 22: Hình ảnh group facebook AIO Q&A.